Trust the Process, Not the Prompt

How to Turn AI Output Into Trusted Work

Benjamin Luchterhand

How To Use This Book

AI often enters real work as a useful shortcut: a clean summary of a messy thread, a draft recommendation for a customer exception, or a policy answer that gives a team somewhere to start. The problem begins when that draft quietly starts doing the work of a decision before anyone has checked the source, the permission, the reviewer, or the person allowed to accept it.

Use this book if you lead or design workflows where AI output is starting to shape customer commitments, operational records, product changes, legal filings, financial decisions, or internal approvals. Its central distinction is simple: a model can produce a proposal, but only a workflow can decide when that proposal has enough evidence and accountability to become accepted work.

Read the chapters in order the first time. The argument builds one operating model: AI outputs begin as proposals, and a workflow promotes them into accepted work only through evidence, freshness, permissions, review capacity, accountable acceptance, and learning.

After that, use the appendices as working tools. Start with one workflow, not a company-wide AI program. Name the decision, the source that controls the most important claim, what the AI output may do before acceptance, the reviewer capacity, the stop condition, and the owner after acceptance. For example, can it only draft an internal recommendation, or can it update a record, send a message, route a case, apply a credit, or trigger another tool?

The running support-credit workflow is a composite design scenario unless a chapter explicitly names a real case. It is meant to make the controls visible, not to establish that every support organization works the same way.

General contact-center and support-center standards support only the background operating discipline behind the example, such as documented processes, service requirements, quality monitoring, and performance indicators. They do not establish the composite service-credit scenario or its assumed numbers (International Organization for Standardization 2017; HDI 2018).

Reader Promise

By the end, you should be able to design one AI workflow your organization can trust, measure, and scale without asking the model to provide evidence, authority, or accountability by itself.

A Note On Sources

The book uses author-date citations. Real cases are included to make a control visible, not to claim that every domain works like the case domain. When a case comes from a company or vendor source, the chapter treats it as a reported pattern rather than independent proof of business outcomes.

AI Use In The Manuscript

AI tools were used as drafting and review aids. Final claims, source choices, structure, and publication judgment remain author responsibilities. This matches the book's own rule: probabilistic systems can help produce proposals, but evidence and acceptance belong to the workflow around them.

Introduction: Trust Belongs To The Workflow

Most teams do not need another reminder that AI can be wrong.

The harder problem is that useful AI output arrives in the middle of real work. A customer is waiting. A ticket is open. A sales team wants language before a renewal call. A reviewer has two hours left in the week. A policy source has moved, but the old knowledge-base page still ranks first. A model produces a clean recommendation that reduces ambiguity for everyone in the thread.

That recommendation may be useful. It may also be unsupported, stale, unauthorized, or incomplete. The danger is not only the model's error. The danger is the workflow treating the output as if it has already earned the right to affect a customer, record, payment, legal filing, codebase, medical judgment, or operational decision.

This book uses one operating rule:

```
AI output starts as a proposal. A workflow accepts it for use only after
required evidence is checked and a named owner accepts the decision or
authorizes the permitted effect.
```

That rule is not anti-AI. It is the condition that lets organizations use AI more confidently. A model can summarize, synthesize, draft, compare, translate, search, reason through ambiguity, and prepare candidate work. Those are real capabilities. They become valuable in production when the surrounding workflow knows what the output may do before acceptance, which evidence supports each claim, who can accept the decision, how review capacity is funded, and what happens when evidence is missing.

The Book's Practical Question

The wrong first question is usually:

```
Can we trust this model?
```

A better question is:

```
What process would make this model's useful work safe to accept here?
```

The answer depends on consequence. A private note summary does not need the same control as a customer-credit recommendation. A draft assistance workflow does not need the same boundary as an agent that

updates records or triggers payments. A low-risk writing aid can leave final judgment with the user. A reviewed proposal needs evidence, freshness, reviewer capacity, acceptance criteria, and audit. A bounded action needs explicit permissions, post-action verification, rollback or correction paths, incident handling, and a learning loop.

The book's promise is narrow on purpose: design one AI workflow your organization can trust, measure, and scale. One workflow is enough to expose the real work. You will find stale sources, contested authority, overloaded reviewers, unclear rejection authority, private-tool bypass, executive impatience, and metrics that count generated output as if it were accepted value. Those findings are not side issues. They are the adoption work.

What This Book Is Not

This is not a prompt cookbook. Prompt quality matters, but better prompts are not decision controls by themselves.

This is not a model-selection guide. Better models matter, but stronger capability moves the trust boundary rather than removing it.

This is not a generic governance manifesto. Governance matters only when it changes what the workflow can access, accept, escalate, log, throttle, stop, or retire.

This is not an argument that AI is too unreliable to use. It is an argument that probabilistic systems should do the work they are good at while workflows own evidence, permissions, verification, acceptance, and learning.

How The Argument Builds

The opening teardown shows a useful AI-generated proposal escaping into organizational reliance before anyone has confirmed the evidence, authority, or review capacity behind it.

Chapters 1 through 3 name the trust mismatch. AI can produce useful judgment-like work without granting itself authority. Important human work did not earn trust raw either. Prompts, retrieval, review labels, policies, and eval dashboards can improve the work object, but they do not control the decision unless the workflow changes what can be accepted.

Chapters 4 through 6 put AI where it is strong. They define task fit, claim-level evidence, source freshness, responsibility, and incentives.

Chapters 7 through 9 turn that shape into enforceable workflow. They define effect boundaries, measure accepted verified output, and make human review a real control.

Chapters 10 and 11 operate the workflow under pressure. They distinguish assistants from bounded agents and make evals, incidents, and governance change the workflow instead of only documenting it.

Chapter 12 assembles the pieces into one three-week pilot. Chapter 13 closes on the discipline that keeps the system honest: grant authority at the pace the workflow can support.

The appendices are working tools. Use them to design your first pilot, not to create a binder of policies. If a field cannot block, route, require, log, throttle, escalate, or stop work, it is not yet a control.

Opening Teardown: The Fluent Output That Escaped

A fluent output escapes when it starts doing the work of an accepted decision while the workflow still labels it a draft. The composite support-credit scenario used throughout this book begins with a genuinely useful AI-generated proposal. It organizes a messy thread, proposes a plausible credit, and gives the account team language. The failure begins when people treat that proposal as the current plan before anyone has checked the evidence, confirmed who can accept the decision, or preserved the reviewer's rejection authority.

A strategic customer had asked for a service credit after an outage. The support manager had a renewal call on Monday, an account team pressing for language by the end of the day, and a ticket thread that had already collected three different versions of the story. The customer wanted to know whether the company would make the incident right. Sales wanted a fast signal. Finance wanted to avoid another informal exception. Compliance had been warning for months that credits handled through side conversations were becoming hard to defend.

The support manager asked the AI assistant for a proposed path. The answer had real utility. It summarized the ticket clearly. It identified the customer's business impact. It found a policy family that seemed relevant. It proposed a discretionary credit and drafted language the account team could use:

```
Based on the incident severity and the customer's strategic importance,
approve
a discretionary service credit and communicate that the company recognizes
the
impact.
```

The draft solved an immediate work problem. It gave the manager a coherent position where there had been a scattered thread. It used the vocabulary the company used for severe incidents. It sounded measured rather than defensive.

The answer then moved through ordinary work channels.

The support manager pasted a shortened version into Slack with a careful note: "AI draft, likely path, checking finance." The phrase "likely path" did more work than intended. A sales lead replied, "Great, we'll tell

the customer we are working toward credit approval." A ticket note captured the recommendation as the current plan. An account update said the team expected to approve a credit pending final review. Finance entered the thread after the expectation had already formed.

Several checks were still missing: the current approved policy, the customer's contractual credit cap, prior credits in the billing ledger, customer-specific impact evidence from the outage timeline, and compliance review against recent exception decisions.

The AI produced useful work, but the workflow had no durable state between generated text and organizational reliance.

> ***Real case: Air Canada chatbot.*** *In 2024, the British Columbia Civil Resolution Tribunal held Air Canada liable after its website chatbot gave a customer incorrect bereavement-fare guidance. The customer relied on the answer, booked travel, and was later denied the refund path the chatbot had described.*
>
> *Air Canada's attempted fallback was that the correct policy was available elsewhere on the site. The tribunal rejected that as a defense because it did not repair the customer's reliance on the chatbot answer. For this book's workflow lesson, the missing control was a customer-facing answer path that checked chatbot guidance against current policy or routed unresolved fare questions to a human owner. The tribunal ordered compensation.*
>
> *The lesson was larger than the refund amount because the chatbot answer became the organization's represented position. The practical boundary is direct: a customer-facing AI answer is not a private draft. The workflow has to keep policy sources current, route uncertainty, and prevent unsupported guidance from becoming the organization's answer (Moffatt v. Air Canada 2024; Lifshitz and Hung 2024).*

The First Leak Is Social

Most AI adoption failures do not begin with a dramatic system takeover. They begin with ordinary work habits.

A person under pressure receives a useful draft. The draft relieves ambiguity. The person shares it to get agreement on the next step. The recipient reads a proposal as a direction. Another team records the direction in a system of record. By the time the missing evidence appears, reversing the expectation is harder than refusing the proposal would have been.

This can happen without AI. A confident analyst writes a memo from incomplete notes. A manager summarizes a hallway conversation as a decision. An account lead forwards "probably approved" language to a customer team. Mature organizations use approval limits, review packets, current source records, and separation of duties because human drafts can also become commitments before they have earned that status.

AI changes the mechanics. It makes the draft faster, more polished, and easier to replicate into every work surface connected to the team. A human might leak a proposal through one conversation. An AI-assisted workflow can leak through Slack, a ticket note, a CRM update, an email, a copied summary, and a follow-up automation before anyone has asked what the work may change, send, update, route, or trigger.

The useful question is what state the work is in, not whether the draft sounds responsible.

The Proposal Was A Bundle Of Unchecked Claims

The Friday proposal sounded like one decision. It was really a bundle of claims:

- the outage affected this customer;
- the incident met the policy threshold for a credit;
- the customer's contract allowed the proposed remedy;
- the credit amount was within finance authority;
- prior credits did not change the decision;
- compliance risk was acceptable;
- support could tell the account team approval was likely.

Different sources answer different claims. The support ticket shows what the customer asked for, not contract eligibility. The outage timeline describes the service incident, not whether this customer crossed the impact threshold. The policy defines exception criteria, but it does not override the customer contract. The billing ledger shows prior credits; finance still owns approval for a new exception. Compliance notes identify consistency risk without granting finance authority.

The AI draft compressed those distinctions into a paragraph. That compression is one of the reasons the tool was useful. It is also why the

workflow needed a separate acceptance process.

Professionals already know this pattern. A software pull request is not merged because the author is articulate. It is merged after the diff, tests, review, and release risk are acceptable for the change. An incident timeline is not accepted because the first narrative is plausible. It is checked against logs, metrics, deployment records, and witness accounts. A finance approval is not valid because an expense explanation sounds reasonable. It depends on receipts, limits, budget ownership, and approval authority.

Important work becomes trustworthy when the organization can inspect how a proposal was promoted into authority.

"A Human Reviewed It" Is Too Vague

After the Slack thread spread, the first defense was predictable: a human had reviewed the output.

That sentence did not answer the operational questions:

- Which human reviewed it?
- Which claims were they reviewing?
- Which sources could they inspect?
- How much time did they have?
- Could they say no?
- Could they stop customer communication?
- Did the workflow record what was accepted and what was still missing?

Human review becomes useful when it has shape. A reviewer who can edit tone but cannot access the contract is not reviewing eligibility. A support manager who can judge customer urgency but cannot approve the credit limit is not reviewing financial authority. A compliance reviewer added after the account team has already signaled likely approval is reviewing under political pressure created by the workflow itself.

The support manager in this composite case was not careless. The workflow placed the manager in an incoherent role: accountable for the customer response while lacking several forms of authority needed to accept the decision. Finance owned the credit limit, compliance owned policy interpretation, legal or account operations owned parts of the contract record, and incident operations owned the outage evidence. The

manager could assemble a packet and request review, but could not make those claims true by reading a fluent answer.

AI adoption often inherits an old organizational flaw. A person is made accountable for an outcome but not given source access, rejection authority, reviewer capacity, escalation cover, or permission boundaries. The AI system then produces more work for that already ambiguous role.

The phrase "human in the loop" matters only when the loop can change what the workflow accepts.

The Missing State Between Draft And Decision

The first repair is modest: attach a decision state label to the work before it can travel.

For the support-credit workflow, the label could have looked like this:

Work Item	Current State	Allowed Effect	Required Next Check
AI-generated proposal	Proposal	Internal review only	Claim-to-source check
Slack summary	Working note	No customer commitment	Owner correction if overstated
Ticket note	Draft record	No final decision	Evidence packet
Customer message	Not permitted	Blocked	Named owner accepts decision

When this table is wired into ticket status, customer-send permissions, and reviewer routing, it changes the allowable effects. The proposal helps the support manager organize the case without implying approval. The Slack summary coordinates review without creating a customer expectation. The ticket note records work in progress without becoming the final decision. Customer communication stays blocked until required sources are checked and the named owners approve the decision.

The state label also makes failure visible. If the proposal reaches the account team as likely approval, the workflow has failed even if the final answer later turns out to be acceptable. The failure is premature authority. A good outcome reached through an uncontrolled path is still evidence about the path.

This matters because many AI rollouts judge success by whether the generated answer was "right enough." That is too narrow for consequential work. The system can fail by accepting stale evidence, bypassing a required owner, overloading reviewers, hiding uncertainty, or creating political pressure before review. The wrong answer is only one failure mode.

What The Workflow Should Have Done

The improved Friday flow is not complicated.

The AI assistant can still summarize the ticket, list likely policy questions, draft an internal reviewer packet, and propose customer language. Those are valuable tasks. But the workflow should attach the output to a state:

```
Decision state: proposal.
Action limit before acceptance: internal review only.
Missing checks: current policy, customer contract cap, billing authority,
customer-specific outage impact, prior credits, compliance consistency
review.
Customer communication: not permitted to imply approval.
```

That one state changes the social path. The Slack message becomes a review request, not a likely commitment. The ticket note records that evidence is missing. Sales can see why support has not promised a credit. Finance can review the amount before the expectation hardens. Compliance can object before the case becomes politically expensive to unwind.

Controls should scale with consequence. If an employee asks for help rewriting a private note, a heavy evidence packet would be wasteful. But when a workflow can affect customers, money, records, or system state, the work needs a promotion path. It needs to say what the AI output currently is, what it may do before acceptance, what evidence is missing, who can accept it, and what happens when those conditions are not met.

This is the decision acceptance problem in miniature.

A Monday Version

The Monday action is to write a one-page decision-acceptance preflight for one workflow.

Do not start with an AI policy for the whole company. Pick one workflow where AI output could plausibly affect a customer, financial exposure, an operational record, or a system action. For that workflow, write down the output's pre-review state, what it may do before acceptance, what it must not change, send, update, route, apply, or trigger, and what has to be true before anyone treats it as accepted work.

Then answer six questions:

- Is this discovery only, draft assistance, reviewed proposal, or bounded action?
- What may the output do before acceptance?
- Which authority-bearing source answers the most important claim?
- Who can accept, reject, or escalate the decision?
- What reviewer capacity exists this week?
- What stop condition forces the workflow to narrow, throttle, or refuse the automation?

The first output is not a governance program. It is a preflight sheet that a team can inspect before the AI output reaches a customer, record, account, or tool:

```
Workflow:
Authority mode:
What the output may do before acceptance:
What the output must not do before acceptance:
Authority-bearing source:
Required reviewer:
Reviewer capacity:
Accepted state:
Blocked state:
Escalation path:
Stop condition:
Audit record:
```

Before launch, run a small control test against that sheet. Use historical decisions if they are available. Include at least one stale-source drill. Ask reviewers to pass or fail packets using the proposed criteria. Define thresholds for accepted, rejected, escalated, and incomplete work. Define rollback criteria for stale evidence, missing authority, or review overload.

The reader should leave this opening with one practical standard: a useful AI output is not ready because it sounds right. It is ready only when the workflow can say what state it is in, what it may do before acceptance, which source controls the claim, who can accept it, and what happens when any of those checks fail.

The workflow is not safe forever, but it no longer treats a fluent answer as the same thing as an accepted decision.

The opening failure was ordinary: a useful output escaped before anyone had checked the evidence, confirmed who could accept the decision, or preserved rejection authority. The rest of this book builds the contract that should have stood between the draft and the customer. The

first step is naming the category error underneath the escape.

Chapter 1: The Category Error

The category error is treating useful generated work as if usefulness grants authority.

In the same support-credit scenario, the AI-generated proposal behaved like a short decision memo. It summarized the customer history in two pages, used the right internal terms for service credits, and separated the outage timeline from the commercial relationship. It recommended a discretionary credit and gave the account team a short explanation they could adapt for the customer.

The missing controls were in the details. The memo cited an old policy page. It referred to "standard credit discretion" without naming the current approval limit. It assumed the contract allowed the proposed amount. It did not say whether prior credits had already consumed part of the customer's cap.

The memo still had value: it framed the case. But it could not become the customer answer until finance confirmed the amount, compliance checked current policy, and the contract owner resolved the customer-specific cap. Until then, the memo was a proposal.

This is a familiar workplace distinction. A polished memo can be valuable without being authoritative. It can show judgment without owning the right to decide. It can be the beginning of a valid decision process.

AI adoption often goes wrong by losing track of that distinction.

> ***Real case: Mata v. Avianca.*** *In 2023, a federal judge sanctioned lawyers after a court filing included non-existent cases and quotations generated by ChatGPT. The lawyers had a human professional role, but the filing still crossed the court boundary without source verification.*
>
> *The professional gate existed, but verification failed before filing. Lawyers were involved, and the filing looked like legal work, but no one checked the cited authorities against the legal record before filing. The court imposed a monetary sanction and required notice to the plaintiff and to the judges falsely identified as authors of the non-existent opinions.*
>
> *The control lesson is the category distinction: generated legal research may be useful as a lead, but it cannot cross a filing boundary until the cited authorities exist and say what the filing claims. Professional status does*

not convert generated material into authority (Mata v. Avianca 2023).

Partial Frames

Teams reach for familiar labels when they try to understand AI. Each label catches something real and hides something important.

Calling AI a new employee emphasizes delegation. The system can take an assignment, use context, produce a draft, ask for clarification, and sometimes use tools. But an employee is part of an accountability structure. A person can be trained, sanctioned, licensed, promoted, reassigned, or removed. A person can own a decision in the human and institutional sense. An AI system cannot.

Calling AI a search engine emphasizes retrieval. That is sometimes useful. The system can help locate documents, summarize material, and answer from supplied context. But a search engine returns candidate sources. It does not itself make a policy exception true, current, permitted, or accepted. A generated answer can mix retrieved text, learned patterns, user assumptions, and missing context in a single confident paragraph.

Calling AI an automation script emphasizes execution. That frame is useful when the workflow grants tool permissions or runs repeatable transformations. But a script usually does what its program specifies. Modern language models respond probabilistically to prompts, examples, retrieved context, tool outputs, and sampling settings. The same broad instruction can produce different outputs when context changes.

Calling AI "just autocomplete" emphasizes its predictive training objective. It is a useful correction to mystical language, but it can become too dismissive for operational work. Large-scale prediction can produce translation, synthesis, classification, critique, code generation, and other structured outputs.

Research on few-shot behavior and learned internal representations supports the narrow point that these systems can learn some structure from scale and context, while hallucination and uncertainty research keeps the authority claim bounded (Brown et al. 2020; Gurnee and Tegmark 2024; Kalai et al. 2025; Farquhar et al. 2024). Tool use and planning-like behavior add another control problem, but those capabilities are not what these particular citations are being used to support. The bounded claim is simple: probabilistic systems can produce valuable work without owning authority.

The category error appears when a metaphor smuggles authority into the output.

What Large-Scale Prediction Can Produce

A language model is trained to predict text from patterns in data. At use time, it receives a prompt and context, then generates likely continuations according to the model, the decoding settings, and the material it can see. That description is mechanically important, but by itself it does not explain why the systems feel useful at work.

The useful behavior is contextual rather than procedural. Given the right materials, a model can connect a support ticket to a policy page and an outage note, summarize the situation, draft customer-facing language in the company's tone, compare options, and notice when a required contract clause is missing. In code, the same property lets it follow a local pattern instead of inventing one from scratch.

Those are not small abilities. Treating them as fake because the underlying mechanism is prediction does not help a team design a workflow. The analyst's memo was useful even though it lacked authority. AI output can be useful in the same way.

The practical limitation is different. A model can produce judgment-like work without knowing whether the work has satisfied the organization's acceptance conditions. "Approve the credit" may appear before finance authority exists. A policy citation may point to a page that is not the current approved version. Customer language may sound ready even though no one is allowed to create the commitment yet. Tool access can turn those premature claims into effects unless the boundary prevents it.

Capability and authority are separate properties.

Fluency Is Not The Same As Truth

Workplaces have long had to manage fluent but incomplete work.

A senior engineer can submit a pull request with a clear explanation and still miss an edge case. A finance analyst can write a persuasive variance note while using an outdated forecast. An incident lead can offer a plausible root cause before the logs support it. A support manager can summarize a policy exception in language that sounds right while applying the wrong approval limit.

Fluency reduces friction at the exact moment friction may be needed. Coherent work is easier to forward. A complete-looking paragraph attracts agreement. Domain vocabulary reassures busy readers. Under pressure, people often treat "this sounds like our work" as evidence that the work is ready.

AI intensifies that familiar problem because fluency arrives before acceptance. The prose can be clear while the source is stale, the vocabulary can be right while the rule is wrong, and a cautious answer can be followed by an overconfident one. At scale, the workflow may receive more proposals than reviewers can inspect, many of them formatted well enough to look administratively complete.

Distrusting every sentence would waste the system's real value. The practical remedy is to label the output's state before it affects anything important.

Where Authority Enters The Work

Authority enters through the workflow, not through the paragraph.

In the support-credit case, the AI helps before it has authority: drafting the recommendation, extracting the customer's requested remedy, listing policy clauses to check, preparing questions for finance and compliance, and producing customer language after the decision is accepted.

None of those abilities grants the right to approve the credit.

Approval depends on sources and roles outside the model: current policy, contract terms, billing history, customer-specific impact evidence, finance thresholds, compliance guidance, and an accountable owner. If the workflow has not checked those conditions, the answer remains a proposal no matter how well written it is.

This distinction also protects useful AI work from a different mistake: dismissal. If the output is not authority, it does not have to be perfect to be useful. A draft can save time. A summary can reveal missing evidence. A proposed classification can help route work. A critique can sharpen a reviewer packet. The work becomes dangerous when the organization forgets which state it is in.

The same principle applies to human work. A junior analyst's memo may be a draft. A subject-matter expert's recommendation may be ready for review. A finance approver's signed decision may be accepted. A

customer email may be permitted only after acceptance. The labels are not insults. They are how the organization prevents useful work from acquiring the wrong effect.

AI needs the same discipline, with stronger guardrails where scale and tool permissions make leakage easier.

The Proposal Label

The first field in the decision acceptance contract is an output state label.

For a consequential workflow, a starting version can be very small:

Output	Current State	Required Promotion Step
Summary	Draft	Source spot-check
Recommendation	Proposal	Evidence and authority review
Customer communication	Not permitted	Accepted decision and communication owner
System change	Not permitted	Permission boundary and post-action verification

The label has to be attached where work moves, not only written in a policy. If the AI tool produces a recommendation, the interface, ticket note, Slack share, or reviewer packet should carry the state with it. If labels are ignored, the workflow should block the effect rather than rely on visible text alone.

The label should answer three questions:

- What is this output?
- What may it affect right now?
- What must happen before it can be promoted?

Those questions prevent several common failures. A summary can be used for orientation but not treated as proof. A recommendation can be routed for review but not sent as a customer answer. A draft customer message can be edited but not delivered before the decision owner accepts it. A proposed system change can be inspected but not applied before permissions and verification are in place.

This is not a claim that labels solve trust. Labels can become visible text with no effect. People can ignore them. Tools can strip them. Managers can pressure teams to treat "proposal" as "approved unless someone objects." Later chapters add evidence, freshness, permissions, review capacity, audit, and incident learning because a label alone is too weak for

high-consequence work.

But the label is the first correction to the category error. It says that AI output may be useful without being accepted.

Better Models Move The Boundary

One objection is that this distinction will become obsolete as models improve. Better models may cite sources more accurately, reason across longer context, use tools more reliably, and express uncertainty more appropriately. Some failures that are common now may become less common. That matters.

It does not eliminate the authority problem.

A better analyst still submits important work into review. A better engineer still relies on tests and code review before release. A better finance system still respects approval limits. Improving the producer changes how much work can be delegated and where verification is needed. It does not give the producer every decision right in the organization.

Better AI may let the workflow promote more outputs from draft to ready for review by reducing missing-source rates, improving first-pass classification, and making reviewer packets clearer. Those are meaningful gains. They should be measured and used.

But when the output can affect customers, money, records, or systems, the organization still needs to know what state the work is in and who can accept it.

The practical rule is simple: before acting on AI output, label it. Draft, proposal, ready for review, accepted, blocked, or not permitted. The label does not make the work trustworthy by itself. It makes the first and most common mistake visible: treating useful generated work as if authority came bundled with fluency.

Once that category error is corrected, the next question is older than AI. How does important work earn trust at all?

Chapter 2: How Important Work Earns Trust

Important work earns trust through reviewable process, not confidence in the producer. The support-credit case is easier to understand if we first look at a familiar engineering control: a trusted engineer still has to pass release gates.

A trusted engineer opened a small pull request. She knew the service, understood the release pressure, and had fixed similar bugs before. The change adjusted the retry path for a customer-facing workflow that had failed during a partial outage. The description was clear and the tests she added matched the bug report. The team wanted the fix in before the next deployment window.

And yet, the pull request still went through the normal gates. Each gate checked a different claim about the work:

Gate	Claim checked
Maintainer review	The diff matches the intended behavior and does not hide obvious risk.
Continuous integration	The existing test suite still passes with the change.
Release ownership	The retry path will not create unacceptable duplicate work downstream.
On-call review	The new failure mode will be visible in the dashboard.

These gates are not signs that the engineer is untrusted. They are how the team lets a trusted engineer's work become trusted release material.

That is the human-process parallel AI adoption has to recover. Important work does not earn trust because the producer is competent. It earns trust when the producer's work enters a process that can inspect evidence, catch ordinary omissions, reject unsafe effects, and learn from misses.

> ***Real case: Boeing 737 MAX certification.*** *Later reviews of the 737 MAX certification process illustrate why important engineering work cannot earn trust from reputation alone. Boeing and the FAA were experienced aviation institutions, but those reviews found that the certification process did not give independent reviewers enough visibility into MCAS, its assumptions, and its human-factors risks.*
>
> *The control issue for this book is fragmented delegated review: capable people were involved, but the process did not make the right evidence*

visible to the right reviewers before the aircraft entered service.

The consequence was 346 deaths, a worldwide grounding, a long return-to-service review, and sustained public scrutiny of the certification process. The control lesson is human-process trust: consequential work needs evidence, independent review authority, acceptance gates, and failure states that can stop release. Confidence in the producer is not a substitute for a process that can inspect and reject the work (Joint Authorities Technical Review 2019; Federal Aviation Administration 2020; U.S. House Committee on Transportation and Infrastructure 2020).

Where Expertise Fits

Professional work has long relied on capable people. It has also had to account for capable people being tired, rushed, biased, underinformed, politically constrained, or wrong in any manner of ordinary ways.

Software teams use tests, code review, type checks, release gates, logs, and rollback plans because expertise alone does not make software safe to deploy. Incident teams use timelines, telemetry, action items, and postmortems because a plausible explanation is not the same as a demonstrated cause. Finance teams use limits, approvals, reconciliations, and audit trails because a reasonable expense can still violate policy or budget authority. Compliance teams ask for current evidence because memory and intent are not enough.

These controls show a simple distinction: producing work and accepting work are different states. AI belongs in that same frame. Model outputs can save time and raise the quality of first drafts, but consequential workflows still have to ask what must be true before a proposal may change a customer commitment, record, payment, filing, or system.

The answer depends on the effect:

Work effect	Process weight
Private summary	Lightweight source access and user judgment may be enough.
Reviewed proposal	The workflow needs evidence, reviewer authority, and a failure state.
Automatic credit	The workflow needs explicit permission, post-action verification, and rollback or correction.
Code suggestion	The workflow needs review and tests before merge.
Deployed change	The workflow needs release ownership, observability, and recovery.

The control should scale with consequence.

The same inspection protects the producer. When the process is explicit, the expert does not have to carry every risk alone in memory or reputation. The review path can say which concern belongs to the author, which belongs to the reviewer, which belongs to the release owner, and which should stop the work until another source appears. That division keeps serious work from relying on one capable person to hold every dependency at once.

The Work Of Review

"Human in the loop" is a weak phrase because it does not say what the human can actually do.

In our support-credit case, the support manager was nominally in the loop. The manager saw the AI-generated proposal before the customer did, but that fact alone did not make the workflow controlled. Finance authority, current policy ownership, contract-cap evidence, renewal pressure, and the Friday deadline all shaped what the manager could actually do. Calling that step review does not answer whether the review could change the decision.

A real review job has materials, criteria, and effect. The reviewer needs enough information to know what is being accepted and enough authority for the review to change the path of the work. Without that, review becomes a confidence signal rather than a control.

The same pattern appears outside AI adoption:

Review form	When it stops being a control
Code review	It cannot block the release.
Incident review	It creates action items no one owns.
Audit checklist	It runs after the decision is irreversible.
Policy exception review	The reviewer cannot say no.

What matters is what the review was allowed to do.

Process Can Fail Too

A process can fail in ordinary ways:

- Reviewers become overloaded.
- Checklists become stale.

- Approval limits get routed around.
- Small exceptions become the real process.
- Audits turn into backward-looking paperwork.
- Incident reviews become tidy stories that protect prior decisions.
- Release gates are overridden so often that the override becomes the actual gate.

This matters because the answer to AI risk is not simply to "add process" in the abstract. A process that cannot detect its own failure becomes a performance of control.

The support-credit workflow shows the problem clearly:

Artifact	What it says	Why it can still fail
Credit policy	Credits above a threshold require finance approval.	The recommendation may reach the account team as a likely commitment before finance sees it.
AI review label	Generated proposals are non-final.	The label may not survive copying into Slack, a ticket note, or a customer update.
Ticket template	Evidence should be attached.	The field may be present even when the authority-bearing source is missing or stale.
Compliance review	Policy exceptions require review.	Compliance cannot protect the decision without rejection authority to slow or block the answer.

The process has to be tested against the ways it will actually be pressured: customer urgency, sales escalation, executive attention, reviewer backlog, stale sources, unclear ownership, and the convenience of side channels.

A trustworthy process changes what can be accepted when the evidence is missing; a complete-looking diagram is not enough.

What AI Changes

AI does not create fallibility. It changes the rate, surface area, and opacity of fallible work.

Rate matters because a team can generate more proposals than reviewers can inspect. A support group that previously wrote ten exception recommendations a week may suddenly have fifty AI-assisted drafts. If reviewer capacity remains the same, the workflow has not gained trust. It has moved the bottleneck.

Surface area matters because AI output can travel through connected tools. A recommendation can become a Slack summary, ticket note, CRM update, customer email draft, and manager dashboard item. Each copy can shed the uncertainty that belonged to the original output.

Opacity matters because AI can blend sources. A human memo often leaves visible clues: the attached spreadsheet, the quoted policy, the email thread, the gaps the author admits. AI can combine current records, stale retrieved documents, user assumptions, and learned patterns in one smooth answer. The reviewer needs help separating those materials back into claims and evidence.

Accountability matters because AI cannot be accountable in the human sense. People and institutions remain accountable for what the workflow accepts. The model can help produce a packet, but it cannot own the consequences of accepting it.

These differences make reliability disciplines more valuable, not less. Better generation raises the importance of knowing when generated work has actually been accepted.

The Process Evidence Check

The next artifact is a small process evidence check. It asks whether a work item has evidence, a reviewer, an acceptance hook, and a failure state.

Work Type	Evidence Required	Reviewer	Acceptance Hook	Failure State
Pull request	Tests, diff, relevant owner review	Maintainer	Merge blocked until approval	Failing checks or unreviewed risk
Incident report	Timeline, logs, change records	Incident lead	Actions assigned and tracked	Unsupported root cause
Policy exception	Current policy, contract, billing evidence	Domain owner	Decision held until accepted	Escalate or block

For the support-credit workflow, the row matters more than the table:

Work Type	Evidence Required	Reviewer	Acceptance Hook	Failure State
AI credit recommendation	Ticket, current policy, contract cap, billing ledger, customer impact evidence	Support, finance, compliance by domain	Customer commitment blocked until accepted	Missing evidence, stale source, or owner conflict

Each field has to do work.

Evidence required names what the reviewer is allowed to rely on. Reviewer names who has authority for the relevant domain. Acceptance hook says what the process can stop. Failure state says what happens when the packet is incomplete.

If a field never blocks, escalates, logs, throttles, assigns corrective work, or changes permission, it is not part of the control. It may be useful background, but it should not be confused with the mechanism that earns trust.

Accountability Preview

The broad lesson is enough for now: important work earns trust through a reviewable process, not through confidence in the producer alone.

The next question is how to make the process concrete: which evidence can support which claim, who has the authority and capacity to accept or reject the work, and what the reviewer must see before the workflow can move forward.

The Reader Handle

For any important AI workflow, do not begin with the model. Begin with the work item.

Ask:

- What effect can this work have?
- What evidence is required before that action or change is allowed?
- Who reviews which claim?
- What can the reviewer block, escalate, or log?
- What failure state stops acceptance?

Those questions are useful because they are old. They are the same kind of questions teams ask about releases, incidents, approvals, audits, and customer commitments. AI changes the implementation details, but it does not remove the need for work to earn trust.

The next failure is also familiar. Teams add pieces of process that improve the work but do not change acceptance. Better prompts, retrieval, labels, policies, evals, and dashboards can all help. They can also leave the authority boundary untouched.

Chapter 3: When Artifact Fixes Do Not Control The Decision

Artifact fixes improve what the AI produces or how the packet is presented. They do not, by themselves, control which work can become a decision.

After the Friday support-credit incident, the team made several reasonable changes. They rewrote the prompt so the AI had to say when evidence was missing. They connected retrieval to the policy knowledge base. They added a label at the top of every answer: "AI-generated draft; human review required." They published a short policy telling teams not to send AI-generated proposals to customers without approval. They created a dashboard showing sample quality scores from recent outputs.

The next recommendation had better output properties. The language was cleaner. The answer linked to a policy page. It listed two assumptions. The review label was visible.

The same recommendation could still become the working decision before review. It reached Slack as the likely answer. A support note captured it before finance had reviewed the amount. The account team asked whether it could tell the customer approval was coming. The workflow had improved the draft without changing the moment at which the draft became organizationally relied upon.

The failure is specific: artifact fixes improved the work object but did not control the decision. Artifact fixes help. They become risky when the organization treats them as decision controls.

> ***Real case: CNET AI-written articles.*** *CNET's AI article program had several artifact-level controls: some marked AI-written articles carried or later received disclosures and warning labels, the outlet said AI-assisted articles were edited and fact-checked by humans, and corrections were later added. Those controls did not keep unsupported work from publication.*
>
> *The Verge reported that CNET corrected 41 of 77 AI-written stories after internal review found errors. After the errors and weeks of debate, CNET leadership temporarily paused AI-generated content across its sites. The consequence was correction work, public criticism, credibility risk, and an interrupted publishing program. The control lesson is that disclosure, human-review language, and correction notes help, but they do not substitute for a publishing path that can reject unsupported work before*

readers receive it as accepted journalism (Sato and Roth 2023; Christian 2023).

Better Prompts Improve Shape

The prompt change told the AI to cite sources, list missing evidence, avoid overclaiming, and mark the answer as a recommendation rather than a decision.

That improved the output. The recommendation became easier to review. It used a consistent structure. It separated facts from assumptions more often. It asked for the contract cap when that field was absent.

The prompt did not control the workflow.

A prompt can shape what the model is likely to produce, but it does not control the path that follows the answer. It cannot prevent a person from pasting a proposal into Slack as a likely answer, verify policy freshness without workflow support, grant finance authority, preserve a label after text is copied, or stop customer communication while downstream tools still allow the message to go out.

Prompts matter. They are part of the system. But a prompt is not an acceptance rule unless it is connected to something that can block, escalate, log, throttle, or constrain an effect.

Test the prompt against the effect: if the model ignores this prompt, what prevents the bad effect? If the answer is "we hope the reviewer notices," the prompt is an input improvement, not the control.

Retrieval Improves Context

The retrieval change connected the assistant to the policy knowledge base and ticket records. The model no longer had to answer from the user's short description alone.

That also helped. Retrieval brought relevant materials into the answer. It made some citations visible. It reduced the chance that the model would invent a policy name. It helped the support manager find the right family of exception rules faster.

Retrieval still did not answer the authority question.

A retrieved document can improve the answer and still fail the decision. The source may be real but stale, relevant but not controlling, or useful for

one claim while leaving another unsupported. A knowledge-base summary is not the same thing as the approved policy, and neither source can account for a missing customer contract amendment or the wrong operational state.

In the support-credit case, retrieval may be exactly how a bad recommendation gets stronger. The AI finds an old but genuine policy page. The source link makes the answer feel grounded. A busy reader sees a citation and lowers their guard. The policy has not become current merely because it was retrieved.

Retrieval is infrastructure for better candidate context, but the workflow still needs a rule for accepting the decision.

The workflow still needs to ask which source is allowed to support which claim, how fresh that source must be, and what happens when sources conflict.

Review Labels Improve Awareness

The review-label change put a visible label at the start of every AI answer: the output was a draft requiring human review.

That label was better than silence. Labels can slow careless forwarding. They can remind users that generated output is not accepted work. They can make the desired state language visible.

But labels are weak when the surrounding workflow rewards speed.

A label is easy to lose or ignore. It may disappear when text is copied, fade into background noise, or yield to managerial pressure. Most importantly, it can say that review is required without giving the reviewer time, evidence, authority, or a button that blocks the effect. The organization feels covered, but the actual path has not changed.

The support thread showed the weakness. "AI draft, likely path, checking finance" contained caution and leakage in the same sentence. The caution said draft. The leakage said likely. The account team reacted to the part that solved its immediate problem.

Review labels become controls only when they are attached to effect boundaries. If the output is labeled proposal, the customer-send path should remain blocked. If the ticket note includes an AI-generated proposal, the system should preserve its state. If a Slack share would

remove required context, the share should carry the state or route to the reviewer packet instead.

Awareness helps, but it does not enforce the boundary.

Policies Improve Expectations

The policy change said AI-generated proposals could not be sent to customers without human review and the required approval path. It required current sources for policy exceptions. It told employees not to rely on AI for final decisions.

The policy was reasonable. The company needed a shared expectation. Managers needed language for coaching. Compliance needed a standard to point to when reviewing exceptions.

The policy still lived outside the runtime path.

If the ticket system allows an unsupported recommendation to be marked final, the policy has not changed the ticket system. If Slack remains the fastest way to create pressure, the policy has not changed Slack. If finance is notified only after support and sales have aligned, the policy has not changed the order of operations. If reviewers have no capacity, the policy has not created time.

This is familiar outside AI. A release checklist helps only if missing checks can stop the release. An expense policy helps only if approvals and reconciliation can catch violations. An incident review policy helps only if actions are owned and tracked. Written expectations matter, but they become controls when they are connected to a decision rule, escalation, logging, blocking, throttling, or permissions.

The practical question for an AI policy is not whether it sounds responsible. It is where the policy changes the path of work.

Evals Improve Measurement

The fifth fix was an eval dashboard. The team sampled recommendations and scored whether they were coherent, cited sources, and matched reviewer outcomes. The dashboard showed improvement after the prompt and retrieval changes.

That information was useful. Aggregate quality matters. A team should know whether a model upgrade improves source use, whether a prompt

increases missing evidence disclosure, and whether reviewers keep correcting the same class of mistake.

But a dashboard can hide the single escape that matters.

An eval score may say the system is usually good. The next high-risk case may still involve a stale policy, a contract conflict, or a credit amount above the approval threshold. Aggregate measurement does not automatically block a specific unsupported decision. A dashboard that no one uses to change routing, scope, reviewer capacity, or launch thresholds is observability without control.

Measurement becomes part of the decision rule when it changes behavior. If stale-source failures exceed a threshold, narrow the workflow. If reviewer overrides spike, pause automation. If a model change worsens missing-evidence disclosure, roll it back. If certain case types repeatedly require escalation, exclude them until source ownership is fixed.

Evals become more than reporting when they create stop conditions. They are incomplete when they only show that the report looks better this month.

The Rule For Accepting The Decision

The missing piece in all five fixes was the same: an acceptance rule.

An acceptance rule says what has to be true before AI output can affect a customer, money, a record, or a system. It also says what happens when those conditions are not met.

For the support-credit workflow, an early rule could be:

```
An AI-generated credit recommendation may be used only as an internal
proposal
until the current policy, customer contract cap, billing ledger, and
customer-specific impact evidence are attached or marked unavailable.
Customer
communication that implies approval is blocked until finance and compliance
approve or clear the claims they own, or the case is escalated.
```

This rule is narrower than a governance slogan and stronger than a prompt. It names the supporting sources, the blocked effect, the roles that must approve or reject their claims, and the incomplete state.

The rule does not make every answer correct. It makes the workflow inspectable. If the recommendation escapes anyway, the failure is visible: the acceptance rule was bypassed, misimplemented, or too weak. If

reviewers cannot keep up, the failure is visible: the workflow lacks capacity and must narrow or throttle. If sources are stale, the failure is visible: the source map is wrong or source ownership is missing.

That is the difference between an artifact improvement and a control.

Classifying The Workflow

Not every AI use needs the same acceptance rule. A useful classification has four levels.

The first is discovery only. The AI helps find, compare, or inspect information without drafting work that can be accepted. The main controls are source access, data boundaries, and clarity that the output is only exploratory.

The second is draft assistance. The AI helps an individual think, draft, or summarize for private use. Personal-assistant use belongs here when it has no independent operational effect. The main concerns are data boundaries, user judgment, and avoiding accidental disclosure.

The third is a reviewed proposal. The AI produces work that may influence a decision, but a human process must accept it before it affects customers, money, records, or systems. The support-credit recommendation belongs here if the AI only drafts and assembles the packet.

The fourth is bounded action. The AI can change workflow state, send messages, modify records, trigger payments, alter configurations, or take another effect through tools. This requires stronger permission boundaries, post-action verification, audit trails, kill switches, and incident obligations.

Many adoption problems come from treating a reviewed proposal as if it were a draft assistance task, or allowing it to behave like a bounded action without the controls required for action.

The classification should be written into the workflow:

Workflow Class	Allowed Effect	Minimum Control
Discovery only	Source inspection	Source access and data boundary
Draft assistance	Private user aid	Data boundary and user responsibility
Reviewed proposal	Internal recommendation	Evidence, reviewer, acceptance rule
Bounded action	State-changing effect	Permissions, verification, audit, rollback

The support case should not begin by granting bounded action. Let the AI summarize, identify missing evidence, draft internal packets, and prepare customer language after acceptance. Do not let it approve credits, update final ticket disposition, or imply customer commitment until the workflow has earned those effects.

Turning Artifact Fixes Into Controls

The right lesson is to connect artifact fixes to the authority boundary.

Promote artifact fixes by tying each one to an effect. Prompts should produce structured fields reviewers can accept or reject. Retrieval should supply sources with freshness checks. Labels should travel with the output and block customer sends when the state is not accepted. Policies should become routing, permission, escalation, and audit rules. Eval thresholds should change launch, scope, or rollback decisions.

The authority-boundary test turns each artifact fix into a control question:

Fix	What It Improves	What It Still Cannot Stop	How To Promote It
Better prompt	Output shape	Unsupported decision	Require structured missing-evidence fields
Retrieval	Candidate context	Stale or wrong authority	Map sources to claims and freshness
Review label	Awareness	Review without evidence, time, or rejection authority	Block effects until state changes
Eval dashboard	Aggregate quality	Single high-risk escape	Tie thresholds to launch and rollback
Policy	Expected behavior	Runtime action	Encode into permissions and escalation

The last column is the work. It turns reminders into controls.

For every proposed AI fix, ask: what can this fix block, escalate, log, throttle, or prevent from becoming an accepted decision? If the answer is nothing, keep the fix if it improves quality, but do not count it as an authority control.

Choose work that can be controlled. Some tasks are good AI tasks because the objective is clear, evidence is accessible, effects are bounded, and verification is cheap. Others are poor first candidates because the workflow cannot yet say what acceptance means.

Chapter 4: The Shape Of A Good AI Task

A good AI task defines objective, evidence, permission, output, verification, and recovery.

In the support-credit workflow, a strategic customer had asked for a service credit after an outage. The support queue showed the ticket, the customer's latest complaint, and a link to an exception-policy page. Sales had already posted in the account channel that the renewal call was coming. The operations lead added one sentence to the ticket:

```
Handle this exception.
```

Three capable people read the same assignment and saw three different jobs.

One analyst thought the task was to summarize the ticket and prepare a customer update. Another thought the task was to recommend whether the credit should be approved. A third started looking for the customer's contract and billing history because "handle" sounded like a decision with financial effect. None of them was being careless. The work order had not said which result would count as acceptable.

The AI assistant made the ambiguity easier to miss. It could summarize the ticket, find the policy page, draft a recommendation, and produce a polished customer note in one pass. The output looked like progress. But the task shape was still unresolved. Was the system helping a person understand the case? Was it preparing a reviewed proposal? Was it allowed to move a customer record or trigger a credit workflow? The same input could support all three, but each one requires a different acceptance rule.

The first AI design choice is the shape of the work, before model choice.

> ***Real case: NEDA's Tessa chatbot.*** *The National Eating Disorders Association made Tessa available as an eating-disorder prevention and body-image resource while it was also winding down a long-running human helpline. The task boundary mattered. A prevention exercise is not the same task as supporting people who arrive with active eating-disorder concerns.*
>
> *The weak boundary was relying on stated intent instead of separating prevention content from support, triage, crisis routing, and clinical advice. NEDA said Tessa was not meant to replace human interaction, but users still met it in a support context. After reports that Tessa gave weight-loss*

and body-measurement advice, NEDA took the chatbot offline.

The consequence was a public controversy in a setting where the wrong kind of advice can reinforce the problem the service exists to reduce. This was not a clean generative-LLM case; it is mainly a task-boundary case. The control lesson is task shape: "help people with eating disorders" is too broad. The workflow has to separate prevention, education, triage, crisis routing, and clinical advice before an automated tool belongs in the path (Vinall 2023; Wells 2023; Knight 2023).

The Underspecified Work Order

Human teams already know this problem. A manager who tells an analyst to "look into this" may receive a spreadsheet, a memo, a recommendation, or an executed change, depending on how the analyst interprets the hidden constraints. Experienced teams avoid that ambiguity when the stakes are high. They name the objective, the source material, the decision owner, the deliverable, the deadline, and the escalation path.

The support exception needed that same discipline before AI entered the workflow. A summary task would be useful and low risk: collect the customer request, outage timeline, and open questions. A recommendation task would be more consequential: compare the current policy, contract, customer impact, and billing authority, then produce a proposal for review. A bounded action task would be stricter still: apply a credit, update CRM status, or send customer communication only after required approvals exist.

The phrase "handle this exception" did not distinguish among those tasks. It gave background, not authority. It exposed a ticket, not a decision contract.

That failure is common because teams often ask "where can we use AI?" as if the answer were a department, role, or tool category: support, finance, engineering, research, or customer success. Those labels are too broad. The same department contains tasks that are excellent AI fits and tasks that should not be automated without stronger controls.

The useful unit is the task: its objective, evidence, permission, output, acceptance criteria, and recovery path.

Task Shape Before Model Choice

Model capability matters, but it cannot repair an incoherent assignment. A stronger model may infer missing context better than a weaker one. It may ask better clarifying questions. It may produce a more careful draft. But even a stronger model cannot repair an assignment that hides the controlling source, permitted effect, verification method, or stop condition. Fluency just turns that ambiguity into premature confidence.

A task-fit diagnostic is a small way to force the question before the output arrives:

Dimension	Good Fit Signal	Risk Signal
Objective	Clear outcome	Ambiguous goal
Evidence	Accessible sources	Hidden or contested sources
Permission	Read-only or bounded effect	Irreversible action
Output	Observable artifact	Vague judgment
Verification	Cheap check	Expensive or impossible check
Recovery	Easy rollback or escalation	No safe fallback

The diagnostic works as an intake control. It asks whether this workflow can judge the result well enough to let AI help.

In the support case, the summary task passes earlier than the approval task. The objective is clear: summarize the ticket, outage note, current customer ask, and missing evidence. The evidence is accessible if the ticket and outage timeline are available. The permission is read-only. The output is observable. A human can compare the summary to the sources. Recovery is easy: correct the summary or discard it.

The recommendation task needs more. The current exception policy must be available. The customer contract and billing ledger must be reachable or the output must state they are missing. The acceptance criteria must say what counts as supported, blocked, or escalated. The reviewer must know whether the AI is proposing a decision or merely listing options.

The bounded action task needs still more. The system must know who can approve the credit, what amount crosses a finance threshold, whether customer communication is allowed, what audit trail is required, and how the action can be stopped or reversed. A prompt that says "be careful" is not a recovery path.

Good Early Uses

Start where AI can prepare work before it can affect decisions.

AI is strongest early when it turns messy material into work a person can inspect: a short case brief from a long ticket thread, dates and claimed impact from the record, a standard template from messy notes, customer language for a human to edit, options with their required evidence, or a critique of missing sources and inconsistent claims.

Those uses are not trivial. They can save real time and improve consistency. They also keep the acceptance problem visible. The output is a proposal, summary, checklist, or draft. The workflow can inspect it against sources before it affects money, records, or customers.

In software work, the same pattern appears when AI drafts tests, suggests a refactor, or explains a failing build. The output can be valuable without being accepted automatically. The pull request still enters review. The tests still run. The owner still decides whether the change fits the system. The AI has increased the proposal rate; it has not removed the acceptance rule.

For the support team, a strong early task might be:

```
Create an exception-review packet for this ticket. Include the customer's
request, incident summary, policy source used, contract source status,
billing
authority status, missing evidence, and a proposed decision state. Do not
recommend approval unless all required sources are current and present.
```

That task is much clearer than "handle this exception." It names the artifact, limits the effect, and gives the workflow something to reject.

Conditions That Make The Task Risky

Some task shapes should slow adoption down.

Hidden state is one risk. If the material needed to judge the answer lives in a private spreadsheet and an account owner's memory, the task is not ready. The same is true when the AI can reach only an outdated knowledge base while the controlling contract amendment sits elsewhere. In both cases, the system may produce a coherent answer from partial evidence. The bad output will not necessarily look bad. It may be well written and sincerely wrong.

Contested authority is another. If support, finance, legal, and sales disagree about which source controls the decision, AI cannot settle that disagreement by choosing the most convenient source. The task should produce an escalation packet, not an accepted recommendation.

Irreversible or externally visible effects also change the task. Sending a customer note, changing account status, applying a credit, closing a ticket, or marking a compliance review complete creates facts other people will act on. Those effects need stronger boundaries than a draft.

Verification cost matters as much as generation quality. A workflow is a poor early fit if producing the AI output takes ten seconds but verifying it takes three hours of scarce expert time. That does not mean the task is impossible. It means the workflow must narrow the output, sample instead of accept, create better source access, or fund review capacity.

The hard question is not "can AI do this?" It is "can the workflow tell when the result is good enough, and can it recover when it is not?"

Applying The Four Modes To Task Fit

The decision acceptance contract uses four recurring task modes.

The first is discovery only. AI helps a person inspect, compare, or locate information without producing work that can be accepted. The main controls are source access, data boundaries, and clarity that the output is exploratory.

The second is draft assistance. AI helps an individual think, draft, search, or reformat, but it has no operational effect by itself. Personal-assistant use is the individual-use form of draft assistance. The main controls are data boundaries, user responsibility, and clarity that the output is not accepted work. This mode is appropriate for private summarization, option generation, personal notes, and low-authority drafting where the user can inspect the source.

The third is a reviewed proposal. AI produces a packet, recommendation, draft, or classification that may influence a consequential decision. Evidence, freshness, reviewer capacity, rejection rights, and auditability become mandatory. The support-credit recommendation belongs here until the organization has shown that stronger controls work.

The fourth is bounded action. AI can change workflow state, call tools, update records, send messages, or trigger downstream processes. Permissions, approval thresholds, post-action verification, logs, rollback, and kill switches become part of the task. This mode should be narrow at first because the cost of a mistake is no longer confined to the draft.

These modes are not maturity badges. Draft assistance may be the right permanent design for some work. A bounded action mode may be appropriate for a low-risk, reversible, well-instrumented task. The classification exists to stop teams from smuggling one mode into another.

The support queue did not need to choose forever. It needed to choose for this workflow, this quarter, with the sources and review capacity it actually had.

The Task-Fit Diagnostic In Use

A practical intake review can be brief.

For each proposed AI workflow, the owner fills out six fields:

Field	Question
Objective	What outcome should this task produce?
Evidence	Which sources can the workflow use and which source controls each claim?
Permission	What may the AI read, draft, change, send, or trigger?
Output	What artifact will the workflow inspect?
Verification	Who or what can check the artifact, and at what cost?
Recovery	What happens when evidence is missing, the answer is contested, or the effect fails?

The diagnostic should change what happens next. If it only sits in a launch document, it has not controlled the workflow.

If the objective is ambiguous, narrow the task. If evidence is hidden, keep the workflow exploratory or invest in source access. If permission is too broad, remove effects until approval rules exist. If the output is vague, require a packet with fields the reviewer can inspect. If verification is expensive, reduce volume, sample, or fund reviewers. If recovery is missing, do not launch bounded action.

For the support exception, the first acceptable version might be a reviewed proposal, not a bounded action:

```
Mode: reviewed proposal.
Action limit before acceptance: create review packet and draft internal
```

```
recommendation.
Not allowed: customer communication, credit application, CRM commitment, or
ticket closure.
Stop condition: current policy, contract clause, billing authority, or
customer-specific impact evidence missing or stale.
Recovery: escalate packet to finance and compliance with missing fields.
```

That small contract makes the task usable. The AI still does the tedious work of assembling a packet, finding gaps, and drafting language. Missing authority, however, remains missing authority.

When The Diagnostic Feels Too Conservative

The predictable objection is that this approach excludes the most valuable work. The large gains seem to come from letting AI handle messy, high-volume, ambiguous tasks where humans are slow.

That objection is partly right. Many valuable workflows are messy. The answer is sequencing, not refusal. Start where verification is cheaper than the value created. Use early deployments to learn which sources are missing, which decisions are contested, which reviewers are overloaded, and which effects need stronger controls. Then widen the task only when the workflow can accept the result without hiding missing acceptance conditions.

Another objection is that expert users can handle ambiguity. Sometimes they can. But if the workflow depends on expert correction, the task design should say so. It should name the expert, the evidence available to that expert, the time budget, and the right to reject. Otherwise "expert in the loop" becomes a polite way to hide missing acceptance criteria.

A third objection is that advanced agents can plan through ambiguity. They can often help. They can ask questions, search sources, compare alternatives, and identify blockers. But ambiguity about facts is different from authority to act. An agent can discover that the contract is missing. It cannot decide that the missing contract no longer matters.

A Rule For Choosing The First Workflow

Choose the first AI workflow where the output can do only one narrow thing before acceptance, the evidence is reachable, the output is inspectable, verification is cheaper than the value created, and recovery is defined.

That rule makes learning cheaper. A team that starts with a well-shaped reviewed proposal can observe real failure modes: stale policy links, missing contract clauses, reviewer overload, incentive pressure, and unclear approval thresholds. Those findings are useful because they are tied to a specific workflow.

The support case has now moved one step. The team no longer has a vague wish to "use AI in support." It has a candidate task: produce an exception-review packet and internal recommendation under defined limits. The workflow then has to decide whether the packet contains evidence the workflow can actually accept.

Task shape tells the organization what kind of output it is asking for. Evidence tells it whether that output has earned review.

Chapter 5: Evidence, Freshness, And Context

Evidence work separates context, evidence, freshness, and authority.

In the support-credit workflow, a reviewer found that the policy page in the AI packet was last month's version. The draft summarized the ticket, pulled in the outage note, found a policy page in the knowledge base, and drafted a clean recommendation:

```
Approve a discretionary service credit. The outage appears to qualify under
the severe-incident exception policy, and the customer is strategically
important.
```

The draft was not sloppy. It used the right nouns, reflected the customer's frustration, and gave the support manager language that could be sent, with a little editing, to the account team.

The stale source did not make the AI draft worthless. It made the decision unready. The ticket showed that the customer had asked for a credit. The outage note described what happened to the service. The old policy explained how exceptions used to be handled. The current policy had changed the threshold. The contract contained a narrower credit cap and a notice requirement. The billing ledger showed prior credits. The compliance note raised a consistency concern.

All of that was context. Only some of it was evidence. Only some of the evidence was current. And only some current evidence had authority for the claim being made.

The workflow had plenty of context. It had not yet established authority.

> ***Real case: Ofqual's 2020 grading algorithm.*** *When GCSE, AS, and A-level exams in England were cancelled during the pandemic, Ofqual used a standardisation model to calculate grades from inputs such as teacher assessments, school rank orders, and schools' historical performance.*
>
> *The evidence boundary mattered. A school's prior results could help calibrate overall grade distributions, but they were weak evidence for an individual student's actual performance that year. The weak control was treating aggregate context as authority for individual outcomes.*
>
> *After many A-level grades were downgraded and public confidence did not hold, Ofqual and the government withdrew the calculated grades and*

reissued results using the higher of the centre-assessed grade and the calculated grade. The consequence appeared across several records: students' university and career plans were disrupted, the national grading process had to be reversed, and Ofqual's chief regulator resigned. The control lesson is evidence authority: useful context can support a decision, but it should not become the controlling evidence for a claim it cannot establish (Ofqual 2020; House of Commons Education Committee 2020; University of Bristol Centre for Multilevel Modelling n.d.; Richardson 2020).

How Context Gets Mistaken For Evidence

People often talk about context as if it were one substance. More context sounds better than less context. A larger prompt sounds safer than a smaller prompt. A retrieval system sounds more grounded than a model answering from memory. A human summary sounds more reliable than an unassisted generated answer.

Usually, more relevant context does help. A model that can see the support ticket, the policy page, and the incident note will usually produce a better draft than a model that sees only the user's one-line question. Retrieval can connect outputs to external material instead of relying only on learned parameters (Lewis et al. 2020). Documentation practices such as model cards, datasheets, and FactSheets exist because provenance, intended use, and limitations matter in real systems (Mitchell et al. 2019; Gebru et al. 2021; Arnold et al. 2019). NIST's generative AI profile also treats information integrity, provenance, and lifecycle controls as adoption concerns, not cosmetic details (NIST 2024).

But context and evidence do different jobs. Context gives the model material to reason from, summarize, classify, and draft with. Evidence gives the workflow a basis for deciding whether a claim is true enough, current enough, relevant enough, and authorized enough to use.

This distinction is familiar outside AI. An analyst preparing a decision memo may have a spreadsheet, a Slack thread, a policy document, a prior approval, and a manager's memory of how a similar case was handled. Those materials are not equal. The spreadsheet may be useful but stale. The Slack thread may explain pressure but not authority. The prior approval may show precedent but not policy. The manager's memory may be honest but incomplete.

High-consequence work survives by separating those things. Finance does not usually treat an old approval matrix as current authority for this quarter's spend limit. Incident teams distinguish a preliminary timeline from confirmed telemetry. Software teams distinguish a design note from a passing test. Decision makers distinguish assumptions from facts and facts from approvals.

AI does not remove that older discipline. It makes the discipline easier to skip because weak context and strong evidence can appear in the same fluent paragraph.

Checking Claims Instead Of Answers

The support recommendation sounded like one answer. It was really a bundle of claims.

- The outage affected this customer.
- The incident qualifies as severe under the current policy.
- The customer contract allows this kind of credit.
- The proposed amount is within finance authority.
- Prior credits do not change the recommendation.
- Compliance risk is acceptable.
- The support manager may tell the customer approval is likely.

Each claim needs different evidence.

The support ticket shows that the customer asked for help, not that the customer is contractually eligible for a credit. The outage timeline documents the service incident, not this customer's credit cap. The current policy describes the general exception rule but does not override the contract. The billing ledger shows prior credits and account standing without establishing outage severity. A compliance note identifies consistency risk without approving the financial effect.

A source citation can mislead here. A source can support one claim while leaving another claim unsupported. A retrieved document can be accurate and still not be authoritative for the decision. A summary can be faithful to its source and still omit the fact that another source controls the case. Research on summarization factuality and hallucination shows why fluent generated text needs source-grounded checking, but the operational

problem is more specific than hallucination alone (Ji et al. 2023; Maynez et al. 2020). A claim can be fluent, sourced, and still not accepted.

The practical move is to verify at the claim level.

Instead of asking, "Is this AI-generated proposal ready to use?" ask:

- What claims does it make?
- Which source is allowed to support each claim?
- How fresh must that source be?
- Which claims are unsupported, stale, contested, or only partially supported?
- What decision state follows from that evidence?

That last question matters. Evidence is not collected for decoration. It changes what the workflow can accept.

When Real Sources Go Stale

Stale evidence is dangerous because it feels responsible.

The AI did not invent the policy page. It found a real document. The policy page had been correct last month. The language matched the kind of exception the customer wanted. A human under pressure might have made the same mistake.

Freshness has to be handled as part of the decision rule.

In low-risk work, freshness may be informal. If AI summarizes a personal note from yesterday's meeting, the user can often judge whether it is recent enough. In consequential work, freshness needs a rule. The approved policy must be the current approved version. The billing threshold must reflect current account standing. The incident record must be approved or clearly marked preliminary. The contract clause must be the executed customer contract, not a template or sales summary.

Authenticity is not enough. A source can expire, remain retrievable, and be summarized accurately while still supporting a bad decision.

The workflow should not ask only whether the answer is grounded. It has to ask grounded in what, for which claim, as of when, and with whose authority.

This is also where AI breaks the familiar human analogy. A human analyst often leaves some visible trace of the source path: the file name, the spreadsheet tab, the policy URL, the forwarded email, the uncertainty in the memo. AI can compress many source types into one polished answer. Model memory, user context, retrieved snippets, current records, stale policies, and assumptions can land in the same tone of confidence.

The workflow needs to restore the distinctions the prose erased.

Mapping Which Source Can Answer Which Claim

The first artifact is intentionally boring: a small table that says which source is allowed to answer which kind of claim.

For the support-credit workflow, a starting version looks like this:

Claim Type	Authority-Bearing Source	Freshness Rule	Failure Signal
Customer requested credit	Support ticket	Current ticket state	Ticket summary only
Incident occurred	Outage timeline	Incident record closed or approved	Informal Slack recap
Customer was affected	Customer-specific impact evidence	Current telemetry or approved impact note	General outage summary only
Credit is allowed	Current policy and contract clause	Current approved version	Stale KB article
Credit amount is permitted	Billing ledger and finance threshold	Current account standing	Prior spend estimate
Exception is consistent	Compliance note or policy owner review	Current guidance	Prior ad hoc exception

This table is a small guard against one common failure: treating a useful source as if it answered every claim.

The support ticket is necessary, but it does not settle eligibility. The outage timeline is necessary, but it does not settle customer-specific impact. The policy is necessary, but it may not be sufficient if the contract narrows the customer's rights. The billing ledger is necessary for financial authority, but it is not a substitute for policy interpretation.

The table also makes missing evidence visible. If the current policy cannot be found, the output enters a different decision state. If customer-specific impact evidence is missing, the workflow should not use the general outage timeline to establish customer harm. If the contract clause conflicts with the policy, the correct output may be escalation, not a recommendation.

The artifact is small because it maps one consequential decision well enough to show where authority is still missing.

Putting The Draft Into An Evidence Table

The source map tells the workflow where evidence may come from. The claim-to-evidence matrix applies that map to a specific AI output.

For the draft recommendation, the matrix might look like this:

AI Claim	Evidence Required	Current Status	Decision State
Customer qualifies for service credit	Current policy, contract clause, customer impact evidence	Incomplete	Proposal only
Incident severity qualifies as severe	Outage timeline and customer-specific impact	Partially supported	Needs evidence
Recommended amount is within authority	Billing ledger and finance threshold	Missing	Blocked
Customer can be told approval is likely	Accepted recommendation and communication owner	Not accepted	Not permitted

The matrix changes what the team can argue about.

Without it, the team argues about the answer. Sales says the recommendation looks reasonable. Support says the customer needs a response. Finance asks whether the amount is allowed. Compliance worries about precedent. The AI draft floats between them as if its coherence were the shared object.

With the matrix, the team argues about claims and evidence. The disagreement becomes easier to locate.

The recommendation can remain useful. The AI has already summarized the ticket, identified likely policy language, and exposed the missing evidence. But the workflow can now say, with precision, that the output is a proposal only. One claim is partially supported. One claim is missing finance evidence. One customer-communication claim is not permitted because the decision has not been accepted.

This is how a probabilistic output starts to become operationally useful: the fluent answer is broken into claims the workflow can inspect.

Authority Depends On The Decision

One trap in evidence work is looking for "the source of truth" as if every workflow has a single controlling record.

Some decisions do have a clear controlling source. A deployed configuration may be the current configuration. A signed contract may control a customer term. A policy owner may publish the approved policy. But most operational decisions combine several sources, each authoritative for a different part of the decision.

In the support case, the current policy is not more authoritative than the contract in the abstract. It is authoritative for policy interpretation. The contract is authoritative for the customer's obligations and limits. The billing ledger is authoritative for current account standing. The outage timeline is authoritative for incident facts. Customer-specific impact evidence is authoritative for whether this customer was affected.

The workflow fails when it lets one source borrow authority from another.

That can happen with humans. A manager remembers a policy and applies it too broadly. An analyst copies a clause from a prior customer memo. A team treats an incident recap as if it established customer-specific impact. Mature workflows try to catch those mistakes through versioning, approvals, review, and audit.

AI makes the borrowing harder to see. It can put the ticket, policy, outage note, and contract language into one coherent paragraph. The paragraph may sound as if all sources support one conclusion. The matrix forces the paragraph back into parts.

Decision States After The Evidence Check

Chapter 5 adds two pieces to the decision acceptance contract:

- an authority-bearing source and freshness map;
- a claim-to-evidence matrix.

These pieces make acceptance inspectable.

With those controls in place, the support-credit workflow can distinguish at least five states:

- **Draft:** AI has produced useful language, but the claims have not been checked.
- **Needs evidence:** one or more claims require source support.
- **Blocked:** a required source is missing, stale, or outside the allowed authority path.
- **Ready for review:** claims have adequate evidence for a human reviewer to judge.
- **Accepted:** an accountable owner has approved the decision under the workflow's rules.

Those states matter because they prevent a draft from becoming a decision by accident.

In the naive workflow, the AI-generated proposal moved from draft to Slack to expected customer answer before anyone knew whether the policy, contract, billing threshold, and customer-impact evidence supported it. In the improved workflow, the same AI-generated proposal produces a different result:

```
Decision state: blocked.
Reason: current policy found, contract clause conflicts with discretionary
credit language, billing authority missing, customer-specific impact not yet
confirmed.
Valid next action: prepare escalation packet for finance and compliance.
Customer communication: not permitted to imply approval.
```

That output is the workflow doing its job. The AI helped summarize the case and identify the relevant claims. The workflow prevented unsupported authority from escaping.

When The Artifact Is Too Heavy

Sometimes it is.

If a person asks AI to summarize a meeting note for private use, a source map and evidence matrix may be unnecessary. The user can often inspect the original note, decide whether the summary is good enough, and move on. Controls should scale by consequence.

The support-credit recommendation is different because it affects money, customer treatment, audit exposure, and future precedent. Circulation also creates pressure: if sales sees a polished recommendation before finance and compliance have approved or cleared the claims they own, the recommendation starts to become politically real.

The practical question is whether this workflow can safely accept this output without a small evidence artifact.

For consequential decisions, the cost of a small evidence artifact may be lower than the cost of reconstructing the case after the wrong answer has already spread.

When Retrieval Is Already In Place

Chapter 3 treated retrieval as an artifact fix. Here the same fix is tested against claim-level authority.

Retrieval helps. It still leaves acceptance work for the workflow.

A retrieval system can bring relevant documents into the model's context (Lewis et al. 2020). That can reduce reliance on model memory and make outputs more grounded. But retrieval does not automatically decide whether a document is current, whether it is authoritative for the claim, whether another source overrides it, or whether the retrieved passage has been applied to the right decision.

In the support case, retrieval may be exactly how the stale policy entered the answer. The model did not need to hallucinate. It needed only to retrieve a real but superseded document and apply it fluently.

That is why provenance and documentation matter. Model cards, datasheets, and FactSheets all point toward a broader lesson: AI workflows need records about sources, intended use, limitations, and evaluation context (Mitchell et al. 2019; Gebru et al. 2021; Arnold et al. 2019). In operational adoption, those records have to connect to the workflow's acceptance rules.

Better retrieval gives the workflow better candidates. The workflow still has to decide which candidate has authority.

When The Sources Are Messy

The evidence check exposes an operating problem: the team does not know which source can support which claim.

Many organizations do not have a clean source for every claim. Policies live in knowledge bases, PDFs, shared drives, ticket macros, and someone's memory. Contracts have amendments, billing systems lag, incident records can be preliminary, and customer data is messy. This is one of the reasons

AI adoption becomes hard in production.

Hiding that mess inside a fluent recommendation leaves the real problem in place.

If the authority-bearing source is contested, the output should say so. If the policy and contract conflict, the decision should escalate. If the billing threshold is unknown, the workflow should block financial approval. If no one owns the current policy, the AI workflow has exposed an operating problem.

That exposure can be valuable. AI can reveal where the organization was already relying on informal knowledge. The practical response is to start with a narrow workflow and make source uncertainty explicit enough that people can decide what to fix, what to escalate, and what not to automate yet.

A Rule For Drafting The Evidence Packet

Context is material the model can use.

Evidence is material the workflow can use to support or reject a claim.

Relevance, authority, authenticity, completeness, and acceptance are separate states. A source may be relevant without being authoritative, authoritative for one claim and irrelevant to another, or authentic and stale. A summary may be faithful and incomplete. A recommendation may be useful and still blocked.

This distinction is the first practical step from AI as a drafting tool to AI as part of a trustworthy workflow. Once the packet names missing evidence, the organization can assign who owns the delay, who resolves source conflict, who funds review cost, and who has authority to slow or stop the decision. The next question is who has the budget and organizational cover to use that authority.

Chapter 6: The Responsibility And Incentive Split

Accountability requires evidence, authority, capacity, incentives, and protection.

In the support-credit workflow, the ownership meeting assigned a name before it assigned control. The support manager would own the customer answer. Everyone nodded because the sentence sounded clean. Support had the ticket. Support talked to the customer. Support would use the AI assistant to prepare the exception recommendation and send the response after review.

The control gaps appeared in the questions from finance, compliance, sales, and support.

Finance asked who owned the credit exposure if the amount exceeded the normal threshold. Compliance asked who owned consistency across customers if this exception became precedent. Sales asked why review needed to slow down a renewal conversation that was already at risk. The support manager asked whether she could delay the answer until the contract and billing ledger were checked.

The room had named an accountable person. It had not given that person enough authority to be accountable.

The AI assistant did not create that problem. It made the missing authority look resolved. A human name could appear in the workflow while the human lacked finance authority, compliance backing, reviewer budget, and political cover to say no. The recommendation would still look reviewed. The customer answer would still carry a manager's name. The decision would still be shaped by people who benefited from speed and paid little of the downside when the answer was wrong.

Accountability has to be more than a label: rights, evidence, capacity, and consequences all matter.

> ***Real case: Post Office Horizon.*** *In the UK Post Office Horizon litigation, the High Court found a material risk that Horizon bugs, errors, and defects could cause branch-account shortfalls. Subpostmasters were held responsible for discrepancies while the institution and supplier controlled much of the system, data, disclosure, and investigative posture. The weak control was individual accountability without matching*

evidence rights. People closest to the shortfall were expected to answer for numbers they could not fully inspect or challenge.

The consequence was years of harm to people held responsible for system assertions they could not independently test. Later public inquiry and compensation processes addressed institutional accountability, not only individual blame. The control lesson is ownership: accountability without access to evidence, authority to challenge, and protection from institutional pressure becomes blame, not governance (Bates and Others v. Post Office Ltd 2019; Post Office Horizon IT Inquiry n.d.).

Accountability Without Control

Organizations often assign ownership faster than they assign control.

That habit appears outside AI. A project manager may be told she owns a launch, while security owns the approval, finance owns the budget, legal owns the terms, and engineering owns the rollback. If the launch slips, her name is on the status report. If she tries to slow the launch for a real blocker, she may discover that her authority is weaker than her accountability.

Human workflows have developed ways to make that split visible, at least when the stakes are high. Approval limits clarify who can spend. Compliance review clarifies who can reject. Incident roles clarify who commands the response and who communicates externally. Code ownership clarifies who can accept a change. Those controls do not remove politics, but they give the workflow something firmer than blame after the fact. Accountability, internal-control, and finance model-risk guidance make the same general distinction: owners need designed controls, monitoring, evidence, and effective challenge, not only a name beside the outcome (GAO 2021; GAO 2025; Board of Governors et al. 2026).

The support-credit workflow needed the same clarity. The support manager could own the customer relationship and the quality of the response. She could not, by title alone, own whether the credit exceeded finance limits, whether the exception was consistent with policy, or whether a contract clause permitted the proposed remedy.

If the workflow says she is accountable anyway, it has confused message ownership with decision authority.

When You Do Not Control The Whole Workflow

Many readers will not control the whole chain. A support leader may own the customer answer but not the finance threshold. A staff engineer may own the AI integration but not the policy source. A governance lead may own the review standard but not the queue budget. A product manager may be asked to ship the assistant while security, legal, operations, and data owners each control a piece of the acceptance path.

That situation is normal. It is also where AI adoption becomes easy to misrepresent. A team can be asked to "own" the AI workflow while only owning the model call, prompt, or user interface. The rest of the decision still depends on sources, permissions, review capacity, escalation rights, and post-acceptance accountability that sit elsewhere.

The practical rule is simple: own only the authority you can actually control, and document the authority you cannot.

If your team controls only the draft assistance surface, keep the AI in discovery only or draft assistance. You can improve intake, summarize messy records, identify missing claims, prepare review packets, and draft language marked as not accepted. You should not let the workflow imply approval, update authority-bearing records, apply credits, or tell a customer what the organization has decided.

If your team controls the packet but not the decision, make the packet honest. Name the claim, source, freshness rule, missing evidence, reviewer, and unowned approval. Do not hide the missing owner inside a confident recommendation. A packet that says "finance approval unavailable" is more reviewable than a polished answer that implies finance approval already exists.

If your team is accountable for an outcome but lacks the control to accept or reject it, escalate the mismatch before launch. The escalation should be specific:

```
We can own draft quality and packet completeness.
We cannot own credit approval above the threshold.
We need finance approval before customer commitment.
Until that is assigned, the AI workflow must remain reviewed proposal only.
```

This is not bureaucracy. It is boundary clarity. The alternative is worse: a team accepts responsibility for work it cannot verify, cannot fund, cannot block, and cannot correct when the answer escapes.

Partial authority still leaves room for progress. You can start with a lower-authority path, collect evidence about missing owners, measure review debt, and show where the workflow needs investment. The pilot can produce a useful answer even if the final decision is "do not expand authority yet."

The important thing is to avoid letting AI fluency make missing authority look resolved. A model can make an unowned decision sound finished. The workflow has to keep it unfinished until the right owner, source, and review path exist.

Three Kinds Of Work

AI adoption becomes clearer when three kinds of work are separated.

The model does strategy inside the task. It reads available context, proposes language, compares options, identifies likely sources, and drafts artifacts. That work can be valuable. In the support case, the AI can produce a concise case packet faster than a busy manager can assemble one from ticket notes, policy pages, and incident updates.

The workflow owns contracts: which sources count, what the output may do before acceptance, which claims require review, who can approve, and what happens when the answer is incomplete. That contract rejects stale policy pages as support for approval claims and blocks customer communication that implies approval before finance approves the amount and compliance clears the policy issue.

Humans exercise judgment inside assigned authority. Finance judges credit exposure. Compliance judges policy consistency. Support judges customer tone and operational context. Sales contributes commercial urgency, but urgency is not the same as approval authority.

When these jobs are mixed together, the system becomes hard to trust. The model appears to make policy. The workflow hides missing evidence. The human reviewer is left to absorb pressure without rejection authority.

The phrase "human in the loop" does not solve that problem. A human can be in the loop as a real decision owner, a reviewer without evidence, a source of local knowledge, a reviewer without rejection authority, or a person under queue pressure clicking through work. The workflow has to say which condition it is creating.

The Incentive Map

The support exception is an incentive problem as well as an evidence problem.

Sales wants speed because the renewal conversation is close. Support wants closure because open escalations consume time and damage the relationship. Finance wants budget control because credits accumulate and set expectations. Compliance wants consistency because exceptions can become hard to defend. The customer wants an answer that recognizes the outage and the business impact.

Those interests are legitimate. They are not identical.

A responsibility and incentive map makes the split explicit:

Role	Wants	Pays Cost	Owns Risk	Veto Right	Escalation
Sales	Speed, retention	Low review cost	Renewal risk	No credit veto	Executive sponsor
Support	Clear response, ticket closure	Case assembly and customer time	Customer message quality	Weak until defined	Support lead
Finance	Budget control	Credit review time	Credit exposure	Yes above threshold	Finance leader
Compliance	Consistent exceptions	Policy review time	Fairness and audit risk	Yes on policy conflict	Legal or compliance lead
Customer owner	Relationship continuity	Account coordination	Commercial fallout	No policy veto	Account leadership

Enforcement view:

Condition	Owner	Enforcement Hook	Failure Consequence
Support lacks finance authority	Finance owner	Block credit approval above threshold	Escalate or narrow what the output may do before acceptance
Compliance source conflict	Compliance owner	Block customer commitment	Escalate policy interpretation
Review capacity unfunded	Executive sponsor	Throttle queue or fund review	Pilot narrows or stops
Informal override pressure	Workflow sponsor	Require documented escalation	Slack approval does not change decision state

The exact roles will vary. The map matters because a workflow can separate the people who benefit from speed from the people who pay review cost or own the downside. If the workflow does not expose that split, the AI system will be judged by the politics around it rather than the

quality of its controls.

The map should change operations. If finance has rejection authority above a threshold, the workflow should not ask support to approve credits above that threshold. If compliance owns policy consistency, the workflow should not let a support manager override a policy conflict because the customer is important. If sales benefits from speed, sales may need a defined escalation path, not informal pressure on the reviewer.

No map removes disagreement. A good map makes disagreement honest enough to handle.

Who Pays For Review

In this pilot, the risk to test is whether AI increases proposal volume faster than the organization funds review capacity.

That is one reason accountability breaks down. A team launches an assistant that creates more recommendations, summaries, classifications, or draft decisions. The benefit appears in the team that receives more throughput. The cost appears in finance, compliance, legal, security, or senior reviewers who must check the work. If that review capacity is not funded, the process is at risk of degrading.

The degradation may be quiet, so the pilot should test for it directly: whether reviewers skim, approve based on surface plausibility, stop requesting sources because the queue is long, or face escalation work no one has time for. The workflow may still say a human reviewed the answer while review has become a scarce resource allocated by pressure.

The support manager in the ownership meeting could not solve that alone. If the AI assistant produces twice as many exception packets, someone must decide who reviews them, what standard applies, and when the queue stops accepting new work. Otherwise the workflow moves the bottleneck from drafting to reviewing and leaves the bottleneck hidden.

Executive sponsorship becomes practical here. Sponsorship is not only permission to buy a tool. It is responsibility for resolving the cost and risk split. If sales receives the benefit of faster renewal support, finance and compliance cannot be expected to absorb unlimited review work as a side duty. The sponsor has to narrow the workflow, fund review, reduce what the output may do before acceptance, or accept that some decisions will wait.

What Accountable Owners Need

An accountable owner needs more than a name in a RACI chart.

First, the owner needs evidence access. A support manager cannot be accountable for an exception decision if she cannot see the current policy, relevant contract clause, customer-specific impact evidence, and billing status or a clear statement that those sources are missing.

Second, the owner needs authority to reject or halt. If every rejection creates an escalation from sales, the right to reject is ceremonial. The workflow must state that missing evidence, stale sources, policy conflict, or insufficient review capacity are valid stop conditions.

Third, the owner needs capacity or budget. Review time is work. If review is expected, it must be scheduled, staffed, sampled, or throttled. Otherwise the system depends on unpaid attention from people who already have full jobs.

Fourth, the owner needs an escalation path. Some conflicts cannot be resolved inside the immediate workflow. A policy and contract may disagree. A large customer may ask for an exception that finance and sales view differently. A reviewer may be pressured to approve. The workflow needs a named path for those cases before the first high-pressure ticket arrives.

Fifth, the owner needs escalation cover for delay. A person cannot be accountable for safe acceptance if the organization punishes the person every time safety slows a desirable answer.

Test the role directly: if the owner says "no" for a reason named in the workflow, what happens next?

Rejection Authority

Rejection authority is not obstruction. It is the workflow's permission to reject, return, block, or escalate a packet when required evidence, authority, capacity, or acceptance conditions are missing.

In the support case, rejection authority might look like this:

```
Decision state: blocked.
Reason: current contract clause unavailable and requested credit exceeds
support approval authority.
Owner: support manager.
Escalation: finance review required before customer commitment.
Customer communication: acknowledge investigation, do not imply approval.
```

That output protects the customer relationship better than a premature yes. It allows support to respond without fabricating authority. It tells sales what is missing. It gives finance a clear request. It leaves an audit trail showing why the answer slowed down.

Rejection authority also protects the AI adoption program. Without it, teams learn that the official workflow is useful only when it approves quickly. The moment it blocks a high-value case, people start looking for side paths: a private AI tool, an informal spreadsheet, a copied customer note, a Slack approval that never reaches the record. Shadow AI is not always a rebellion against rules. It can be a pressure response when the official path cannot reject a packet and still help the business move.

Rejection authority does not mean every reviewer gets unlimited power to block work. It must scale by consequence. A low-risk internal draft may need only user judgment. A reviewed proposal may need a domain owner to reject unsupported claims. A bounded action that changes money, records, or customer commitments needs stronger approval and escalation rights.

Stopping work is healthy when the workflow protects it. The failure comes when stopping work is informal, punished, or bypassed.

When This Sounds Like Politics

It is politics, in the ordinary organizational sense: people have different goals, constraints, costs, and exposure to downside risk.

Calling that politics should not make the topic feel less technical. Incentive design changes system behavior as surely as a permission check does. If a reviewer is measured only on queue speed, the reviewer will experience evidence requests as friction. If a sales team is measured only on renewal timing, it will experience compliance review as delay. If support is measured only on ticket closure, it will experience blocked decisions as failure. The AI system sits inside those measurements.

The responsibility map is a way to make the operating system visible.

Review the map after blocked decisions, overrides, and repeated reviewer pressure. The review asks whether the named owner had the evidence, authority, capacity, escalation path, and protection needed to make the decision safely. If not, the fix is not another reminder to "use AI responsibly." The fix is to change the workflow's rights and costs.

A Rule For Naming Owners

Do not name an accountable owner unless the workflow also gives that owner evidence, rejection authority, escalation, capacity, and escalation cover for delay.

This rule is deliberately stricter than the way many organizations talk about ownership. It has to be. AI increases the number of plausible proposals that can arrive at a human's desk. If the human cannot inspect, reject, or escalate those proposals, the name in the loop becomes an accountability label without control.

The support workflow now knows which evidence a recommendation needs, who can accept or reject it, who pays for review, and who owns the downside. The next step is to turn some of those responsibilities into boundaries the system can enforce.

Chapter 7: Effect Boundaries For AI Workflows

Effect boundaries define what AI may access, produce, communicate, mutate, or trigger before and after acceptance.

A familiar finance boundary makes the next step visible: an employee may prepare a payment request without being allowed to approve, release, and reconcile it. That boundary is not a comment on her competence. She knows the vendor, the contract, and the reason the invoice needs to be paid quickly. She can fill out the request accurately, attach the invoice, route it to the right cost center, and answer questions from finance.

But the approval button was unavailable because the amount crossed a threshold. Release required a different authority. Reconciliation checked whether the released payment matched the approved one. The workflow preserved useful work while preventing one actor from creating, approving, executing, and validating a consequential effect alone.

AI workflows need the same distinction.

Back in the support-credit workflow, the support assistant is useful inside the work: summarizing the ticket, drafting an exception packet, listing missing evidence, and preparing customer-safe language for an owner to review. Applying a credit, updating the customer record, sending a commitment, or closing the ticket changes the world outside the draft. Those effects need enforceable conditions, not only polite instructions.

Good rails preserve judgment and constrain effects.

> ***Real case: Cruise pedestrian-dragging incident.*** *In October 2023, a human-driven vehicle struck a pedestrian in San Francisco and pushed her into the path of a driverless Cruise vehicle. The Cruise vehicle braked and made contact. It then attempted to pull over, dragging the pedestrian forward instead of remaining stationary. Cruise's recall report said the automated driving system inaccurately characterized the collision as a lateral collision and commanded the vehicle to pull over.*
>
> *The weak control was the post-collision effect boundary: the post-collision subsystem could still command a maneuver when the safe state should have been "stop and stay stopped."*
>
> *The immediate regulatory and operational consequences were concrete. California suspended Cruise's driverless testing and deployment permits.*

Cruise paused driverless operations nationwide. All 950 affected ADS software units were recalled for a software update. Reported broader operational fallout later included more than 900 job cuts during a wider restructuring.

The transferable analogy is effect-boundary design: once an automated system can produce physical or institutional effects, the workflow needs states where action authority is removed, not merely reduced. The Cruise sources support the vehicle and regulatory facts; later chapters use other cases for financial and record-changing effects (NHTSA 2023; California Department of Motor Vehicles 2023; NHTSA 2024; Associated Press 2023; The Guardian 2023).

Rails As Effect Boundaries

Many teams use the word "guardrails" to mean better prompting: tell the model to be accurate, refuse unsafe requests, cite sources, and ask for clarification when uncertain. Prompting matters. It can improve behavior and make the model's job clearer. But a prompt is a weak boundary for actions the workflow cannot afford to get wrong.

The useful question is what the workflow will allow to happen, not only what the model is told to do.

An effect boundary defines what the AI may access, what it may produce, what it may communicate, what records it may mutate, which tools it may call, and which conditions must be true before a consequential action proceeds. It treats an AI output as a proposed move inside a system, not as an answer floating in a chat window.

For the support-credit workflow, the boundary might start like this:

Action	AI Allowed?	Required Condition	Failure State
Summarize ticket	Yes	Data class allowed	Redact or block
Draft recommendation	Yes	Evidence status shown	Proposal only
Request missing evidence	Yes	Source owner identified	Needs evidence
Communicate to customer	No by default	Accepted decision and communication owner	Not permitted
Apply credit	No by default	Finance approval and audit record	Blocked
Update CRM commitment	Limited	Accepted decision state	Proposal only or blocked

Action	AI Allowed?	Required Condition	Failure State
Retry after blocker	Limited	New evidence supplied	Escalate after repeated blocker

This table needs to be executable. Each row names an action, a condition, and a valid failure state. If the only thing preventing a customer commitment is a system prompt that says "do not promise approval," the boundary is too weak.

Access Is An Effect

Teams often think of effects as writes: send the email, update the record, apply the credit, trigger the workflow. Reads can also be effects.

An AI assistant that can read customer contracts, billing history, support tickets, incident notes, and internal compliance comments has access to material that may be sensitive, regulated, or irrelevant to some tasks. Even if the assistant never changes a record, it can expose information in summaries, blend data across contexts, or carry details into a draft seen by people who would not otherwise have permission.

The first boundary is therefore data class and source access. The workflow should say which sources are available for which task mode. A personal drafting assistant may not need billing history. A reviewed proposal may need the billing status but not unrelated contract terms. A finance-approved action may need exact ledger fields and an audit trail.

This is another place where the human parallel helps but does not fully hold. A human employee usually has social and organizational context about why a document is sensitive. AI systems operate through tool permissions, retrieval indexes, prompts, and integration code. If those mechanisms expose the wrong source, the model may use it fluently without understanding the organizational boundary it crossed.

Access control is not separate from answer quality. If the model cannot see a required source, the workflow should block or downgrade the task. If the model can see a prohibited source, the workflow may create a privacy or policy failure even when the answer is accurate.

Communication Is An Effect

A draft becomes more consequential when it leaves the private workspace.

In the support case, an internal recommendation that says "likely approve" can create pressure. A customer-facing note that says the same thing can create a commitment. An account-channel message can shape sales expectations even if it is not sent to the customer. A CRM update can become the next team's source of truth.

For that reason, communication boundaries need to be explicit. The AI may draft language for an owner. It may not send the message, imply approval, or update customer-visible status unless the decision state allows it.

A useful boundary separates several surfaces:

Surface	Allowed Before Acceptance	Allowed After Acceptance
Private analyst workspace	Drafts, options, missing evidence	Final notes and rationale
Internal review packet	Claims, evidence status, blockers	Accepted decision record
Sales/account channel	Neutral status only	Approved talking points
Customer communication	Acknowledgment without commitment	Owner-approved response
CRM or ticket status	Evidence-needed marker	Accepted outcome and audit link

This may look bureaucratic until the first premature statement escapes. Once a customer hears that a credit is likely, the later finance review is no longer a neutral decision. Once CRM says "credit approved pending processing," the next team may treat the approval as real. Once the account channel sees a polished recommendation, the reviewer who blocks it may be framed as the person causing the customer problem.

The workflow should constrain the surface, not merely the sentence.

Machine Checks And Human Decisions

Some boundaries can be enforced by software. Others require human authority. The workflow should not confuse them.

Software is well suited to mechanical boundaries: prohibited tool calls, CRM writes before accepted decision state, missing approval identifiers, source version logs, prompted actions, tool calls, reviewer decisions, and repeated retries against the same missing source.

A system may not be able to decide whether an exception is commercially wise, whether a policy conflict should be escalated, or whether a customer-specific impact note is persuasive enough. Those decisions belong to domain owners.

The boundary between machine checks and human decisions should be visible. If a rule can be encoded cleanly, encode it. If it requires judgment, route it to the owner with the evidence packet and a real reject path. Do not leave an ambiguous judgment hidden inside a prompt.

For the support workflow, a machine check can enforce:

- no customer send before accepted decision;
- no credit tool call without finance approval;
- no approval claim when current policy is missing or stale;
- no repeated retry after the same blocker without new evidence;
- audit logging for source versions, decision state, owner, and effect.

A human owner still decides whether the evidence supports the exception, how to weigh customer context, and whether to escalate a policy conflict.

The point is to keep consequential effects from depending on model obedience alone.

Typed Blockers

When a boundary stops work, it should say why.

Generic refusal is operationally weak. "Cannot complete" does not tell the support manager what to fix. "Policy issue" does not tell finance whether the amount, source, threshold, or approval path is missing. A useful rail returns a typed blocker that matches the workflow's recovery path.

For the support-credit workflow, common blockers might include:

Blocker	Meaning	Valid Next Step
Missing source	Required evidence is unavailable	Request source owner or escalate
Stale source	Evidence exists but is not current enough	Fetch current version or block
Authority conflict	Sources disagree or approval owner differs	Escalate to named owner
Permission denied	AI lacks access or effect rights	Reduce task or request authorized path
Review capacity exceeded	Required reviewer unavailable within limit	Queue, throttle, or narrow scope
Repeated retry	Same blocker recurred without new evidence	Stop and escalate

Typed blockers keep the workflow from treating every stop as model failure. The AI may be doing the right thing when it refuses to approve. The system may be doing the right thing when it prevents a send. The blocker tells the team whether to supply evidence, change permissions, add review capacity, or accept that this task should remain exploratory.

Typed blockers also improve audit. A later reviewer should be able to see that the decision was blocked because the contract clause was missing, not because the model was vague. If blocked packets accumulate for the same reason, the organization has learned something about its source systems or authority model.

Retries, Escalation, And Kill Switches

AI workflows often fail in loops before they fail visibly.

The loop can look productive while nothing important changes. The assistant asks for a source, receives a stale page, tries again, fails the same evidence check, receives reviewer pressure about the customer's importance, and drafts a stronger recommendation. The underlying blocker remains.

Retries need boundaries too. A retry is valid when new evidence, changed authority, or corrected input has arrived. A retry is not valid when it merely rephrases the same unsupported proposal until someone accepts it.

A practical rule is to tie retries to blocker resolution. If the blocker is missing contract evidence, the next attempt must include the contract source or a named escalation that says the contract is unavailable. If the blocker is finance authority, the next attempt must include finance approval or reduce the requested effect. If the blocker is review capacity, the workflow must queue, throttle, or downgrade the task rather than producing more proposals for the same unavailable reviewer.

Kill switches are the broader version of the same idea. If customer communications escape before acceptance, disable customer sends. If credits are applied without audit records, disable the credit tool. If stale policy pages keep entering evidence packets, remove that source from retrieval or mark it ineligible until ownership is fixed. A kill switch is not an admission that AI failed. It is an operational control that keeps a bounded failure from becoming a routine practice.

The Audit Trail Has To Show The Boundary

An audit log that records only the final answer is not enough.

For consequential AI workflows, the log should show the decision state, source versions, tool calls, blocked actions, approvals, owner identity, and final effect. It should preserve enough evidence to reconstruct why the workflow accepted, blocked, escalated, or rolled back an effect.

In the support case, a useful record might say:

```
Draft packet created from ticket, outage timeline, current policy, and
billing
status. Contract clause missing. Recommendation state: blocked. Customer
send:
not permitted. Finance approval: absent. Escalated to finance and
compliance.
No credit applied.
```

That record is more valuable than a transcript that merely shows the assistant was asked to be careful. It shows whether the boundary acted. It also gives the team a basis for improvement. If many cases block on missing contract clauses, the source integration may be the constraint. If many cases block on review capacity, Chapter 8's problem has arrived.

When Boundaries Feel Like They Limit AI

They do. That is the point.

The question is whether they limit the useful part or the dangerous part. A well-designed boundary should leave low-risk drafting, summarization, extraction, and critique easy. It should become stricter as the workflow moves toward customer communication, financial effect, record mutation, or system operation.

This is how mature human workflows behave. They do not prevent an employee from preparing a payment request. They prevent the same person from creating, approving, releasing, and reconciling the payment alone. They do not prevent an engineer from writing code. They require review, tests, deployment controls, and rollback for production changes. The constraint is aimed at the effect, not at the existence of judgment.

AI-specific boundaries must be stronger in some places because tool-mediated effects can happen many times before a person notices. If the integration allows it, a model may draft many messages, update many records, or retry many actions. A prompted instruction may reduce the

chance of misuse, but permissions and workflow state decide what can actually happen.

A Rule For Effect Boundaries

For each AI workflow, list the allowed actions, prohibited actions, required conditions, valid blockers, audit fields, and rollback or kill-switch path.

If that list is impossible to write, the workflow is not ready for bounded action. It may still be ready for discovery only, draft assistance, or reviewed proposal mode. That classification protects the work from authority it has not earned.

The support exception workflow can now say:

```
Allowed: summarize ticket, draft internal packet, identify missing evidence.
Conditionally allowed: update internal review state after accepted decision.
Prohibited by default: customer commitment, credit application, CRM approval
status, ticket closure.
Blockers: missing source, stale source, authority conflict, permission
denied,
review capacity exceeded, repeated retry.
Audit: source versions, decision state, reviewer, approval id, blocked
effects,
final action.
Kill switch: disable customer send and credit tool if effects escape before
acceptance.
```

That specification lets the organization use the AI where the work is still a proposal and stop it where the workflow has not earned authority.

The workflow now has enforceable boundaries for some responsibilities. The next problem is capacity: once the AI can generate more reviewable packets, can the organization verify them without turning review into an unfunded approval step?

Chapter 8: Measuring Accepted Verified Output

Accepted verified output measures AI-assisted work that has passed the workflow's evidence, permission, review, and acceptance checks.

In the support-credit workflow, the adoption dashboard counted generated recommendations. The support AI had produced forty-seven service-credit recommendations that week. The team had asked for speed, and speed was visible. The pilot lead could show usage, generated drafts, average drafting time, and a clean chart moving up and to the right.

The review queue counted the bottleneck.

Six reviewer-hours were available before Friday's customer deadline. A complete review took about fifteen minutes when the packet contained the claim list, current policy, contract clause, outage evidence, billing threshold, and compliance note. Fourteen of the forty-seven recommendations were missing at least one required source. They should not have entered expert review at all. The remaining thirty-three packets were complete enough to review, but six hours at fifteen minutes per packet could cover only twenty-four of them.

The workflow had not produced forty-seven accepted decisions. It had produced forty-seven proposals, fourteen incomplete packets, twenty-four review slots, and nine complete packets of review debt.

The meeting changed once the team separated generated work from accepted work. The question was no longer whether the model could generate more recommendations. It plainly could. The question was how many recommendations became accepted verified output: work that passed the workflow's evidence, freshness, permission, review, and acceptance checks.

Accepted verified output is a practical operating metric for consequential AI workflows with verification bottlenecks. It is not a universal economic law, and the support case here is a composite illustration rather than proof of typical support operations. The metric is useful because it forces the workflow to count what volume charts hide.

> ***Real case: Michigan MiDAS.*** *Michigan's unemployment agency used the MiDAS automated system to accuse claimants of fraud and enforce*

penalties. The weak metric was processed determinations: the system could produce and enforce fraud findings, but that did not mean the findings had been accepted after evidence, review, and due process. Court records and later reporting described automated fraud determinations and inadequate human review, and a class action settlement resolved claims that automated false accusations led to seizure of property without due process.

The consequence was not bad dashboard optics. More than 3,000 class members shared a $20 million settlement, and the public repair had to address refunds, reversed determinations, and limits on automated functions without human review. The measurement lesson is direct: generated or enforced output is not accepted verified output (Cahoo v. SAS Analytics Inc. 2019; Michigan Attorney General 2022; Oosting 2024; University of Michigan Science, Technology, and Public Policy Program 2024).

Generated Work And Accepted Work

Workplaces already know the difference between activity and accepted work.

A software team can open many pull requests without increasing shipped reliable software. A claims team can prepare many files without increasing approved claims. A finance team can receive many reimbursement requests without closing the books. The visible production step matters, but it is only one part of the system.

Human teams use review queues, work-in-progress limits, quality gates, sampling, and throttles because production and verification are different capacities. Software inspection and testing research makes the cost shape visible: finding, checking, and correcting defects is its own work, not free capacity attached to production (Shull et al. 2009; NIST 2002). A developer can write code faster than reviewers can understand it. An analyst can draft more memos than approvers can accept. A support manager can prepare more exception requests than finance and compliance can responsibly review.

AI makes this old bottleneck easier to miss. Proposal generation becomes cheap, fast, and polished. The output looks closer to finished work than a rough human draft. It can spread into tickets, documents, Slack threads, and account notes before the workflow has established whether it is supported and permitted.

Generated output is useful. It is not the same as accepted output.

For the support-credit workflow, generated output means the AI produced a recommendation or packet. Accepted verified output means the recommendation made it through the acceptance contract:

- required claims were identified;
- authority-bearing sources were current enough for those claims;
- permissions and effect boundaries were respected;
- a reviewer had enough evidence, time, and authority to accept;
- the decision and its basis were logged.

The metric is intentionally stricter than "AI-assisted output." A draft can be AI-assisted and still blocked. A packet can be complete and still rejected. A review can occur and still leave the decision unaccepted. Accepted verified output counts the work that actually crossed the trust boundary.

The Queue Math

The first version of the calculator should be simple enough to do on a white board.

In the Thursday support queue, the inputs were:

Input	Example
Proposals expected	47 per week
Complete-packet rate	70 percent
Reviewer-hours available	6 per week
Average review time	15 minutes
Accepted packet rate after review	75 percent scenario assumption

From those inputs, the workflow can derive the first decision numbers.

Forty-seven proposals at a 70 percent complete-packet rate produce 32.9 complete packets, rounded to thirty-three. That leaves fourteen incomplete packets. Incomplete does not mean bad faith or useless. It means the packet is missing required evidence and cannot yet consume expert acceptance time.

Six reviewer-hours at fifteen minutes per review produce twenty-four review slots:

```
6 reviewer-hours x 60 minutes = 360 reviewer-minutes
360 reviewer-minutes / 15 minutes per packet = 24 reviewed packets
```

Thirty-three complete packets minus twenty-four review slots leaves nine complete packets of review debt.

If the team assumes, for scenario planning, that reviewers accept 75 percent of reviewed complete packets, then twenty-four reviewed packets produce eighteen accepted verified recommendations:

```
24 reviewed packets x 0.75 accepted rate = 18 accepted verified outputs
```

That last number is a planning estimate, not a claim about the true value of the pilot. The accepted rate has to be measured during the pilot, and the value of an accepted output depends on the workflow. A service-credit recommendation, a legal clause review, and an incident summary do not have the same economic value or risk.

The calculator exists to prevent a team from mistaking forty-seven generated recommendations for forty-seven accepted decisions.

Incomplete Packets Should Not Become Reviewer Homework

One common failure is using expert reviewers to reconstruct missing evidence.

In the support case, fourteen packets lacked at least one required source. Some were missing current policy. Some had the outage timeline but no customer-specific impact evidence. Some had the ticket and draft customer message but no billing threshold. A reviewer could chase those sources, but that would turn review into investigation. The bottleneck would worsen, and the workflow would hide the true cost of incomplete packets inside reviewer effort.

Human workflows often learn this lesson the hard way. A compliance team that receives incomplete launch packets can either become the research staff for every operating team or return the packet. A senior engineer reviewing a pull request can either infer the missing test plan or ask the author to supply it. A finance approver can either search for receipts or reject the expense as incomplete.

The AI analogue is sharper because incomplete output can look finished. A recommendation may contain confident language, a plausible rationale, and a source citation while still missing the source that controls

the decision. If that packet enters review, the reviewer pays for the missing discipline.

The operating rule should be explicit:

```
Required evidence missing: auto-reject to proposal state.
Reviewer action: not required until packet is complete.
Valid next step: gather missing source or narrow the recommendation.
```

Auto-reject does not mean discard. It means the output remains a proposal. The AI may still be useful by listing the missing evidence, preparing the source request, or narrowing the claim. But the workflow should not spend scarce reviewer capacity on packets that are not ready.

Accepted verified output is more useful than raw generation count here. The fourteen incomplete packets are not invisible. They are measured as incomplete proposals, and their causes can be reviewed. If most incomplete packets are missing customer-specific impact evidence, the workflow has a source integration problem. If they are missing current policy, the retrieval path or policy ownership may be wrong. If they are missing billing thresholds, the proposed scope may exceed what the pilot can support.

Review Debt Is A Stop Condition

Review debt is not merely an inconvenience. In consequential workflows, it is a risk signal.

The Thursday queue had nine complete packets over funded capacity. The team had to change either capacity or scope: add reviewer-hours, narrow the pilot to the highest-value or lowest-risk requests, throttle generation above twenty-four open proposals, defer lower-priority recommendations, or stop the workflow until capacity matched demand.

What it should not do is let proposals pile up while the dashboard celebrates generation.

Unreviewed complete packets create pressure. Sales sees a recommendation and wants to move. Support sees a customer waiting and wants to close the ticket. Managers see the model producing useful-looking work and ask why the team is slow. The queue becomes a political object, not just an operational object.

Older workplace controls exist for this reason. Work-in-progress limits are not only about efficiency. They protect quality by making overload

visible. Quality gates are not only about inspection. They prevent downstream systems from absorbing work that has not met the acceptance standard. Throttles are not a failure of adoption. They are the mechanism that keeps the workflow honest about capacity.

For AI-assisted decisions, a practical threshold might read:

```
Throttle condition: complete review queue exceeds funded review capacity for
the week, or any high-risk packet waits beyond the service-level target.

Allowed responses: add reviewer capacity, narrow scope, defer low-risk
packets,
or stop generation for this workflow.
```

The exact threshold depends on the workflow. A low-risk internal summarization tool may tolerate more backlog. A customer-credit recommendation, compliance approval, clinical decision support packet, or security exception may need a harder stop. The important feature is that the number changes what the workflow can accept.

What The Metric Can And Cannot Show

Accepted verified output is a proposed operating metric, not a complete ROI model.

The metric tells the team how many generated proposals became accepted work under the workflow's rules. It also exposes review debt, separates incomplete packets from rejected complete packets, distinguishes better packet quality from mere volume, and shows when review capacity, not drafting capacity, is the limiting resource.

It cannot, by itself, establish organizational value.

The team still needs assumptions or measurements for value per accepted output, reviewer cost, avoided rework, escaped-error rate, customer impact, and the cost of delay. Those numbers should be labeled carefully. In the support case, the team might estimate avoided rework from rejected incomplete packets, but that estimate is not the same as measured savings. It might assign a scenario value to each accepted recommendation, but that value depends on the customer's contract, the credit amount, retention risk, and downstream audit cost. It might track escaped errors through incidents and sampled audits, but early pilot data will be thin.

The honest scorecard keeps those categories separate:

Measure	What It Says	What It Does Not Say
Generated proposals	AI activity	Accepted work
Complete-packet rate	Source readiness	Correctness
Reviewed packets	Reviewer throughput	Acceptance
Accepted verified output	Work crossing the trust boundary	Full economic value
Review debt	Capacity mismatch	Root cause by itself
Escaped-error findings	Known misses	Total unknown risk

This separation protects the pilot from two bad stories. The first story says "AI generated more, so the pilot worked." The second says "review found problems, so the pilot failed." Neither conclusion follows. A pilot can be valuable because it exposes incomplete sources, review capacity limits, or a scope that is too broad. It can also be unready because accepted verified output is too low relative to cost and risk.

The metric does not decide the business case alone. It makes the business case harder to overstate.

A sponsor still has to compare the workflow with alternatives: keep the current manual process, narrow the AI task to drafting only, fix source ownership before adding AI, fund more review without AI, or stop the pilot. Accepted verified output earns a place in that comparison because it counts work that crossed the trust boundary. It does not establish that the workflow is worth funding.

The executive question is therefore:

```
Does the workflow produce enough accepted verified output, with low enough
escaped-error risk and funded enough review capacity, to justify the source,
review, tooling, and governance cost?
```

If the answer is no, the honest result is not a failed book method. It is a useful stop signal. The team has learned that this workflow is not yet a good AI adoption candidate, or that the work should remain draft assistance until source ownership, review capacity, or recovery paths improve.

A Weekly Scorecard

A practical weekly scorecard for the support-credit workflow can fit on one page:

Field	Week Example
Proposals generated	47

Field	Week Example
Incomplete packets auto-rejected	14
Complete packets	33
Reviewer-hours funded	6
Max reviewed packets	24
Accepted verified recommendations	18
Rejected reviewed packets	6
Review debt	9
Escaped-risk findings	Track from audits and incidents
Throttle decision	Narrow scope or add capacity

The scorecard owner should be the workflow sponsor, not the model vendor and not the reviewers alone. The sponsor is the person or group asking the workflow to create value. That owner has to fund reviewer capacity, accept the throttle, or narrow the use case. Reviewers should not be made responsible for a capacity problem they cannot control.

The source authority for the scorecard should be mundane: queue logs, packet states, reviewer time, accepted and rejected decisions, sampled audits, and incident feedback. If the workflow cannot produce those records, then it cannot honestly claim the metric.

The update cadence should be frequent during the pilot. Weekly is often enough to expose the trend without turning measurement into daily ritual. After an escaped high-consequence error, the scorecard should be reviewed immediately because the risk model has changed.

The Accepted-Output Test

Generated volume is a weak success measure for consequential AI work because it stops before the trust boundary.

Accepted verified output asks a narrower and more operational question: how many AI-assisted proposals became accepted work after evidence, permissions, review, and logging did their jobs?

The answer may be lower than the adoption dashboard suggests. That is useful because it shows where the real constraint is: incomplete packets, unready source systems, too little review capacity, an overbroad pilot scope, or accepted work that is not valuable enough to fund the bottleneck.

The failure test is simple. If the team cannot measure accepted verified output from logged accepted decisions, or estimate expected throughput from proposal count, packet completeness, review capacity, acceptance rate, and escaped-risk feedback, it is measuring activity around the workflow rather than the workflow itself.

Once the queue makes review capacity visible, the next question is whether the review itself can change acceptance. A human in the loop answers who is present. The workflow still needs a packet the reviewer can judge, time to judge it, and authority to reject or escalate when the right answer is no.

Chapter 9: Conditions For Useful Human Review

Useful human review requires evidence, standards, time, independence, and rejection authority.

In the support-credit workflow, the reviewer screen broke the AI draft into claims, sources, and actions. The draft was a smooth recommendation: approve a discretionary service credit for the strategic customer after the outage. The screen showed the proposed action, the claims that had to be checked, links to the sources, freshness status, missing evidence, the risk if the decision was wrong, and three actions: accept, reject, or escalate.

One source was stale. The policy page in the AI packet had been superseded last month. One claim was missing evidence. The outage timeline showed a service incident, but the packet did not establish customer-specific impact. The proposed credit also exceeded the support manager's ordinary authority.

In the old workflow, this would have become a Slack negotiation. Sales would ask whether the reviewer could "just approve this one." Support would point to the customer deadline. Finance would ask for the amount. Compliance would ask which policy version controlled the exception. The reviewer would become the person slowing the deal down.

In the improved workflow, the packet made the no easier to give:

```
Decision state: escalate.
Reason: stale policy source, missing customer-impact evidence, proposed
amount
above support authority.
Rejection authority: reviewer may reject or escalate incomplete packets
without
manager override outside the approval path.
```

The AI output remained useful. It summarized the case, identified the likely policy issue, and prepared a focused review object. But the human review was useful only because the reviewer could see the evidence, apply a standard, and stop the decision.

"Human in the loop" is a staffing description. It becomes a control only when the review can change what the workflow accepts.

Real case: Uber automated-vehicle crash. *The NTSB found that an Uber test vehicle's automated driving system detected a pedestrian before a fatal 2018 crash. The system did not classify and respond effectively, and the design relied on the vehicle operator to intervene.*

The weak control was human presence as fallback. It failed because the operator was visually distracted, while the broader safety program had not adequately managed automation complacency, oversight, and risk assessment.

The consequence was a fatal crash in a workflow that had assigned last-mile safety to a human who was not positioned to perform that role reliably. As an automation-oversight analogy, the control lesson is that human review is useful only when the reviewer has attention, information, time, authority, and an interface that supports intervention (National Transportation Safety Board 2019).

Presence Is Not Oversight

Workplaces often rely on review because people are fallible. That does not mean any review works.

A senior engineer reviewing a pull request needs the diff, tests, context, ownership boundaries, and enough time to think. A finance reviewer approving expenses needs receipts, policy limits, cost centers, and authority to reject. A compliance reviewer needs the launch packet, acceptance criteria, and a path to escalate without being punished for delay. In each case, the reviewer is not magic. The reviewer is part of a contract.

The human probability issue is familiar. Reviewers miss things, anchor on the author's confidence, overload under queue pressure, and drift toward perfunctory approval when rejection is socially costly. They may accept incomplete work because reconstructing the evidence would take too long. A human can be present in the workflow and still lack the conditions needed for judgment.

AI can make that failure mode sharper in the workflows this book is concerned with. Generated recommendations may arrive in larger numbers and with a more finished surface. The prose may sound complete even when the evidence is incomplete. The reviewer may feel that the model has already done the hard work. If the interface shows only the answer and an "approve" button, the workflow is asking the reviewer to certify an object they cannot inspect.

The useful standard is narrower:

```
For consequential AI work, human review becomes meaningful when the reviewer
has evidence, criteria, time, expertise, independence, authority to reject,
and
a practical interface for the judgment being requested.
```

That sentence is this book's operating standard, not a sourced universal law about all review. Some low-risk workflows need less. Some high-risk workflows need more. Under this standard, a consequential AI workflow that lacks those conditions should not treat human oversight as a meaningful control.

The Reviewer Packet

The reviewer packet is the artifact that turns a fluent recommendation into a judgment task.

For the support-credit workflow, a useful packet includes:

Field	Purpose
Proposed action	What the AI recommends
Claim list	What must be checked
Source links	Where the evidence came from
Freshness status	Whether sources are current, stale, or unknown
Missing evidence	What blocks acceptance
Risk consequence	What happens if the decision is wrong
Reviewer action	Accept, reject, or escalate
Rejection authority	Workflow permission to reject or escalate

Each field must do work. A field that does not change the reviewer's judgment, the decision state, or the audit trail belongs somewhere else.

The proposed action defines what is at stake. "Approve service credit" is different from "prepare customer response" or "request finance review." A reviewer cannot judge a vague recommendation because the effect boundary is unclear.

The claim list prevents the answer from being reviewed as one block. The reviewer can check whether the customer was affected, whether the current policy applies, whether the contract allows the credit, whether the amount is within authority, and whether customer communication is permitted. Some claims may pass while others fail.

Source links and freshness status keep the review from becoming memory work. The reviewer should not have to hunt across ticket systems, contract folders, policy pages, and incident notes just to determine whether the packet is reviewable. If the policy is stale or unknown, that status should be visible before the reviewer invests in the rest of the packet.

Missing evidence is a decision input. If customer-specific impact evidence is missing, the reviewer should not be asked to infer it from a general outage summary. If billing authority is missing, the reviewer should not be asked to guess the threshold.

Risk consequence keeps review proportional. The same missing evidence may have different consequences in a private draft, a reviewed customer recommendation, and an agentic workflow that can issue a credit. The reviewer needs to know what acceptance would allow.

Accept should not be the only easy button. Reject and escalate should be just as reachable, logged, and protected by policy. If rejection requires a side conversation, the workflow has tilted toward approval.

Rejection authority names the policy that backs the reviewer. Without it, the reviewer may be technically authorized and politically exposed.

Time And Independence

Review degrades when the schedule makes judgment impossible.

In the Thursday queue from the prior chapter, six reviewer-hours could cover twenty-four complete packets at fifteen minutes each. If managers still expect all forty-seven generated recommendations to be reviewed by Friday, the review standard has already been compromised. The reviewer can either work unpaid overtime, skim, approve late, or become the visible blocker.

Human organizations have seen this pattern before. Late-stage compliance review often becomes contentious when teams bring incomplete packets near a launch deadline. Code review degrades when reviewers face a large diff shortly before a release freeze. Finance approval becomes political when an executive sponsor needs a purchase order after the fact.

The control depends on capacity and independence, not instructions alone.

Time means the workflow funds the judgment it demands. Review-time estimates should be measured, not guessed forever. High-risk packets may need longer review. Incomplete packets should be auto-rejected to proposal state rather than consuming expert reconstruction time.

Independence means the reviewer can apply the standard without being absorbed by the team that benefits from approval. The reviewer may sit in the same organization, but the workflow has to protect the role. If sales pressure, support closure metrics, or executive impatience can override the reviewer through informal channels, the interface cannot fix the review.

For the support-credit workflow, independence might be expressed as:

```
Reviewer may reject or escalate packets that lack required evidence.
Manager override must use the finance/compliance escalation path.
Slack approval does not change decision state.
```

That last sentence matters. Informal approval paths are where review authority often leaks.

Expertise And Criteria

Experts still need criteria.

A finance reviewer may understand credits but not incident severity. A support manager may understand customer history but not contractual limits. A compliance reviewer may understand policy consistency but not account-level billing thresholds. A single human reviewer may not be qualified to accept every claim in an AI-generated recommendation.

The packet should therefore match claims to review authority. Some claims can be accepted by support. Some require finance. Some require compliance. Some should be escalated when policy and contract conflict.

AI explanations do not replace that authority. They can help a reviewer understand why the model proposed an action, expose the source path or assumptions, and make review faster. The explanation remains an input, not acceptance.

A practical review standard includes:

- claim-level criteria;
- source types allowed for each claim;
- risk tiers that change review depth;
- reviewer roles for each decision part;

- escalation rules for conflicts or missing authority.

This prevents two opposite failures. In the first, every packet goes to an overqualified expert, and the workflow cannot scale. In the second, every packet goes to the nearest available human, and review becomes ceremonial. Claim-level criteria let the workflow reserve scarce expertise for the claims that need it.

The Interface Can Create Bad Review

The review interface is part of the control.

If the screen shows the AI answer first, hides source status behind clicks, buries missing evidence, and makes rejection awkward, the workflow may make acceptance easier than judgment. If the screen shows claim status, source freshness, missing evidence, and reject or escalate controls next to accept, it gives judgment a better path.

Interfaces decide what work is easy under time pressure. A reviewer with ten packets left before a deadline is likely to follow the path the system makes available. If "approve" is prominent and "reject" means writing a custom explanation in Slack, the workflow is applying pressure toward approval.

For the support workflow, the review screen should make these states explicit:

```
Accept: all required claims supported, permissions valid, reviewer authority
matches proposed action.

Reject: required evidence missing, source stale, claim unsupported, or
proposed
action outside permitted scope.

Escalate: source conflict, authority conflict, high-risk exception, or
business
pressure outside standard policy.
```

The interface should also log why the reviewer acted. The log is not there to punish reviewers for saying no. It is there to let the workflow learn whether the model, source map, policy, or queue design is failing.

Calibration And Learning

Review should improve the system. Otherwise it becomes a tollbooth.

Accepted packets, rejected packets, escalations, reviewer comments, sampled audits, and escaped errors should feed back into the workflow. If

reviewers repeatedly reject packets for stale policy, the retrieval or policy ownership path needs repair. If reviewers repeatedly escalate contract conflicts, the source map may need a clearer precedence rule. If reviewers accept packets that later fail audit, the review criteria or interface may be insufficient.

Calibration is also how the team checks whether reviewers apply the standard consistently. Two reviewers looking at the same packet should not reach different decisions because one felt more pressure from the account team. When they disagree, the workflow should treat the disagreement as evidence. Maybe the policy is ambiguous. Maybe the risk tier is unclear. Maybe the packet hides the deciding source.

Judgment remains necessary. The goal is to make the judgment inspectable enough that the system can improve.

The Reviewer-Control Test

Before claiming human oversight, ask what the human can actually do.

The test is practical: whether the reviewer can see the claims, open the authority-bearing sources, check source freshness, identify missing evidence, apply the acceptance criteria, spend enough time, reach the right expertise or escalation path, reject without negotiating in side channels, and trust the workflow to respect the no.

The failure test is direct. If a reviewer must reconstruct evidence manually, guess the standard, accept outside their authority, or fight the organization to reject an incomplete packet, the workflow does not have meaningful human review. It has human exposure.

The reviewer packet is the practical artifact. It should show the proposed action, claim list, source links, freshness status, missing evidence, risk consequence, reviewer action, and rejection authority.

The packet does not make review free. It makes the review task honest. Once the official path has real review, a new pressure appears: people may compare it with faster unofficial paths. If the sanctioned workflow is too slow, too hard, or too narrow, teams may look for another way to keep work moving. Chapter 10 treats that as a pressure pattern to investigate, not as an assumed fact about every organization.

Chapter 10: From Assistant To Bounded Agent

Assistance and agency differ by authority: drafting work is not the same as tool-mediated action that changes workflow state.

In the support-credit workflow, side paths appeared because the official process could not handle legitimate drafting pressure. The official support AI workflow was slow. It required a complete evidence packet, current policy, contract clause, billing threshold, and review before a service-credit recommendation could be accepted. Those controls were there for good reasons. They also meant the support manager could not answer a strategic customer as quickly as sales wanted.

One account lead copied ticket notes into a private AI tool to draft a customer message. A support lead kept a spreadsheet of prior exceptions because the official packet took too long to assemble. A manager reused an old template from a shared drive because it sounded right. A side Slack thread collected "looks good to me" approvals that did not change the formal decision state.

Security saw customer context moving through an unapproved tool. Finance saw credit decisions forming outside the credit workflow. Compliance saw stale templates returning through the side door. Support saw the same operating problem from another angle: the sanctioned path could not meet the pressure placed on it.

Treat the bypass as both a policy violation and a signal that the official path lacks a usable lower-authority option.

That pressure pattern is the bridge to bounded action: before granting more tool authority, the workflow needs a sanctioned lower-authority path for the work people are already trying to do.

An assistant may draft, summarize, classify, and prepare packets without changing the world. A bounded agent can take actions through tools: update a ticket, route an approval, send a message, issue a credit, change a configuration, or close a case. That move changes authority, not only capability.

Move from assistance to agency only when the workflow has scoped tools, explicit permissions, valid failure states, sanctioned alternatives,

logging, and post-action verification.

> ***Real case: Knight Capital.*** *In 2012, Knight Capital deployed code into an automated equity router with direct market access. A dormant function that was no longer intended for use was triggered. While processing 212 customer orders, the router sent millions of orders into the market and obtained more than 4 million executions.*
>
> *The weak control was the absence of effective action-specific stops: the system could keep acting in the market even though its behavior was erroneous, and Knight lacked controls that could stop orders from exceeding firm-wide capital thresholds.*
>
> *The consequence was visible in market and capital terms. During the event, Knight acquired several billion dollars in unwanted positions. Unwinding those positions left the firm with a loss of more than $460 million.*
>
> *The control lesson is direct: once software can act through tools, markets, records, or money, the workflow needs action-specific limits, automated stops, incident authority, and clear conditions for disconnecting the system from the effect (Securities and Exchange Commission 2013a; Securities and Exchange Commission 2013b).*

Shadow Processes Are Older Than AI

Workarounds existed before generative AI.

Teams have long used spreadsheets beside official systems, copied templates instead of current forms, local approval lists instead of central workflows, and side-channel messages to move work around a slow process. Sometimes those workarounds are irresponsible. Sometimes they are the only way people can meet conflicting demands. Often they are both.

The human probability issue is not hard to understand. People create local paths when the sanctioned path is slower than the work requires. A support team with a waiting customer will optimize for response. A sales team near renewal will optimize for relationship. A finance team will optimize for control. A compliance team will optimize for defensibility. If the official workflow does not reconcile those pressures, side paths are a pressure pattern worth investigating.

AI changes the mechanics. A private tool can ingest sensitive context, produce polished customer-facing language, and spread unsupported recommendations faster than an old spreadsheet. An unofficial agent can

act through connected tools if permissions are loose. A copied stale template can become the seed for many fluent drafts.

The mature response asks why the bypass exists, which risks it creates, and what sanctioned path can handle the legitimate need without giving away authority.

In the support case, the first repair is a low-friction sanctioned draft path, not a credit-issuing agent:

```
Allowed: summarize ticket, draft customer-neutral holding response, list
missing evidence, prepare source request.

Not allowed: imply credit approval, send customer commitment, update
billing,
or change decision state.
```

That path gives the team speed where speed is safe, while preserving the acceptance contract for financial and policy effects.

Assistance, Proposal, And Action

The word "agent" often hides the decision that matters.

The useful distinction is effect authority. What can the AI-mediated system change?

At the lowest tier, discovery only has no operational effect and produces no accepted work. It can help a user find, compare, or inspect information. The main controls are source access, data boundaries, and clarity that the output is exploratory.

At the draft assistance tier, the system can help a user think, summarize private notes, or draft text the user will inspect. Personal-assistant use belongs here when it remains private and low authority. The main controls are data boundaries, user responsibility, and clarity that the output is not accepted work.

At the reviewed proposal tier, AI prepares work for an accountable workflow. It may draft a support recommendation, classify a request, assemble evidence, or suggest a route. The output can influence a decision, but it cannot accept the decision. Evidence, review, permissions, and logging become mandatory.

At the bounded action tier, the system can mutate workflow state. It might update a ticket status, request a finance review, send an approved message, or apply a credit under a threshold. This tier requires stricter

controls because the system is no longer only producing language. It is causing effects.

For the support workflow, the tiers might look like this:

Tier	Allowed Work	Not Allowed
Discovery only	Inspect approved sources	Customer commitment, drafting, or restricted data
Draft assistance	Private drafting with non-sensitive context	Customer commitment or restricted data
Reviewed proposal	Assemble packet and recommend action	Approve credit or send final message
Bounded action	Route approved packet, update permitted fields	Exceed threshold or bypass review

This distinction keeps the organization from granting agency by accident. A chat tool connected to the ticket system may feel like an assistant until it can write ticket notes customers or auditors later read. A drafting tool may become a decision tool when its output is automatically copied into a CRM field. A routing helper may become an authority system when downstream teams treat its status as accepted.

The control should follow the effect, not the marketing label.

The Tool Inventory

The operator artifact is a tool inventory with authority tiers.

For the support-credit workflow, an initial inventory might read:

Tool Or Path	Data Class	Allowed Use	Effect Authority	Logging
Chat assistant	Low/internal	Draft and summarize	None	Optional
Official support AI	Ticket/contract	Reviewed proposal	No direct credit	Required
Credit agent	Restricted	Bounded action	Threshold-limited	Required
Private AI tool	Unknown	Not sanctioned	None	Missing

Disposition view:

Tool Or Path	Owner	Deadline	Enforcement Hook	Failure Consequence
Chat assistant	Team lead	Current sprint	Data-class reminder	Remove restricted examples
Official support AI	Workflow owner	Pilot week 1	Required packet and audit log	Proposal cannot be accepted

Tool Or Path	Owner	Deadline	Enforcement Hook	Failure Consequence
Credit agent	Finance/security	Future gate	Disabled until post-action verification exists	No direct credit action
Private AI tool	Security/governance	Pilot week 2	Block restricted data and provide sanctioned draft path	Incident review if restricted data appears

The table is simple, but it does several jobs.

It separates data access from effect authority. A tool may read a ticket without reading a contract, draft a message without sending it, or route an approval without approving the credit. Those distinctions matter because most operational harm does not come from "AI use" in the abstract. It comes from particular data crossing a boundary or a particular effect occurring without acceptance.

It names logging requirements. Personal draft assistance may not need the same audit detail as a credit agent. A tool that touches restricted customer data or changes workflow state needs logs sufficient for review, incident response, and learning. If the organization cannot tell which tool produced a customer message or changed a status field, it cannot govern the workflow after the fact.

It exposes private paths. The private AI tool in the table is not sanctioned for this workflow. The workflow owner should ask why it is being used. If the official path is too slow for safe drafting, the organization should provide a sanctioned drafting path. If people are trying to avoid review for financial decisions, the organization should block the effect and escalate the behavior.

Tool inventory without an operational hook becomes paperwork. The hook is: sanction a low-friction path, retire duplicate tools, block unsanctioned effects, or reduce agency until logging and rollback exist.

Permissions Should Match The Failure State

Permissions are not only about what the system can do when everything goes well. They are about what happens when the system is uncertain, wrong, blocked, or under pressure.

The support workflow needs valid failure states:

```
Missing required evidence: remain proposal-only.
Stale source: block acceptance and request current source.
Authority conflict: escalate to finance or compliance.
Queue over capacity: throttle generation or narrow scope.
```

```
Post-action check failed: reverse permitted action or open incident.
```

Those states should be available to the AI-mediated system and to the humans around it. If the only available tool action is "continue," the system will continue through uncertainty. If the only socially acceptable human action is "approve," the reviewer will absorb the risk.

Permissions should be scoped to those states. A reviewed proposal tool does not need permission to issue credits. A packet assembler does not need permission to send customer commitments. A bounded credit agent, if one exists later, should have a threshold, approved source requirements, logging, and a way to fail closed when evidence or authority is missing.

The AI analogy breaks from ordinary human delegation here. A human employee understands social context, can be disciplined, and remains accountable in ways a model cannot. An AI agent acts through tool permissions. If the permission exists and the workflow does not block the action, the system may produce real effects even though no accountable human has accepted the decision. Accountability stays with the organization.

The permission model should therefore encode the acceptance contract rather than rely on the model to be careful.

Post-Action Verification

Bounded action requires checking after action, not only before action.

Even a well-scoped agent can fail by updating the wrong field, routing to the wrong queue, applying a threshold incorrectly, or acting on stale state. External systems can lag, permissions can be misconfigured, policies can change, and users can supply misleading context. The workflow needs to detect and recover from bad effects.

For a support-credit agent, post-action verification might include:

- confirm the action matched the accepted packet;
- confirm the credit amount stayed within the approved threshold;
- confirm the customer message did not imply unapproved terms;
- confirm the ticket, billing note, and audit log agree;
- sample accepted actions for later review;
- open an incident or rollback path when verification fails.

Post-action verification changes the design conversation. The question is not "Can the model do this?" The question is "Can the workflow detect when the system did the wrong thing, limit the damage, and learn from the failure?"

Some actions are easy to reverse: a ticket label can be corrected, a routing decision can be changed, and a draft message can be deleted before sending. Other actions are harder. A customer commitment, a financial credit, a legal filing, or a production change can create consequences that cannot be cleanly undone.

The harder the reversal, the stronger the boundary should be before action.

Sanctioned Alternatives Reduce Bypass

A policy that only says no can move risk elsewhere.

If support cannot use private AI tools but the official workflow cannot produce a holding response quickly, the pressure remains. People may paste less data into private tools, use personal accounts, copy old language, or ask colleagues for side approvals. The organization will have less visibility, not less need.

The better pattern is to provide sanctioned alternatives at lower authority tiers. For the support case, the sanctioned assistant can help with safe work:

- summarize the customer's request from approved ticket fields;
- draft a neutral response that does not promise credit;
- list missing evidence for the packet;
- prepare a request to finance or compliance;
- explain why the decision is blocked in customer-safe language.

This path gives the team a place to work while the acceptance contract does its job. It also creates logs and feedback. If the same missing evidence appears every week, the workflow can fix the source path. If the same customer-safe holding response is needed repeatedly, the team can approve a standard template. If review debt keeps rising, the sponsor can fund capacity or narrow scope.

Sanctioned alternatives also make enforcement more credible. Blocking private tools for restricted data is easier when teams have a usable approved path, and unofficial credit recommendations are easier to prohibit when the official assistant can still help them prepare the packet.

The Agency-Readiness Test

The move from assistant to bounded agent should be made slowly and explicitly.

Test the move to agency with specific questions:

- What data can the system access?
- What effect can it cause?
- Which evidence is required before that effect?
- Who accepts the decision?
- What happens when evidence is missing?
- How is the action logged?
- How is the action verified afterward?
- What sanctioned lower-authority path exists when the answer is no?

If the workflow cannot answer those questions, it is not ready for agency. It may still be ready for assistance. That distinction lets the organization get useful speed without smuggling authority through a tool connection.

The failure test is also concrete. If teams use unofficial tools because the sanctioned path cannot meet demand, treat the bypass as evidence. If an agent can change customer, financial, legal, security, or production state without checked evidence and post-action verification, reduce its authority. If logs exist only to assign blame after the fact, redesign them so they support learning, audit, and recovery.

AI adoption does not become trustworthy when every assistant becomes an agent. It becomes trustworthy when each workflow knows which actions are still drafts, which are reviewed proposals, and which bounded effects the system is allowed to cause.

Once bounded agents exist, failures will still happen. The organization then has to decide whether failures and near misses are paperwork events or signals that should change evals, controls, and governance.

Chapter 11: Evals, Incidents, And Workflow Governance

Evals, audits, incidents, and governance are controls only when they change what the workflow can accept.

In the support-credit workflow, a sampled audit found a stale policy source in an accepted packet. No customer had complained. No money had moved outside the approved range. The support manager had followed the visible process: an AI-generated service-credit recommendation came in with a summary, a reviewer looked at it, and the case was accepted as a low-risk exception.

The recommendation had used a policy page that looked official but was no longer current. The reviewer had seen the policy language in the packet, but not the source age. The account history was current. The ticket was current. The outage note was current. The one source that controlled the exception rule was stale.

The team could not explain the failure by pointing to one careless person. The AI assistant had retrieved a real document. The reviewer had checked the packet that the screen presented. The support manager had accepted a recommendation that appeared to satisfy the policy. The governance document said current policy should be used, but the workflow had no freshness gate that could stop the packet.

The team could update the policy document, remind reviewers to be careful, and record the incident as handled. Or it could change the workflow that had accepted the bad packet.

The workflow change is the useful response. Evals, audits, incidents, and governance matter when they change what the workflow can accept, stop, escalate, measure, throttle, or learn. When they only record that risk exists, they may satisfy a paperwork need, but they have not made this decision process more controlled.

The support case here remains a composite illustration. It is not proof that this exact failure is typical. It is a useful pressure test because the failure is ordinary enough to recognize: the visible process can be followed, and the process can still accept a packet it should have blocked.

Real case: Waymo flooded-road recall. *In NHTSA recall 26E026, filed in May 2026, an unoccupied Waymo vehicle encountered a flooded higher-speed roadway, detected potentially untraversable water, but proceeded at reduced speed. The weak control was the prior operating scope: the vehicle could still enter a situation the system had not handled well enough. Waymo's Safety Board reviewed the issue and decided to conduct a recall. The interim remedy changed the approved operating scope with weather-related controls and map changes while a final remedy was developed.*

The consequence was a formal recall and a narrower operating envelope while the remedy work continued. The control lesson is governance as an operating loop: incident evidence changed where and when the system could operate, rather than remaining a post-launch report. Treat this flooded-road recall as one date-bound example, not a complete account of every later Waymo recall; a later June 2026 NHTSA recall concerned freeway construction-zone behavior (NHTSA 2026a; NHTSA 2026b; NHTSA 2026c; NHTSA 2026d).

The Near Miss

Operations teams know this pattern from incident review.

After an outage, the weak version of a review finds the person closest to the failure and asks them to try harder. The stronger version asks what the system accepted, what it failed to observe, which alarm did not fire, which runbook was wrong, which test was missing, and which owner can change the condition before the next event. Postmortems, regression tests, monitors, runbooks, and ownership changes are not valuable because they are solemn. They are valuable because they alter the operating system around the work.

The same distinction applies to AI governance.

The stale-policy packet did not call for a new slogan about responsible AI. It called for a workflow change. The packet needed a freshness check for sources that carry authority. The reviewer screen needed to show source age next to source status. The eval set needed a stale-source drill. The audit sample needed to include accepted decisions, not only rejected or escalated ones. The governance owner needed a way to retire the older control that asked reviewers to "confirm policy" without giving them the policy version.

A policy reminder might still be appropriate. But the reminder is not the fix. The fix is the change that prevents the same acceptance failure from passing silently next time.

Evaluate The Workflow, Not Only The Model

Model evals matter. They can show whether a model follows instructions, extracts claims, cites sources, refuses unsafe requests, or handles known test cases. Standards and risk-management guidance increasingly treat evaluation, monitoring, documentation, governance, and risk treatment as lifecycle or management-system concerns rather than one-time launch checks (NIST 2023; NIST 2024; International Organization for Standardization 2023).

But the support-credit failure was not only a model failure.

The model retrieved and summarized a real source. The source was stale. The reviewer packet hid the source age. The acceptance rule did not require a current-source check. The audit did not sample enough accepted decisions to catch the pattern earlier. The queue pressure made reviewers prefer packets that looked complete. Each component contributed a small part of the failure.

An eval that asks only, "Did the model answer correctly?" is too narrow for this class of work. The workflow also needs to ask:

- Did retrieval bring in an authority-bearing source?
- Was the source current for the claim it supported?
- Did the model distinguish proposal, evidence, and accepted decision?
- Did the reviewer see the missing or stale evidence?
- Did the queue give the reviewer enough time to reject?
- Did the accepted packet leave an audit trail that can be sampled later?
- Did bypass paths appear because the official process was too slow?

Model evaluation belongs inside the decision system where the model is used.

Evaluation scope follows effect scope. A discovery only or draft assistance task may need only a model eval because the output has no operational effect. A reviewed proposal also needs source mapping, packet completeness, reviewer calibration, and accepted-decision sampling. A

bounded action adds permission checks, post-action verification, rollback behavior, and incident reporting.

The workflow is the unit of trust assessment: model, sources, tools, permissions, reviewers, incentives, logs, and stop conditions together.

Turn Failures Into Regression Cases

Software teams already understand one useful discipline: when a bug escapes, a good fix often includes a regression test. The test is not a punishment. It is a memory aid. It keeps the system from accepting the same failure after people forget the details.

AI workflows need the same habit.

The stale-policy near miss should become a regression case. The case should include an old policy page, a current policy page, a customer contract with a narrower cap, a ticket that creates pressure, and a draft recommendation that looks plausible if the stale page is accepted. The workflow should pass only if it marks the stale source, blocks acceptance, and routes the packet to the right owner.

Other regression cases come from different failure shapes:

Failure Found	Regression Case
Accepted decision based on stale evidence	A stale-source drill with an old policy and current contract
Escaped unsupported decision	A packet with a fluent recommendation but missing authority
Reviewer overload	A queue simulation where complete packets exceed funded review hours
Bypass use	A case where the official path is slower than a sanctioned draft assistance path
Tool overreach	A bounded agent tries to apply a credit without checked evidence

These cases should not be decorative. They should run before expanding scope, after changing the model, after changing retrieval, after changing source systems, and after materially changing reviewer screens or permission tiers.

The workflow should preserve the lesson the incident paid for without freezing itself around one historical case.

Audit Accepted Decisions And Control Failures

Audits often focus on obvious exceptions: escalations, overrides, rejected packets, or high-value cases. Those are important, but they are not enough.

The near miss lived inside an accepted decision. That is why it mattered. The workflow had converted an AI proposal into a decision with operational authority without noticing that the authority-bearing source was stale.

Accepted-decision sampling asks a simple question: when the workflow said yes, what exactly did it accept?

For the support-credit workflow, a useful audit sample would inspect:

- the claim list in the accepted packet;
- the source used for each authority-bearing claim;
- source version and freshness status;
- reviewer decision and any override;
- queue state when the reviewer accepted the packet;
- whether the customer-facing communication matched the accepted decision;
- whether a bypass path existed for the same case.

The audit should also inspect control failures, not only decision outcomes. If ten packets were blocked for missing billing evidence, that may show the gate is working. It may also show the billing source has no owner or the integration is too slow for the workflow. If reviewers reject many packets for the same missing source, governance has found an operating problem, not merely a model-quality issue.

Governance can protect reviewers here. A reviewer who rejects a packet for missing contract authority should not have to argue from personal caution. The workflow should give them a named stop condition, an escalation path, and a record showing that the rejection followed the acceptance contract.

Good governance changes the politics of saying no. It turns "I am not comfortable" into "this packet lacks the source required for acceptance."

Control Validation Loop

The operator artifact is a control validation loop: a small table that connects signals to review actions and workflow changes.

Signal	Review Action	Workflow Change
Accepted decision based on stale evidence	Stale-source drill	Freshness gate
Rejected incomplete packets	Queue and source review	Source owner fix
Reviewer overload	Capacity check	Throttle or WIP limit
Bypass use	Bypass postmortem	Sanctioned fast path
Escaped unsupported decision	Incident review	Regression case

This table is deliberately plain. Its value is that every row has an operational hook.

The owner of governance maintains the loop. Workflow owners implement changes. Incident records, sampled decisions, eval results, audit logs, reviewer calibration, and bypass reports provide the evidence. The cadence can be monthly in steady state, but high-consequence escapes should trigger immediate review. A control fails when it records risk but changes no acceptance rule, permission boundary, evidence requirement, escalation path, logging behavior, throttle, or retirement decision.

The last item matters. Some controls should be retired.

If a checklist item does not block packets, change routing, inform sampling, affect permission, or help incident review, it may only be adding visible process without a hook. Removing a dead control can feel risky because it reduces visible process. But keeping ineffective controls can hide the fact that the real bottleneck is elsewhere: missing source ownership, reviewer capacity, stale retrieval, or an unauthorized fast path.

Retirement should be cautious, documented, and tied to evidence. The practical question is what the workflow loses if the control disappears.

Governance That Can Say No

One failure mode deserves an explicit test: the governance body approves a tool, records caveats, and then has little practical control over daily use. The safer claim is an operating test: governance is weak for this workflow when it cannot change acceptance, permission, evidence, escalation, logging, throttling, or retirement.

That test keeps the discussion out of slogans.

Governance review has to change the workflow it inspects. Stale policy evidence calls for a source-freshness fix. A six-hour reviewer budget for a twelve-hour queue calls for throttling, narrower scope, or more review capacity. Private AI drafts caused by a slow sanctioned path call for a low-friction draft assistance alternative, not only a warning. A direct-credit agent that applies changes without post-action verification should lose that permission until the missing control exists.

Governance becomes operational when it can alter the path of work.

The most important governance question is therefore not, "Do we have a policy?" It is, "What changes when the policy discovers a failure?"

The Workflow-Change Test

For every eval, audit, incident review, dashboard, policy, or committee, ask one question:

```
What can this artifact change in the workflow?
```

Acceptable answers are concrete: it can block acceptance, change a permission, require evidence, route escalation, add logging, trigger a throttle, create a regression case, or retire a dead control.

If the artifact cannot do any of those things, it may still have legal, communications, or coordination value. But it is not yet governing the work.

The near miss did not require the team to become more solemn about AI. It required the team to make the next stale-source packet harder to accept. That is the difference between governance as a record of concern and governance as part of the workflow.

The pieces now fit into one narrow operating design: a Week 0 setup, a three-week pilot, and a stop/go decision that blocks scaling until the workflow can accept, reject, escalate, throttle, and learn.

Chapter 12: Designing One Trustworthy Workflow

A trustworthy pilot assembles the controls into one narrow workflow.

The worked example uses a recognizable support exception: a strategic customer asked for a service credit after an outage. Support wanted a faster answer, sales wanted the renewal protected, finance owned the credit limit, compliance owned policy interpretation, security wanted customer data to stay inside approved tools, and the AI team wanted to help. The design premise was fixed: a useful AI draft is not the same as an accepted decision.

The team chose one workflow.

Not all support cases. Not all customer communications. Not every finance exception. One narrow decision: should the support team recommend a service credit for this class of outage-related customer request?

The answer would begin as a reviewed proposal only. The AI assistant could draft and prepare the packet. It could not apply a credit, promise approval, or update the customer-facing account record. Acceptance required current policy, the customer contract, the billing ledger, customer-specific outage evidence, and a finance or compliance reviewer when the case crossed a threshold.

The decision acceptance contract now runs through one Week 0 setup and three operating weeks. The support case remains a composite operating illustration. Its value is not that it establishes a universal pattern. Its value is that it forces the workflow to show its evidence, permissions, review capacity, stop conditions, and learning loop before the organization scales the idea.

> ***Real case: Morgan Stanley's internal GPT-4 assistant.*** *Morgan Stanley's public materials describe an internal wealth-management assistant designed to answer from Morgan Stanley content, link to source documents, and keep financial advisors at the center of client relationships. Because the case is company-reported, it should not be treated as independent proof of outcomes. Its value is as a positive control pattern. The workflow narrows the corpus, keeps the tool internal, shows source documents, collects feedback, and preserves professional authority.*

The contrast with the incident cases is that the control is designed before the output reaches clients. The lesson for a trustworthy pilot is to assemble controls around one bounded workflow before treating the assistant as client-facing authority (Morgan Stanley 2023; OpenAI 2024).

Week 0: Set Up The Decision, Not The Program

Week 0 is where many AI programs skip the control work.

The team would rather compare models, choose a platform, draft a governance announcement, or promise a quarterly productivity target. Those tasks may matter later. For this pilot, Week 0 has a different job: define the smallest decision that can be accepted or rejected under explicit rules.

The team writes the first version of the packet:

Field	Pilot State
Decision	Recommend service credit?
Action limit before acceptance	Draft and reviewed proposal only
Source authority	Current policy, customer contract, billing ledger, customer-specific outage evidence
Checker	Finance or compliance reviewer for threshold and conflict cases
Stop condition	Missing source, stale source, contract-policy conflict, or queue over capacity
Owner	Support manager after accepted recommendation

That table is intentionally small. It does not describe the company's AI strategy. It describes what this workflow can accept.

Week 0 also exposes the first source-owner delay. The support knowledge base has a policy page, but compliance says the approved version lives in a controlled document repository. The contract summary in the CRM is useful for triage, but legal says the executed contract and amendments are authoritative. The billing ledger has current credit history, but only finance can confirm the threshold. Security classifies contract and billing fields as restricted, which means the assistant can use them only inside the approved workflow.

The pilot is already doing useful work before the model has answered a single customer question.

The workflow has discovered which sources can support authority-bearing claims, who owns them, and which data boundaries

apply. If those facts remain vague, the AI will inherit the organization's ambiguity and make it faster.

Week 1: Reviewed Proposals Only

In Week 1, the team runs the assistant in reviewed proposal mode.

The AI may summarize the ticket, identify claims, draft a recommendation, and prepare the reviewer packet. It may not update CRM fields, send customer language, or apply a credit. Incomplete packets are auto-rejected before they reach a human reviewer.

That last rule feels harsh until the queue starts moving.

Before the pilot, reviewers spent time reconstructing missing evidence. A support manager would send a polished recommendation and the reviewer would hunt for the contract clause, ask for billing history, check whether the outage actually affected the customer, and wonder which policy version had been used. The labor was real, but it was hidden inside Slack threads and ticket comments.

In Week 1, the packet has to show the claims and sources:

Claim	Required Evidence	Packet Result
Customer requested credit	Current support ticket	Present
Outage affected this customer	Customer-specific impact evidence	Missing in some cases
Credit is allowed	Current policy and contract clause	Present or blocked
Amount is within authority	Billing ledger and finance threshold	Missing in some cases
Customer can be told approval is likely	Accepted recommendation and communication owner	Not permitted until accepted

Incomplete packets do not disappear. They become visible as incomplete. If a packet lacks the contract clause, it returns to source collection. If the policy version is stale, it blocks. If the customer-specific outage evidence is missing, the assistant can draft a request for the missing evidence, but it cannot convert the recommendation into approval.

The AI is still useful. It reduces blank-page work, standardizes packet shape, and helps reviewers see what is missing. But the workflow refuses to buy speed by moving evidence reconstruction into the reviewer's unpaid time.

Week 2: Capacity Becomes A Product Constraint

By Week 2, the worked example has enough volume to confront review capacity.

Use these numbers as scenario assumptions, not as evidence about typical support operations. They are here to show the decision math a leader needs in the meeting. In this example, the workflow expects 47 proposals each week. At the start, about 70 percent arrive with complete packets. The target is 85 percent. Reviewers have six funded hours a week. At the current service time, they can review about 24 proposals well. The queue math is not a moral judgment. It is the operating limit of the workflow.

Field	Pilot Assumption
Proposals expected	47/week
Complete-packet rate	70% at start, target 85%
Reviewer-hours funded	6/week
Max reviewed proposals	24/week
Throttle threshold	Queue above 24 open proposals or review debt above 9 complete packets
Stop/go rule	Continue reviewed proposal only if complete packets rise and sampled audits find no escaped unsupported decisions

This is the moment when "human in the loop" becomes a serious design question.

If the workflow generates 47 plausible recommendations and can review 24 of them well, the loop is underfunded. If the team ignores that fact, reviewers may work unpaid overtime, accept packets too quickly, or push decisions back into informal channels. None of those outcomes should be counted as AI success.

The pilot therefore throttles. It prioritizes cases with complete evidence and clear authority. It auto-rejects incomplete packets with a specific missing source. It escalates contract-policy conflicts. It stops new proposals when review debt crosses the threshold.

The throttle frustrates Sales. That is the point at which governance has to be politically real. The reviewer is not slowing the business because they dislike AI. The workflow has a stated capacity limit, and the stop condition protects the organization from accepting unsupported decisions faster than it can inspect them.

Week 2 also runs a stale-source drill. The assistant is given a real-looking old policy and a current policy with a changed threshold. The workflow passes only if the packet marks the old policy as stale and blocks acceptance. This is not a trick. It is Chapter 11's stale-source near miss turned into a launch drill.

Week 2: The Bypass Check

The same week, the team looks for shadow paths.

In the worked example, they find a pressure pattern to investigate: a few account teams have been using private AI tools to draft customer updates because the official workflow feels slow. Some drafts use copied ticket notes and old templates. The account teams were trying to answer customers before the sanctioned path caught up.

Discipline matters, but the durable response is to design a path people can use.

The team creates a sanctioned draft assistance path for low-risk customer language. Its inputs are approved ticket summaries and public incident language, not contract terms, billing history, or credit thresholds. It cannot state that a credit is approved or likely. When a customer asks about money, the draft path links into the reviewed proposal workflow.

This does three things. It gives teams a fast legal path for low-risk drafting, and it removes one excuse for copying restricted data into private tools. It also preserves the distinction between assistance and authority. The fast path may help people write. It may not make financial decisions.

The bypass shows that the official path was too slow for some work. The team creates a lower-authority path that fits the actual work instead of granting agency prematurely.

Week 3: The Stop/Go Decision

At the end of Week 3, the team does not ask whether the AI was impressive.

It asks what the workflow can now accept.

The review is concrete:

Item	Final State
Accepted	Low-risk credit recommendations with current policy, contract cap, billing ledger, and customer-specific outage evidence
Blocked	Recommendations missing current policy, contract clause, billing evidence, or reviewer capacity
Escalated	Contract-policy conflicts and executive override requests
Shadow path	Private AI draft path replaced by sanctioned low-friction draft assistance path
Automation rejected	Direct credit application deferred until post-action verification, audit, and capacity data improve

That last row is important. The pilot succeeds partly by refusing a tempting automation step.

Applying credits directly would make the demo more exciting. It would also move the workflow from reviewed proposal to bounded action. That requires stronger permissions, post-action verification, audit sampling, rollback behavior, and a clear rule for duplicate or erroneous credits. The team does not yet have enough evidence that those controls work. So direct credit application remains out of scope.

The decision scales authority more slowly than drafting speed.

The Final Case Decision

The customer still needs an answer.

In the final review, the packet has current policy, the executed contract, the billing ledger, and customer-specific impact evidence for a low-risk credit below the finance threshold. Finance accepts the amount. Compliance records that the case fits the current exception rule and does not require an executive override. The support manager owns the customer communication after acceptance.

The customer-safe message is restrained:

```
We reviewed your request against the current service-credit policy, your
contract terms, the incident record, and your account history. Your case
qualifies for a service-credit recommendation under the current exception
process. Our support manager will send the formal credit details after
finance
records the accepted amount.
```

The message stays inside the accepted decision: no stale-policy citation, no implied executive override, and no direct customer send from the AI assistant.

One related case is escalated because the contract and policy conflict. One incomplete case is blocked for missing customer-specific impact evidence. A private draft path is replaced with the sanctioned low-risk drafting assistant. Direct credit application remains rejected for now because post-action verification, duplicate-credit recovery, and accepted-decision sampling are not yet strong enough for bounded action.

This is the payoff the opening lacked. The organization gives the customer an answer, protects the reviewer, absorbs the shadow path, and refuses the tempting automation step that would grant the system more authority than the workflow can support.

The Assumption Ledger

The pilot should leave behind a ledger that a skeptical operator can inspect. Because this is a composite worked example, the right labels are assumption and decision effect, not measured before-and-after result.

Measure	Starting Visibility	Worked-Example Decision Effect
Generated recommendations	Counted as activity	Counted separately from accepted work
Accepted verified output	Unknown	18/week as a worked-example estimate under the 47 x 70% x capacity x 75% assumptions
Review debt	Hidden in Slack and tickets	9 complete packets over funded capacity triggers throttle
Incomplete packets	Reviewer reconstruction work	14/week auto-rejected with missing evidence
Escaped unsupported decisions	Not sampled	No unresolved known escaped unsupported decisions; sampled audit required
Shadow path	Private drafts visible only after leak	Sanctioned draft assistance path plus bypass check
Automation scope	Pressure for direct credit action	Direct credit application deliberately deferred

This ledger changes the executive conversation.

Before the pilot, the visible metric was generated recommendations. That number would reward output volume even if reviewers were overloaded or decisions were unsupported. After the pilot, generated recommendations are still counted, but they are not treated as accepted work. Accepted verified output is smaller. It is also more accurate.

The ledger does not mark the workflow as done. It shows the next work: complete-packet rate has to rise, review capacity is still scarce, source ownership still matters, shadow paths need ongoing checks, and the direct-action agent is deferred rather than forgotten.

The pilot has earned a narrower claim: this reviewed proposal workflow can run for low-risk cases when the packet is complete, source freshness is checked, review debt stays under the threshold, and accepted decisions are sampled.

That is a better claim than a broad promise that AI will remake support.

The Payoff Model

The capacity model says whether the workflow can review the work. It does not say whether the workflow is worth funding. That requires a modest payoff model with assumptions labeled plainly.

For the support-credit workflow, the model can fit in one table:

Field	Worked-Example Assumption	Confidence	Decision Use
Accepted verified output	18/week as a worked-example estimate	Medium	Shows throughput under current review capacity
Value per accepted output	Scenario-specific estimate	Low until measured	Compares benefit with review cost
Reviewer cost	6 funded hours/week	High	Makes the control cost visible
Source-fix cost	Compliance and finance owner time	Medium	Decides whether to repair source integration
Cost of delay	Renewal/customer-risk estimate	Low to medium	Decides whether to fund capacity or narrow scope
Rework avoided	Incomplete packets no longer reconstructed by reviewers	Low until measured	Supports source-packet investment
Expected escaped-error cost	Incident and audit estimate	Low until sampled	Sets stop/go risk tolerance

The table exposes weak numbers instead of hiding them inside a confident ROI claim. The executive decision becomes explicit:

```
Continue reviewed proposals if accepted verified output is valuable enough
to
fund review and source fixes, review debt stays under threshold, and sampled
accepted decisions show no unsupported escapes.
Narrow or stop if value is speculative, review debt grows, source ownership
does not improve, or escaped unsupported decisions appear.
```

This is only a rough financial model, but it prevents the pilot from calling generated volume "ROI."

When The Answer Is No

A trustworthy pilot must be allowed to reject the workflow.

The support-credit example could fail the stop/go review in several legitimate ways. If the current policy source has no owner, the workflow cannot verify the most important claim. If finance can fund only a fraction of the required review time, generated recommendations will create review debt rather than accepted work. If account teams keep moving restricted customer details into private tools, the official workflow has not solved the bypass pressure. If sampled accepted decisions show stale evidence or unsupported customer expectations, the workflow is not ready to scale.

Those findings should change the decision:

Finding	Decision
Source owner missing	Stop or run draft assistance until the source is owned
Review capacity unfunded	Narrow scope, fund review, or stop proposals
Bypass persists	Add a sanctioned lower-authority path or pause rollout
Escaped unsupported decision appears	Stop scaling and turn the case into a regression test
Direct action cannot be verified or reversed	Do not grant bounded action

This is where the book's method is deliberately less exciting than an AI demo. Some workflows are bad first candidates. Some should stay in draft assistance or reviewed proposal only. Some should wait until source systems, review budgets, security boundaries, or recovery paths are repaired. A process that cannot say no has not controlled the decision. It has only added review language to work that is still moving toward launch by default.

The Operating Cadence

A trustworthy pilot needs a cadence plain enough to survive contact with work.

Week 0 establishes intake, task fit, data classes, source owners, what the output may do before acceptance, reviewer commitment, stop conditions, and the initial eval cases. Week 1 runs reviewed proposals only and rejects incomplete packets before review. Week 2 checks queue math, runs

stale-source drills, and looks for bypass. Week 3 makes the stop/go decision and updates the workflow before scaling.

The meetings should be short and evidence-bound:

- a queue review that compares proposal volume with reviewer capacity;
- a packet review that samples accepted, rejected, and escalated decisions;
- an incident review for stale evidence, unsupported acceptance, or bypass;
- office hours for source owners and frontline users;
- an executive review that uses accepted verified output, not generated drafts, as the primary adoption signal.

This cadence is not process for its own sake. Each meeting can change a rule: throttle the queue, narrow scope, repair a source, change a permission, add a regression case, sanction a fast path, or stop the pilot.

If a meeting cannot change the workflow, it should not be part of the operating cadence.

The One-Page Contract

At the end of the pilot, the leader needs one artifact that can go into the next meeting. For the worked example, the filled decision acceptance contract looks like this:

Field	Filled Contract	What Changes When It Fails
Decision	Recommend a service credit for outage-related support exception	If decision changes, rerun task-fit and source map
Mode	Reviewed proposal	No direct credit action or customer promise
What the output may do before acceptance	Draft packet, internal recommendation, missing-evidence request	Actions outside this list are blocked
What the output must not do before acceptance	Apply credit, promise approval, update final account status	Tool permission disabled or escalation required
Source/freshness rules	Current policy, executed contract, billing ledger, customer-specific impact evidence	Missing/stale source moves packet to blocked or needs-evidence
Reviewer roles	Support for customer context, finance for amount, compliance for policy conflict	Claim routes to correct reviewer or escalates
Machine gates	Required fields present, source version shown, data class allowed	Packet cannot enter review queue

Field	Filled Contract	What Changes When It Fails
Queue thresholds	24 reviewed packets/week, review debt threshold 9 complete packets	Throttle, narrow, or fund more review
Audit fields	Claim list, source versions, reviewer, decision state, blocked effects, final customer message	Accepted decision cannot close without log
Bypass response	Sanctioned draft assistance path for low-risk language	Restricted private-tool use triggers incident review
Stop/go rule	Continue only with complete packets improving, review debt controlled, and zero escaped unsupported decisions found in the sampled audit	Narrow, reviewed-only, or stop
Owner and cadence	Workflow sponsor; weekly queue/sample review; Week 3 stop/go	No owner or cadence means no scale decision

The blank version is the Monday template:

Field	Fill This In	What Changes When It Fails
Decision		
Mode	Discovery only / draft assistance / reviewed proposal / bounded action	
What the output may do before acceptance		
What the output must not do before acceptance		
Source/freshness rules		
Reviewer roles		
Machine gates		
Queue thresholds		
Audit fields		
Bypass response		
Stop/go rule		
Owner and cadence		

If a row has no failure consequence, the row is not part of the contract yet. It is a note.

What Scaling Means

At the end of the pilot, scaling should not mean copying the assistant into every adjacent workflow.

It should mean one of four decisions:

- Continue reviewed proposals for the narrow case.
- Narrow the workflow until the evidence and review capacity are adequate.
- Expand only to a similar case with the same source and authority pattern.
- Stop until source ownership, permissions, or review capacity improve.

Direct automation is a separate decision. It should wait until the workflow can verify actions after they happen, recover from mistakes, sample accepted actions, and show that the permission boundary is not broader than the decision requires.

Smallness matters here. A small pilot is not trivial if the decision is real, the sources are messy, reviewers are scarce, and the outcome has consequence. Small enough to learn is different from small enough to ignore.

The Scaling Rule

The first trustworthy AI workflow should be narrow enough to inspect and consequential enough to matter. It should leave behind a contract that names the decision, what the output may do before acceptance, source rules, reviewers, gates, thresholds, audit fields, bypass response, stop/go rule, owner, cadence, and failure consequence.

Everything else is an escalation tier.

The support pilot ends with a useful but restrained result. The AI assistant drafts and assembles reviewed proposals for low-risk service-credit recommendations, exposes missing evidence, reduces reconstruction work, and helps the organization see where source ownership, review capacity, and shadow paths are blocking trust.

The restraint matters: the assistant still cannot apply credits directly, promise approval, make stale evidence current by summarizing it well, or turn review debt into capacity.

That restraint is operating discipline. It lets useful AI work become accepted work while keeping the trust problem visible.

The closing chapter returns to the human assignment underneath the machinery: AI can propose, explore, and draft at a scale people could not. People and institutions still own judgment, accountability, and the workflow that turns probable output into accepted action.

Chapter 13: Working With The Machine

AI outputs are proposals until workflow controls accept them for a permitted effect.

The Live Obligation

The support manager still has to answer the customer. Evidence, permissions, review capacity, evals, governance, and workflow design do not remove that live obligation: a customer is waiting, sales wants to protect the renewal, finance owns the credit limit, compliance owns the policy, and support owns the relationship in the moment when the answer has to be given.

AI can help in that moment by summarizing the ticket, comparing policy language, preparing a reviewer packet, drafting customer language, and surfacing the contract clause that conflicts with the general exception rule. That work is useful before the organization is ready to grant the system any authority.

Usefulness and acceptance are different states.

Proposal Before Authority

The argument is simple enough to state plainly: AI output starts as a proposal. A workflow accepts it for use only after required evidence is checked and a named owner accepts the decision or authorizes the permitted effect.

That sentence is a design constraint, not a slogan for buying less AI or more governance software. A probabilistic system can produce work that is valuable, incomplete, surprising, stale, overconfident, or exactly what the team needed. The workflow has to decide which of those states it is in before the output changes a customer record, releases money, updates code, advises a patient, files a report, or tells a person what the organization has decided.

People learned this lesson before AI.

We do not trust a finance manager's memory of the policy when a payment crosses a threshold. We do not trust a developer's confidence without review, tests, and deployment controls. We do not trust an

incident story merely because it is coherent. We do not trust a compliance packet because it is formatted well. Those controls are imperfect. They can lose their operational hook. They can be starved of time, ignored by executives, or bent by incentives. But the underlying lesson is durable: important work earns trust through evidence, authority, verification, and accountability, not through fluency alone.

> ***Real case: Deloitte Australia report.*** *In 2025, Deloitte Australia agreed to partially refund the Australian government after a published report was found to contain citation and reference errors, including errors in a court case summary. The report was for the Department of Employment and Workplace Relations; public reporting described suspected AI-generated errors, and department correspondence said generative AI tools likely contributed to identified errors in publishing-related tasks.*
>
> *The episode exposes a source-verification lesson for professional delivery: the report carried institutional weight before the citations, references, and case summary had been checked well enough. The report was revised and republished with an added disclosure that Azure OpenAI had been used, and Deloitte agreed to a partial refund while maintaining the report's findings and recommendations.*
>
> *The consequence was a public correction, a financial concession, and a public challenge to the reliability normally expected of a professional report. The lesson matches the book's closing claim: professional review does not make probabilistic output authoritative unless the workflow verifies claims, sources, and citations before delivery (McGuirk 2025; Australian Department of Employment and Workplace Relations 2025a; Australian Department of Employment and Workplace Relations 2025b; Australian Department of Employment and Workplace Relations 2026).*

What AI Changes At Scale

AI changes the mechanics of that older problem.

It changes volume, mixture, reach, and drift. The system can generate more proposals than people can review, combine current records with stale documents and learned patterns in one confident paragraph, act through tools when the workflow grants them too broadly, make bypass easier when the official path is slow, and change behavior when the model, prompt, retrieval system, vendor, or source corpus changes.

What The Output Can Affect

Consequence should set the control level. A private summary of a meeting note does not need the same acceptance contract as a customer credit recommendation. A draft assistance path does not need the same permission boundary as an agent that changes workflow state. The point is to know which kind of work you are doing.

The minimum practical question is:

```
What can this output affect?
```

If it affects only the user's private thinking, the controls can be light: clear data boundaries, user responsibility, and easy access to the source. If it becomes a reviewed proposal, the workflow needs evidence, freshness, reviewer capacity, rejection rights, and audit. If it can take bounded action, the workflow also needs explicit permissions, post-action verification, rollback or correction paths, incident handling, and a learning loop.

The pilot should measure the operating cost. The cost may not be the model. It may be the missing source owner, the stale policy repository, the overloaded reviewer, the executive metric that counts drafts as impact, or a private-tool path that appears because the sanctioned path cannot keep up.

AI adoption exposes the operating system around work. It shows where knowledge was informal, where authority was unclear, where review was underfunded, where security boundaries were assumed rather than designed, and where speed pressure made the unofficial path rational. That evidence is the material from which a trustworthy workflow is built.

Start With One Workflow

Start small.

Pick one workflow and make the contract explicit: the decision, what the output may do before acceptance, the source that can support each important claim, the checker and review capacity, the stop condition, and the owner who accepts the decision after the workflow has done its job. Run the pilot long enough to see stale evidence, incomplete packets, queue pressure, bypass, and at least one contested escalation.

Use those results to decide.

The result may be reviewed proposals, a draft assistance path, a narrower scope, more funded review, source repair, reviewer protection, or

a decision to delay direct-action agents. Each choice grants authority at the pace the workflow can support.

Grant The Next Effect From Evidence

Better models matter. They may make better drafts, catch more inconsistencies, summarize more context, use tools more effectively, and reduce some failure rates. A workflow should take those gains seriously. The argument of this book does not depend on models staying weak.

The opposite is closer to true: capability increases the need for clear authority. A model that writes a poor draft is easy to keep at the edge of the workflow. A model that prepares a strong recommendation, cites plausible sources, routes a packet, and offers to update a record will be invited closer to the decision. The trust boundary moves because the work becomes more useful.

That is when the old shortcut becomes dangerous. The organization sees better output and concludes that the workflow can grant more authority. Sometimes it can. But the evidence has to come from the workflow, not from the impressiveness of a single answer.

The question is not whether the new model sounds better. The question is what changed in the operating record:

- Did accepted verified output increase?
- Did stale-source blocks still fire?
- Did reviewers keep enough time to inspect the packet?
- Did unsupported decisions escape less often?
- Did incident review change a control, not only a prompt?
- Did the lower-authority path reduce private-tool bypass?
- Did the owner after acceptance still have evidence and rejection authority?

If the answer is yes, the workflow may be ready for more authority. If the answer is no, the model improvement is still useful, but the authority boundary has not moved yet.

More agency raises the same question with higher stakes. A draft assistant can be wrong without changing a record. A bounded agent can route, update, notify, retry, schedule, apply, or trigger. Each added effect

needs a matching failure state, audit record, correction path, and owner. The workflow should grant the next effect only when it can observe and recover from that effect.

This discipline is not caution for its own sake. It is how useful systems earn more room. Authority should increase when the workflow can show that it accepts, rejects, escalates, throttles, and learns under real pressure.

That also means authority should sometimes not increase. The right next step may be a better draft assistance path, a source repair, a review budget decision, or a plain refusal to automate a consequential action. The book's promise is not that every AI workflow can be made reliable enough by adding more controls. The promise is that the workflow can expose when trust has been earned and when it has not.

The Work Ahead

The best AI system helps people find the evidence, limits, and consequences sooner.

Working with AI therefore requires a practical kind of ambition: build workflows where machine speed increases useful output without replacing human accountability. Confident text should not replace uncertainty management. A working demo should not become authority before the organization knows how the workflow accepts, rejects, escalates, and learns.

The customer still needs an answer. AI can draft that answer faster. The packet can expose the sources. Stale-source blocking can stop unsafe claims. Named acceptance can show who accepted the recommendation, who authorized the effect, and why.

That is the work ahead: designing the process that lets useful AI become trusted work.

Appendix A: Decision Acceptance Contract

The decision acceptance contract defines when AI output can move from proposal to accepted work.

Minimum Viable Contract

Field	Answer
One decision	What exact decision may be accepted?
Controlling source map	Which source or sources control the highest-risk claims?
Action limit before acceptance	What may the AI output do before acceptance?
One checker	Who or what verifies the controlling claim?
One stop condition	What forces the workflow to block, throttle, or refuse automation?
One owner	Who accepts the decision after the workflow has done its checks?

Everything else is an escalation tier.

Full Contract

```
Workflow:
Authority mode: discovery only / draft assistance / reviewed proposal /
bounded action
What the output may do before acceptance:
What the output must not do before acceptance:

Authority-bearing source map:
Required reviewer:
Reviewer capacity:
Accepted state:
Blocked state:
Escalation path:
Stop condition:
Audit record:
Owner after acceptance:

Failure states:
- Missing source:
- Stale source:
- Source conflict:
- Review capacity exceeded:
- Required owner unavailable:
- Unsupported action requested:
```

The contract is useful only if its fields can change workflow behavior. A field that cannot block, route, require, log, throttle, escalate, retire, or stop

work is documentation, not a control.

Appendix B: Claim-To-Evidence Matrix

Use this when a fluent AI-generated proposal bundles several claims into one recommendation.

Claim	Authority-Bearing Source	Freshness Rule	Verification Method	State
				missing / stale / supported / conflicted / verified
				missing / stale / supported / conflicted / verified
				missing / stale / supported / conflicted / verified

Rules:

- Context helps the model understand the situation.
- Evidence supports or rejects a specific claim.
- Freshness decides whether the source is current enough for the decision.
- Verification checks a claim or condition against a standard.
- Acceptance decides whether the output may be used.

In the support-credit workflow, the ticket, policy, contract, billing ledger, outage evidence, and compliance notes answer different claims. A proposal that cites one of them can still be missing the source that controls the decision.

Appendix C: Responsibility And Incentive Map

Use this when someone is named accountable but may not control the evidence, budget, authority, or escalation path.

Role	Owns	Does Not Own	Budget / Capacity	Veto Right	Escalation Cover	Incentive Risk
Support manager	Customer relationship and packet assembly	Finance threshold, legal contract text, policy interpretation				Speed pressure
Finance	Credit limit and financial approval	Customer relationship				Cost pressure
Compliance / legal	Policy interpretation and consistency	Renewal pressure				Late-stage blocker pressure
Source owner	Current source record	Business exception decision				Maintenance underfunding
Executive sponsor	Scope, funding, and stop/go decision	Claim-level verification				ROI pressure

The map is complete only when every named owner has the control needed for the responsibility they carry. If not, the workflow has accountability without authority.

Appendix D: Review-Capacity Scorecard

Use this before increasing AI-generated proposal volume.

Measure	Current Value	Threshold	Action When Threshold Is Crossed
Expected proposals per week			
Complete-packet rate			
Funded reviewer-hours per week			
Average review time per complete packet			
Maximum well-reviewed packets per week			
Review debt			throttle / narrow scope / add capacity
Incomplete packets			auto-reject / return to source collection
Escaped unsupported decisions		0	stop or rollback

Formula:

```
Maximum well-reviewed packets =
  funded reviewer-hours / average review time per complete packet
```

Accepted verified output counts only work that passed the required checks and was accepted for use. Generated volume is activity, not accepted work.

Appendix E: Reviewer Packet Template

Use this when human review is the control.

Packet Field	Purpose
Proposed action	Shows what effect is at stake.
Claim list	Splits the recommendation into checkable statements.
Source link per claim	Lets the reviewer inspect evidence without reconstruction work.
Source existence	Shows whether the source is verified, illustrative, or missing.
Source authority	Shows whether the source is controlling, supporting, or background.
Source date or version	Shows which source version the reviewer is checking.
Source caveat	Shows what the source does not establish.
Freshness status	Shows whether that source version is current enough for the claim.
Missing/conflicted evidence	Makes blocking conditions explicit.
Allowed reviewer actions	Accept, reject, return incomplete, escalate, or block.
Rejection rule	States when rejection cannot be overridden informally.
Audit note	Records what was accepted, rejected, escalated, or missing.

Review is useful only when the reviewer has evidence, time, expertise, independence, and authority to reject or escalate.

Appendix F: Effect-Boundary Specification

Use this before an AI workflow can access tools, communicate outside draft assistance, or move beyond discovery only.

Surface	Discovery Only	Draft Assistance	Reviewed Proposal	Bounded Action
Source access	Approved sources for inspection	Approved low-risk context only	Required sources for claims	Required sources plus action logs
Communication	None outside the user/team	Internal draft only	Internal reviewer packet	Approved customer/system message
Record mutation	None	None	None or draft note	Only permitted fields
Money / credit / entitlement	None	None	Recommendation only	Within threshold after acceptance
Tool use	Read-only or sandboxed	Tools limited to drafting	Route packet / request missing source	Act within explicit permission
Failure state	Stop / mark missing	Ask user / mark missing	Return incomplete / escalate	Stop, rollback, or incident path

The boundary is real only if the workflow can enforce it.

Appendix G: Control-Validation Loop

Use this after launch.

Check	Question	Action
Accepted-decision sampling	Did accepted work have current evidence and proper authority?	Fix source, packet, review, or acceptance rule.
Rejected-decision review	Did the workflow reject useful work for the wrong reason?	Retune source collection or escalation path.
Stale-source drill	Does the workflow block old but plausible evidence?	Repair freshness rule.
Reviewer calibration	Do reviewers apply the same acceptance standard?	Update packet, examples, or training.
Bypass detection	Are teams using private tools or side channels?	Create a sanctioned lower-authority path or fix throughput.
Escaped-error review	What unsupported work reached authority?	Change the workflow, not only the prompt.
Control retirement	Did a control ever change acceptance?	Retire or redesign controls that never block, route, log, throttle, or stop.

Governance is operational when this loop changes what the workflow can accept, escalate, log, throttle, stop, or retire.

Appendix H: Workflow Readiness Preflight

Use this before deciding whether an AI workflow is ready for draft assistance, reviewed proposals, or bounded action.

The preflight is not a maturity score. It is a refusal tool. If the workflow cannot answer these questions, the AI should stay at a lower authority level until the missing owner, source, reviewer, or effect boundary exists.

Minimum Readiness Check

Question	Ready Answer	If Missing
What exact decision or work state is involved?	One named decision, claim, packet, record, or action.	Keep AI in discovery only or draft assistance.
Which source controls the highest-risk claim?	A named current source with an owner.	Do not accept the claim.
What may the output do before acceptance?	A specific action limit.	Treat all actions as prohibited.
What effect is explicitly prohibited?	Customer commitment, record mutation, payment, filing, routing, or other bounded effect.	Do not grant tool or communication authority.
Who checks the controlling claim?	A reviewer, automated check, source owner, or domain owner.	Return incomplete before review.
How much review capacity exists?	Funded reviewer-hours and queue threshold.	Throttle proposal volume.
What condition requires rejection or escalation?	A named condition that blocks or escalates without informal override.	Do not call human review a control.
Who owns the accepted decision?	A person or role with authority after the workflow completes.	Do not promote output to accepted work.
What is logged?	Sources, claims, reviewer action, acceptance state, and effect.	Do not expand authority.
What happens when it fails?	Stop, rollback, correction, escalation, or incident path.	Keep the workflow reversible.

Authority Recommendation

If The Preflight Shows...	Use This Mode
The decision is unclear or source ownership is missing.	No production workflow yet. Use discovery only.
Sources exist but acceptance authority is not assigned.	Draft assistance.

If The Preflight Shows...	Use This Mode
Sources, reviewer, and blocked states exist, but effects are high-risk.	Reviewed proposal.
Sources, review, permissions, rollback, logging, and owner after acceptance exist.	Bounded action.

The One-Sentence Output

The preflight should produce a sentence plain enough to test:

```
This workflow may use AI to [permitted work] but may not [prohibited action]
until [source/check] supports the required claims and [owner] accepts
[decision].
```

For the support-credit workflow:

```
This workflow may use AI to prepare a service-credit packet but may not
imply
customer approval or update the credit record until finance approves the
amount,
compliance/legal clears the policy or contract issue, and the workflow
records
the recommendation as accepted against the current contract, policy, and
outage
evidence.
```

If the sentence cannot be written without vague words such as "appropriate," "aligned," "reviewed," or "responsible," the workflow is not ready for more authority.

Appendix I: Vendor And Tool Evaluation Checklist

Use this when a vendor, platform team, or internal tool owner claims that an AI system is safe, governed, grounded, or production-ready.

Vendor features can help the workflow. They do not replace the workflow. The question is not whether the tool has a control-shaped feature. The question is whether that feature can support the decision, source, permission, review, effect, audit, and recovery rules your workflow requires.

Evaluation Questions

Area	Ask For	What It Supports	What It Does Not Establish
Model evaluation	Task-specific eval design, failure cases, regression history, and pass/fail criteria.	The model was tested against known conditions.	That your workflow can accept outputs safely.
Retrieval and grounding	Source display, citation behavior, freshness controls, and missing-source states.	The tool can expose candidate support for claims.	That a cited source is current, authoritative, or sufficient.
Permissions	Role-based access, tool scopes, data boundaries, and environment separation.	The system can restrict what the AI may see or affect.	That the granted permission matches business authority.
Human review	Reviewer queues, packet fields, reject/escalate actions, and rejection controls.	The tool can route work to people.	That reviewers have time, independence, expertise, or rejection authority.
Audit logs	Prompt, source, output, action, reviewer, acceptance state, and effect logs.	The workflow can reconstruct what happened.	That the logged decision was correct.
Data handling	Retention, training use, deletion, tenant isolation, and restricted-data controls.	The system can satisfy data-boundary requirements.	That the workflow should send that data to the tool.
Incident response	Rollback, correction, alerting, rate limits, disable switches, and escalation hooks.	The tool can support recovery.	That the organization has assigned recovery ownership.
Change man agement	Model/version changes, prompt changes, source-corpus changes, and notification policy.	The workflow can detect some drift.	That old acceptance evidence still applies after change.
Export and portability	Logs, source packets, eval results, configuration, and incident records.	The organization can inspect or migrate evidence.	That the vendor's interpretation of the evidence is enough.

Claims To Treat Carefully

Vendor Claim	Safer Interpretation
"Grounded in your data."	The system can retrieve or reference selected sources; freshness and authority still need workflow rules.
"Human in the loop."	A person appears in the path; review is a control only if the person has evidence, time, and authority to reject or escalate.
"Enterprise-grade governance."	Some governance features exist; verify which workflow states they can block, route, log, throttle, or stop.
"Agentic automation."	The system can act through tools; every action needs an effect boundary, failure state, and owner.
"Proven accuracy."	The model performed under a tested condition; acceptance depends on your claims, sources, reviewers, and what the workflow permits before acceptance.

Minimum Procurement Packet

Ask the vendor or internal platform owner for:

- eval examples that match your workflow;
- known failure modes and mitigations;
- data-retention and training-use terms;
- source freshness and citation behavior;
- permission model and tool scopes;
- audit-log fields and export format;
- human-review workflow and reject/escalate states;
- incident-response and rollback controls;
- model, prompt, and retrieval-change notification process;
- security review artifacts required by your organization.

The packet is complete only when your team can map each feature to a workflow control. A feature that cannot change acceptance, permission, effect, logging, review, recovery, or escalation is helpful context, not a trust boundary.

Appendix J: AI Workflow Antipatterns

Use this table when an AI pilot looks successful but the workflow has not earned the authority it is being given.

Antipattern	What It Looks Like	Why It Fails	Repair
Prompt as policy	The prompt says to follow policy, be accurate, cite sources, or escalate uncertainty.	The prompt improves behavior but does not decide which source is authoritative, current, or sufficient.	Add source ownership, freshness rules, missing-source states, and acceptance criteria.
Human-in-the-loop decoration	A person clicks approve after the output is already shaped, routed, or expected.	Presence is treated as review even when the reviewer lacks time, evidence, independence, or rejection authority.	Give reviewers packets, queue capacity, reject/escalate actions, and rejection authority.
Evals without acceptance rules	The model passes test cases, but live work still reaches authority without required evidence.	The eval measures model behavior, not workflow acceptance.	Connect eval failures to source requirements, permissions, review, throttling, and incident changes.
RAG as freshness	The answer cites retrieved material, so the team treats it as current.	Retrieval can find stale, superseded, incomplete, or non-authoritative material.	Add source-of-truth ownership, version rules, freshness checks, and stale-source blocks.
Generated-volume dashboard	Leadership counts drafts, recommendations, summaries, or tickets touched by AI.	Generated work is not accepted work. The bottleneck may have moved to review.	Track accepted verified output, incomplete packets, review debt, escaped errors, and reviewer-hours.
Named owner without control	A person is accountable for the AI workflow but cannot access sources, fund review, block effects, or reject pressure.	Responsibility becomes a label rather than authority.	Add budget, evidence access, rejection authority, escalation cover, and owner after acceptance.
Pilot success without incident learning	The pilot reports usage and satisfaction, but near misses do not change controls.	The workflow cannot improve from failure.	Review accepted and rejected work, update controls, retire unused checks, and record stop/go decisions.
Tool permission before action boundary	The AI receives access to send, update, apply, route, or trigger before the workflow defines what the output may do before acceptance.	A useful draft can become an action too early.	Define permitted and prohibited actions by mode: discovery only, draft assistance, reviewed proposal, bounded action.
Governance document without runtime hook	The policy says what should happen, but the workflow cannot enforce or observe it.	The rule depends on memory and goodwill under pressure.	Convert the policy into packet fields, permission checks, logs, throttles, escalation paths, and stop conditions.

Antipattern	What It Looks Like	Why It Fails	Repair
Sanctioned path too slow	The official workflow is safe but unusable, so people move to private tools or side channels.	Bypass becomes the real operating model.	Create a lower-authority sanctioned path, narrow scope, fund review, or reduce required effects.

The signal is not that a workflow has an antipattern once. Most pilots will. The signal is whether the team can name the pattern and change the operating state before more authority is granted.

Appendix K: First-Workflow Design Canvas

Use this to design the first pilot.

```
Workflow name:
Decision to support:
Why this workflow:
Authority mode:
What the output may do before acceptance:
What the output must not do before acceptance:

Authority-bearing sources:
- Claim:
  Source:
  Freshness rule:
  Owner:

Required packet:
- Claims:
- Source links:
- Missing-evidence states:
- Reviewer action set:

Review capacity:
- Funded reviewer-hours:
- Expected proposals:
- Average review time:
- Throttle threshold:

Stop conditions:
- Missing source:
- Stale source:
- Contract/policy conflict:
- Review debt:
- Escaped unsupported decision:

Bypass check:
- Likely private-tool path:
- Sanctioned lower-authority alternative:

Pilot schedule:
- Week 0 setup:
- Week 1 reviewed proposals:
- Week 2 capacity and bypass checks:
- Week 3 stop/go decision:

Final decision:
- Continue:
- Narrow:
- Throttle:
- Escalate:
- Stop:
```

The canvas is complete only when the pilot can make a real stop/go decision under pressure.

Appendix L: Authority-Pace Checklist

Use this at the end of a pilot or before expanding scope.

Question	Answer
What useful work can AI do now?	
What authority is still withheld?	
Which source controls the highest-risk claim?	
What is the current review-capacity limit?	
Which stop condition fired during the pilot?	
Which bypass path appeared?	
What control changed acceptance at least once?	
What automation step are we deliberately rejecting for now?	
What evidence would justify the next authority increase?	

Grant authority at the pace the workflow can support.

Appendix M: Worked Support-Credit Packet

This appendix shows how the book's templates fit together for the running support-credit workflow. The example remains a composite design scenario. It is not a documented deployment.

Workflow

```
Workflow:
AI-assisted support-credit recommendation after a service incident

Authority mode:
Reviewed proposal

What the output may do before acceptance:
Internal packet preparation and reviewer routing

What the output must not do before acceptance:
- customer commitment
- final ticket decision
- CRM/account status update implying approval
- credit application
- finance ledger mutation

Accepted state:
Support may recommend the credit to the account team only after current
policy,
executed contract terms, billing history, customer-specific outage evidence,
and required finance/compliance checks are complete.
```

The AI assistant can summarize the support ticket, identify claims, find candidate sources, draft customer language, and prepare the packet. It cannot make the credit decision. It cannot make a customer promise. It cannot update the account record with likely approval.

Support-Credit Minimum Viable Contract

Field	Support-Credit Example
One decision	Should support recommend a service credit for this outage-related customer request?
Controlling source map	Current approved exception policy plus executed customer contract for eligibility.
Action limit before acceptance	Internal reviewed proposal only.
One checker	Finance for credit threshold; compliance/legal for policy or contract conflict.

Field	Support-Credit Example
One stop condition	Missing source, stale source, contract-policy conflict, or review queue above threshold.
One owner	Support manager after the packet is accepted by the required owners.

This is intentionally small. It does not solve all support work. It controls one decision.

Claim-To-Evidence Matrix

Claim	Authority-Bearing Source	Freshness Rule	Verification Method	State
Customer requested a credit	Current support ticket	Current ticket thread	Ticket link inspected	Supported
Outage affected this customer	Customer-specific outage impact evidence	Approved incident impact note or current telemetry	Incident record checked	Missing in some packets
Credit is allowed under policy	Current approved exception policy	Approved current version, not stale KB page	Policy owner/version checked	Supported or stale
Contract permits the remedy	Executed contract and amendments	Current executed agreement	Contract clause inspected	Blocked if unavailable
Credit amount is within authority	Billing ledger and finance threshold	Current account standing	Finance reviewer checks threshold	Missing in some packets
Customer can be told approval is likely	Accepted recommendation and communication owner	Only after acceptance	Account/support owner approval	Not permitted before acceptance

The AI-generated proposal compresses these claims into one recommendation. The packet separates them again so review can inspect the actual decision.

Responsibility And Incentive Map

Role	Owns	Does Not Own	Veto / Escalation
Support manager	Customer relationship, packet assembly, final support recommendation after acceptance	Finance threshold, legal contract meaning, policy ownership	Can return incomplete packet; can escalate source conflict
Finance reviewer	Credit threshold, billing history, financial approval	Customer relationship, incident severity	Can reject or require amount change
Compliance/legal	Policy interpretation, consistency, contract conflict	Renewal pressure	Can block customer commitment until conflict is resolved

Role	Owns	Does Not Own	Veto / Escalation
Incident operations	Outage timeline and customer-specific impact evidence	Credit remedy	Can mark evidence preliminary or approved
Executive sponsor	Pilot scope, funded review capacity, stop/go decision	Claim-level verification	Can fund capacity or narrow scope; should not override missing evidence informally

The support manager is accountable for the customer response only after the workflow gives them usable evidence and records the recommendation as accepted. Naming the manager without source access, rejection authority, or escalation cover does not create a control.

Reviewer Packet

```
Proposed action:
Recommend discretionary service credit.

Allowed reviewer actions:
- accept packet for low-risk recommendation;
- reject as unsupported;
- return incomplete for source collection;
- escalate contract-policy conflict;
- block customer commitment.

Rejection authority:
If the contract clause, current policy, finance threshold, or customer-
specific
outage evidence is missing, the reviewer may reject or return the packet
without informal override through Slack or the account team.

Audit note:
Record accepted, rejected, escalated, or missing evidence state with source
links and reviewer identity.
```

The packet protects the reviewer by making the missing work visible. If the reviewer has to reconstruct the evidence manually, the workflow has moved the cost of AI speed into reviewer time.

Review-Capacity Example

Measure	Pilot Assumption	Threshold / Action
Expected proposals per week	47	Scope must narrow if volume rises without capacity.
Complete-packet rate	70 percent at start	Target 85 percent before expanding.
Funded reviewer-hours per week	6	Capacity is a product constraint.
Average review time	15 minutes	Re-estimate weekly.

Measure	Pilot Assumption	Threshold / Action
Maximum well-reviewed packets	24	Queue above 24 open proposals triggers throttle.
Review debt	9 complete packets over capacity	Stop new proposals or add reviewer time.
Escaped unsupported decisions	0 tolerated	Rollback or stop the pilot.

These are scenario assumptions, not claims about typical support operations. Their job is to show the decision math. If the workflow can generate 47 recommendations and review 24 well, review capacity is the limiting constraint.

Final Pilot Decision

At the end of the three-week pilot, the team should be able to say one of five things:

Decision	Meaning
Continue	Low-risk reviewed proposals meet evidence and capacity thresholds.
Narrow	The workflow is useful only for a smaller class of cases.
Throttle	AI proposal volume exceeds funded review capacity.
Escalate	Source ownership, contract conflicts, or policy authority need executive resolution.
Stop	Unsupported decisions escaped, stale evidence passed, or review capacity failed.

The most important decision may be rejecting direct credit application. That does not mean the pilot failed. It means the workflow learned that bounded action requires stronger permissions, post-action verification, audit sampling, rollback paths, and duplicate-credit safeguards before authority increases.

Appendix N: Pilot Operating Guide

Use this guide to run the first trustworthy AI workflow pilot.

Week 0: Set The Boundary

Do not start by writing an enterprise AI policy. Start by defining one decision and what the output may do before acceptance.

Required outputs:

- workflow name;
- authority mode: discovery only, draft assistance, reviewed proposal, or bounded action;
- decision to support;
- what the output may do before acceptance;
- what the output must not do before acceptance;
- authority-bearing source for the highest-risk claim;
- reviewer or checker;
- stop condition;
- audit record;
- owner after acceptance.

Week 0 should expose source ownership. If nobody can say which source controls the claim, the model cannot fix that ambiguity. It can only make the ambiguity faster.

Week 1: Run Reviewed Proposals

Start with reviewed proposal mode unless the workflow is clearly draft assistance.

The AI may:

- summarize source material;
- identify claims;
- draft a recommendation;
- prepare a reviewer packet;

- request missing evidence;
- draft customer or internal language marked as not accepted.

The AI may not:

- communicate final approval;
- update authority-bearing records;
- apply credits, payments, entitlements, or penalties;
- bypass required owner review;
- convert missing evidence into a confident recommendation.

Incomplete packets should return to source collection before reaching human review. This feels inefficient at first. It is usually less wasteful than making reviewers reconstruct evidence by hand.

Week 2: Measure Capacity And Bypass

By Week 2, measure the work instead of debating impressions.

Track:

- proposals generated;
- complete packets;
- incomplete packets;
- accepted packets;
- rejected packets;
- escalations;
- reviewer-hours used;
- review debt;
- stale-source blocks;
- escaped unsupported decisions;
- bypass attempts or private-tool use.

If generated proposals rise but accepted verified output does not, the workflow has not gained trusted capacity. It has moved the bottleneck.

Look for bypass without treating it only as misconduct. Bypass often means the official path is too slow, too broad, too hard to use, or

mismatched to the actual work. The repair may be a sanctioned lower-authority path, such as a draft assistance path that cannot make commitments or touch restricted data.

Week 3: Decide Under Pressure

The pilot should end with a recorded stop/go decision.

Use these questions:

- Which work can the AI do reliably enough to keep?
- Which authority remains withheld?
- Which source blocked the most packets?
- Which reviewer capacity threshold fired?
- Did any unsupported decision escape?
- Did reviewers have rejection authority?
- Which bypass path appeared?
- Which control actually changed acceptance?
- Which control never changed behavior and should be removed or redesigned?
- What evidence would justify increasing authority?

The decision should be one of:

- continue;
- narrow;
- throttle;
- escalate;
- stop.

The pilot is successful when it produces a truthful operating claim. "We can run reviewed proposals for low-risk cases when packets are complete, sources are current, and review debt stays below the threshold" is a better result than "AI transformed support" with no acceptance evidence.

Stop Conditions

Stop or roll back when:

- stale evidence is accepted;
- a missing authority-bearing source is bypassed;
- customer-facing language implies approval before acceptance;
- review debt exceeds the stated threshold;
- reviewers cannot reject without informal escalation pressure;
- private tools become the real path for restricted data;
- incident review produces only prompt changes when workflow controls failed.

Stopping is not anti-AI. It is how the workflow protects the next accepted use inside a trustworthy workflow.

Appendix O: Term Glossary

Use these terms consistently. The book's argument depends on keeping neighboring states distinct.

Output

Any material produced by the model or AI-assisted workflow. An output is not a decision unless the workflow has accepted it.

Draft

Text or work material that has not entered an acceptance path. A draft may help thinking or preparation. It should not create a customer commitment or system change.

Proposal

An output being considered for use. A proposal may be useful, well structured, and mostly correct while still lacking authority.

Claim

A statement in the proposal that can be checked. A recommendation often bundles several claims, each supported by a different source.

Context

Material the model or reviewer can use to understand the situation. Context is not automatically evidence for a specific claim.

Evidence

Material the workflow can use to support or reject a claim. Evidence should be matched to the claim it supports.

Freshness

Whether a source is current enough for the decision. Freshness is not general quality. An old source may be accurate historically and still stale for the current decision.

Verification

Checking a claim, source, state, or condition against a standard. Verification does not grant permission to act by itself.

Review

Human or automated inspection against a standard. Review becomes useful when it has evidence, time, expertise, independence, and authority to reject.

Approval

Permission granted by a named role, policy, or owner. Approval is stronger than a positive reaction such as "looks good."

Acceptance

The workflow state that allows output to be used. Acceptance is not the same as confidence, style quality, or source citation.

Authority

The right to affect a decision, record, customer, money, system state, or professional filing. Fluency, usefulness, and citations do not create authority by themselves.

Control

A hook that can block, route, require, log, throttle, escalate, retire, or stop work. A reminder, label, policy, or dashboard is not a control unless it changes workflow behavior.

Boundary

A limit on access, communication, mutation, tool use, retry, or what the output may do before acceptance. A boundary written only in prose is a preference, not an enforceable boundary.

Bounded Action

A workflow mode in which an AI-mediated system can cause a defined effect only after required evidence, permission, logging, failure states, and post-action verification are in place.

Bounded Agent

A system granted tool-mediated action authority inside a bounded action workflow. Use this term for the acting system, not for the workflow mode.

Bounded Effect

The specific action or state change a bounded action workflow permits, such as routing an approval, updating a permitted field, sending an approved message, or triggering a reversible operation.

Trustworthy

A workflow condition created by evidence, permission, review, audit, and correction. Do not treat trustworthy as a permanent property of a model or output.

Probabilistic

Behavior involving uncertainty, distributional prediction, judgment, or variable outcomes. Probabilistic does not mean unusable.

Deterministic

Behavior where the same defined input and state produce the same defined output. Deterministic is not a synonym for trustworthy, safe, audited, or controlled.

Bibliography

Arnold, Matthew, Rachel K. E. Bellamy, Michael Hind, Stephanie Houde, Sameep Mehta, Aleksandra Mojsilovic, Ravi Nair, Karthikeyan Natesan Ramamurthy, Darrell Reimer, Alexandra Olteanu, David Piorkowski, Jason Tsay, and Kush R. Varshney. 2019. "FactSheets: Increasing Trust in AI Services through Supplier's Declarations of Conformity." arXiv. https://arxiv.org/abs/1808.07261.

Associated Press. 2023. "General Motors' Autonomous Vehicle Unit Recalls Cars for Software Update After Dragging a Pedestrian." https://apnews.com/article/cruise-general-motors-pedestrian-recall-software-crash-bf08c0c6e7914649750b4dde598af5fc.

Australian Department of Employment and Workplace Relations. 2025a. "Statement from the Secretary on Progress Under Targeted Compliance Framework Integrity Assurance Program." https://www.dewr.gov.au/assuring-integrity-targeted-compliance-framework/announcements/statement-secretary-progress-under-targeted-compliance-framework-integrity-assurance-program.

Australian Department of Employment and Workplace Relations. 2025b. "Correspondence Relating to Targeted Compliance Framework Assurance Review." https://www.dewr.gov.au/download/17418/correspondence-relating-targeted-compliance-framework-assurance-review/41834/correspondence-relating-targeted-compliance-framework-assurance-review/pdf.

Australian Department of Employment and Workplace Relations. 2026. "Targeted Compliance Framework Assurance Review: Final Report." Updated February 3, 2026. https://www.dewr.gov.au/assuring-integrity-targeted-compliance-framework/resources/targeted-compliance-framework-assurance-review-final-report.

Bates and Others v. Post Office Ltd. 2019. Judgment No. 6 "Horizon Issues." [2019] EWHC 3408 (QB). High Court of Justice. https://www.judiciary.uk/wp-content/uploads/2019/12/bates-v-post-office-judgment.pdf.

Board of Governors of the Federal Reserve System, Office of the Comptroller of the Currency, and Federal Deposit Insurance Corporation. 2026. "Supervisory Guidance on Model Risk Management." https://www.federalreserve.gov/frrs/guidance/supervisory-guidance-on-model-risk-management.htm.

Brown, Tom B., Benjamin Mann, Nick Ryder, Melanie Subbiah, Jared Kaplan, Prafulla Dhariwal, Arvind Neelakantan, et al. 2020. "Language Models are Few-Shot Learners." arXiv. https://arxiv.org/abs/2005.14165.

Cahoo v. SAS Analytics Inc. 2019. No. 17-10657, Opinion and Order Denying Motions to Dismiss. U.S. District Court for the Eastern District of Michigan. https://www.mied.uscourts.gov/PDFFIles/17-10657CahooOpnDenyMTD.pdf.

California Department of Motor Vehicles. 2023. "DMV Statement on Cruise LLC Suspension." https://www.dmv.ca.gov/portal/news-and-media/dmv-statement-on-cruise-llc-suspension/.

Christian, Jon. 2023. "CNET Secretly Used AI on Articles That Didn't Disclose That Fact, Staff Say." *Futurism*. https://futurism.com/cnet-ai-articles-label.

Farquhar, Sebastian, Jannik Kossen, Lorenz Kuhn, and Yarin Gal. 2024. "Detecting Hallucinations in Large Language Models Using Semantic Entropy." *Nature* 630: 625-630. https://www.nature.com/articles/s41586-024-07421-0.

Federal Aviation Administration. 2020. "FAA Updates on Boeing 737 MAX." https://www.faa.gov/newsroom/faa-updates-boeing-737-max-0.

Gebru, Timnit, Jamie Morgenstern, Briana Vecchione, Jennifer Wortman Vaughan, Hanna Wallach, Hal Daume III, and Kate Crawford. 2021. "Datasheets for Datasets." *Communications of the ACM* 64 (12): 86-92. https://doi.org/10.1145/3458723.

GAO. 2021. "Artificial Intelligence: An Accountability Framework for Federal Agencies and Other Entities." GAO-21-519SP. https://www.gao.gov/products/gao-21-519sp.

GAO. 2025. "Standards for Internal Control in the Federal Government." https://www.gao.gov/greenbook.

Gurnee, Wes, and Max Tegmark. 2024. "Language Models Represent Space and Time." arXiv. https://arxiv.org/abs/2310.02207.

HDI. 2018. "HDI Support Center Standard, Version 5.3." https://www.thinkhdi.com/~/media/HDICorp/Files/Standards/HDI-SC-Standard-v-5-0.pdf.

House of Commons Education Committee. 2020. "COVID-19: The Cancellation of Exams and 'Calculated' Grades: Government Response to the Committee's First Report." Second Special Report of Session 2019-21. https://committees.parliament.uk/publications/2700/documents/26711/default/.

International Organization for Standardization. 2017. "Customer Contact Centres - Part 1: Requirements for Customer Contact Centres." ISO 18295-1. https://www.iso.org/standard/64739.html.

International Organization for Standardization. 2023. "AI Management Systems: What Businesses Need to Know." https://www.iso.org/artificial-intelligence/ai-management-systems.

Ji, Ziwei, Nayeon Lee, Rita Frieske, Tiezheng Yu, Dan Su, Yan Xu, Etsuko Ishii, Yejin Bang, Delong Chen, Wenliang Dai, Ho Shu Chan, Andrea Madotto, and Pascale Fung. 2023. "Survey of Hallucination in Natural Language Generation." *ACM Computing Surveys* 55 (12), Article 248. https://doi.org/10.1145/3571730.

Joint Authorities Technical Review. 2019. "Boeing 737 MAX Flight Control System: Observations, Findings, and Recommendations." https://www.faa.gov/sites/faa.gov/files/2021-08/Final_JATR_Submittal_to_FAA_Oct_2019.pdf.

Kalai, Adam Tauman, Ofir Nachum, Santosh S. Vempala, and Edwin Zhang. 2025. "Why Language Models Hallucinate." OpenAI-hosted PDF. https://cdn.openai.com/pdf/d04913be-3f6f-4d2b-b283-ff432ef4aaa5/why-language-models-hallucinate.pdf.

Knight, Will. 2023. "An Eating Disorder Chatbot Is Suspended for Giving Harmful Advice." *Wired*. https://www.wired.com/story/tessa-chatbot-suspended.

Lewis, Patrick, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Kuttler, Mike Lewis, Wen-tau Yih, Tim Rocktaschel, Sebastian Riedel, and Douwe Kiela. 2020. "Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks." NeurIPS 2020. https://arxiv.org/abs/2005.11401.

Lifshitz, Lisa R., and Roland Hung. 2024. "BC Tribunal Confirms Companies Remain Liable for Information Provided by AI Chatbot." American Bar Association Business Law Today. https://www.americanbar.org/groups/business_law/resources/business-law-today/2024-february/bc-tribunal-confirms-companies-remain-liable-information-provided-ai-chatbot/.

Mata v. Avianca, Inc. 2023. No. 22-cv-1461 (PKC), Opinion and Order on Sanctions. U.S. District Court for the Southern District of New York. https://storage.courtlistener.com/recap/gov.uscourts.nysd.575368/gov.uscourts.nysd.575368.54.0.pdf.

Maynez, Joshua, Shashi Narayan, Bernd Bohnet, and Ryan McDonald. 2020. "On Faithfulness and Factuality in Abstractive Summarization." Proceedings of ACL 2020, 1906-1919. https://aclanthology.org/2020.acl-main.173/.

McGuirk, Rod. 2025. "Deloitte to Partially Refund Australian Government for Report with Apparent AI-Generated Errors." Associated Press. The report concerned an A$440,000 contract, about US$290,000. https://apnews.com/article/australia-ai-errors-deloitte-ab5485868offc4ae6555b31c8fb987f3.

Michigan Attorney General. 2022. "State of Michigan Announces Settlement of Civil Rights Class Action Alleging False Accusations of Unemployment Fraud." https://www.michigan.gov/ag/news/press-releases/2022/10/20/som-settlement-of-civil-rights-class-action-alleging-false-accusations-of-unemployment-fraud.

Mitchell, Margaret, Simone Wu, Andrew Zaldivar, Parker Barnes, Lucy Vasserman, Ben Hutchinson, Elena Spitzer, Inioluwa Deborah Raji, and Timnit Gebru. 2019. "Model Cards for Model Reporting." FAT* 2019 / arXiv. https://arxiv.org/abs/1810.03993.

Moffatt v. Air Canada. 2024. 2024 BCCRT 149. Civil Resolution Tribunal. https://www.canlii.org/en/bc/bccrt/doc/2024/2024bccrt149/2024bccrt149.html.

Morgan Stanley. 2023. "Morgan Stanley Wealth Management Announces Key Milestone in Innovation Journey with OpenAI." https://www.morganstanley.com/press-releases/key-milestone-in-innovation-journey-with-openai.

NHTSA. 2023. "Part 573 Safety Recall Report 23E-086, Cruise LLC." https://static.nhtsa.gov/odi/rcl/2023/RCLRPT-23E086-7725.PDF.

NHTSA. 2024. "Consent Order: Cruise Standing General Order Reporting." https://www.nhtsa.gov/document/consent-order-cruise-standing-general-order-reporting.

NHTSA. 2026a. "Part 573 Safety Recall Report 26E026, Waymo LLC." https://static.nhtsa.gov/odi/rcl/2026/RCLRPT-26E026-6527.pdf.

NHTSA. 2026b. "Recall Acknowledgment: ADS Software May Allow Vehicle to Drive onto a Flooded Roadway." https://static.nhtsa.gov/odi/rcl/2026/RCAK-26E026-9973.pdf.

NHTSA. 2026c. "Part 573 Safety Recall Report 26E035, Waymo LLC." https://static.nhtsa.gov/odi/rcl/2026/RCLRPT-26E035-7637.pdf.

NHTSA. 2026d. "Recall Acknowledgment: ADS Software May Allow Vehicle to Enter a Closed Freeway Construction Zone." https://static.nhtsa.gov/odi/rcl/2026/RCAK-26E035-6843.pdf.

NIST. 2002. "The Economic Impacts of Inadequate Infrastructure for Software Testing." Planning Report 02-3. https://www.nist.gov/document/report02-3pdf.

NIST. 2023. "Artificial Intelligence Risk Management Framework (AI RMF 1.0)." https://www.nist.gov/itl/ai-risk-management-framework.

NIST. 2024. "Artificial Intelligence Risk Management Framework: Generative Artificial Intelligence Profile." NIST AI 600-1. By Chloe Autio, Reva Schwartz, Jesse Dunietz, Shomik Jain, Martin Stanley, Elham Tabassi, Patrick Hall, and Kamie Roberts. https://www.nist.gov/publications/artificial-intelligence-risk-management-framework-generative-artificial-intelligence.

National Transportation Safety Board. 2019. "Collision Between Vehicle Controlled by Developmental Automated Driving System and Pedestrian, Tempe, Arizona, March 18, 2018." Highway Accident Report NTSB/HAR-19/03. https://www.ntsb.gov/investigations/accidentreports/reports/har1903.pdf.

Ofqual. 2020. "Awarding GCSE, AS & A Levels in Summer 2020: Interim Report." https://www.gov.uk/government/publications/awarding-gcse-as-a-levels-in-summer-2020-interim-report.

OpenAI. 2024. "Morgan Stanley Uses AI Evals to Shape the Future of Financial Services." https://openai.com/index/morgan-stanley/.

Post Office Horizon IT Inquiry. n.d. "Post Office Horizon IT Inquiry." https://www.postofficehorizoninquiry.org.uk/.

Oosting, Jonathan. 2024. "Michiganders Falsely Accused of Jobless Fraud to Share in $20M Settlement." *Bridge Michigan*. https://bridgemi.com/michigan-government/michiganders-falsely-accused-jobless-fraud-share-20m-settlement.

Richardson, Hannah. 2020. "Ofqual Chief Resigns After Exams Chaos." BBC News. https://www.bbc.com/news/education-53907035.

Sato, Mia, and Emma Roth. 2023. "CNET Found Errors in More Than Half of Its AI-Written Stories." *The Verge*. https://www.theverge.com/2023/1/25/23571082/cnet-ai-written-stories-errors-corrections-red-ventures.

Securities and Exchange Commission. 2013a. "SEC Charges Knight Capital With Violations of Market Access Rule." Press Release 2013-222. https://www.sec.gov/newsroom/press-releases/2013-222.

Securities and Exchange Commission. 2013b. "Knight Capital Americas LLC." Administrative Proceeding File No. 3-15570, Release No. 70694. https://www.sec.gov/files/litigation/admin/2013/34-70694.pdf.

Shull, Forrest, Raimund Feldmann, Carolyn Seaman, Myrna Regardie, and Sally Godfrey. 2009. "Fully Employing Software Inspections Data." Preprint, NASA Technical Reports Server. https://ntrs.nasa.gov/citations/20090038685.

The Guardian. 2023. "GM's Cruise Slashes More Than 900 Jobs After Recalling Robotaxis." https://www.theguardian.com/business/2023/dec/14/cruise-layoffs-robotaxi-recall-gm.

University of Bristol Centre for Multilevel Modelling. n.d. "The 2020 GCSE and A-Level 'Exam Grades Fiasco'." https://www.bristol.ac.uk/cmm/research/grade/.

University of Michigan Science, Technology, and Public Policy Program. 2024. "Case Over the Michigan Unemployment Insurance Agency's Faulty MiDAS System Resolved in $20 Million Settlement." https://stpp.fordschool.umich.edu/sites/stpp/files/2024-08/stpp-midas-explainer.pdf.

U.S. House Committee on Transportation and Infrastructure. 2020. "Final Committee Report: The Design, Development & Certification of the Boeing 737 MAX." https://democrats-transportation.house.gov/imo/media/doc/final_boeing_737_max_report1.pdf.

Vinall, Frances. 2023. "Eating-Disorder Group's AI Chatbot Gave Weight Loss Tips, Activist Says." *The Washington Post*. https://www.washingtonpost.com/wellness/2023/06/01/eating-disorder-chatbot-ai-weight-loss/.

Wells, Kate. 2023. "Eating Disorder Helpline Takes Down Chatbot After It Gave Weight Loss Advice." NPR / Michigan Radio. https://www.whro.org/2023-06-08/eating-disorder-helpline-takes-down-chatbot-after-it-gave-weight-loss-advice.

Acknowledgments

This book was shaped by the operating lessons behind real workflows: source ownership, review capacity, permission boundaries, incident learning, and the ordinary pressure to move work forward before every check is complete.

Thanks are also due to the readers and reviewers who test whether a technical argument still works when it reaches the people who have to use it.

About The Author

Benjamin Luchterhand writes about AI adoption, engineering workflow, and the operational controls that make probabilistic systems useful at work. His work centers on a practical question for AI adoption: how can useful probabilistic output become trustworthy work without treating the model as the owner of evidence, authority, or accountability?